

SAINT MARY'S
UNIVERSITY SINCE 1802

One University. One World. Yours.

PROJECT REPORT
MCDA 5585 and MCDA 5586

Submitted By:

Mr. Bhavik Kantilal Bhagat
A00494758

under the guidance of:

Dr. Pawan Lingras
Mr. Kishor Deotale

in partial fulfillment for the awarding of the degree:

Master of Science in
Computing and Data Analytics
September 2026

ACKNOWLEDGMENT

I would like to express my heartfelt appreciation to everyone who contributed to the successful completion of this major project report. Their guidance, support, and encouragement were pivotal to its success.

I am especially grateful to Dr. Pawan Lingras, Academic Supervisor and Director of the Master of Science in Computing and Data Analytics (MCDA) program at Saint Mary's University, for providing me with this invaluable opportunity. His constant support and guidance were crucial throughout the duration of this project.

I extend my deepest gratitude to my manager and industry supervisor, Mr. Kishor Deotale, at Citco Technology Management (CTM). His mentorship gave me the opportunity to explore new technology stack that had not previously been adopted for SRE at Citco—such as AWS-native tooling (CloudWatch, X-Ray, Application Signals) and Grafana as alternatives to Dynatrace—and to help lay the groundwork for a more modern observability approach. This involved assessing the benefits, limitations, and cost implications of each tool to shape a sustainable path forward: one that offers an “eagle-eye” view of the infrastructure, enabling both reactive detection of issues as they arise and proactive identification through warning and critical alerts.

Special thanks to senior CTM team members Mr. Ho Hoi Leung (Mr. Leung) and Ms. Kristia Marie Labos (Ms. Labos) for their hands-on mentorship, technical insights, code review and continuous feedback for improvements. I also express my sincere appreciation to my SMU MCDA colleagues and CTM team members, Mr. Sridhar Vadla and Ms. Soundarya Venkataraman, for their active collaboration and shared commitment to technical excellence.

Finally, I would like to thank both Saint Mary's University and Citco Technology Management (CTM) for giving me this incredible opportunity to gain industry exposure and work on such a sophisticated real-world financial technology project.

CERTIFICATE

This is to certify that Mr. Bhavik Kantilal Bhagat (ID: A00494758), a student at Saint Mary's University in the Master of Science in Computing and Data Analytics program, has completed the projects in fulfilling the course requirements.

.....

Dr. Pawan Lingras

ACADEMIC SUPERVISOR

DEPARTMENT OF MATHEMATICS

AND COMPUTING SCIENCE

SAINT MARY'S UNIVERSITY

923 Robie St, Halifax, NS, Canada B3H 3C3

.....

Mr. Kishor Deotale

INDUSTRY SUPERVISOR / MANAGER

CITCO (CANADA) INC.

Halifax, NS, Canada B3J 1M5

CONTENTS

Acknowledgment	i
Certificate	ii
1 ABOUT CITCO	1
1.1 The Citco Group of Companies	1
1.2 Citco Technology Management — Halifax, Canada	2
2 EXECUTIVE SUMMARY	2
3 TEAM COMPOSITION & RESPONSIBILITIES	4
3.1 Team Structure	4
3.2 My Role and Responsibilities	5
4 TOOLS & TECHNOLOGIES	6
4.1 AWS Cloud Services	7
4.2 Observability & Monitoring	8
4.3 Development Tools	9
4.4 AI-Native Development Tools	10
4.5 RPA Platforms	10
4.6 Communication & Collaboration	11
5 PROJECT OVERVIEW	11
5.1 Project Background and the Motivating Incident	11
5.2 Work Streams Overview	12
6 PROJECT GOALS	14
6.1 Project Goals	14
6.2 Deliverables and Technical Scope	15
6.3 Acceptance Criteria	18
7 LEARNING GOALS	20
7.1 Technical Learning Goals	20
7.2 Non-Technical Learning Goals	22
8 REQUIREMENTS ELICITATION	22
8.1 Stakeholder Identification	23
8.2 Requirements Elicitation	23
8.3 Functional and Non-Functional Requirements	25
9 METHODOLOGIES	28
9.1 SRE Principles	29
9.2 Observability Gap Analysis	31
9.3 Dynatrace Parity Analysis	34
10 IMPLEMENTATION	35
10.1 Platform Overview: Four-Layer Architecture	35
10.2 Work Stream 1: CloudWatch Dashboard Platform	37
10.3 Work Stream 2: Grafana AMG Observability Platform	40
10.4 Work Stream 3: IA Resources Inventory API	49
10.5 Work Stream 4: RPA Production Support	52

10.6	Work Stream 5: RPA LLM Agent — Ask Citco RPA	55
10.7	Work Stream 6: Citco Enterprise Custom UI Dashboards	59
10.8	Design Decisions & Tradeoffs	61
11	WEEKLY IMPLEMENTATION BREAKDOWN	65
11.1	Week 1	65
11.2	Week 2	67
11.3	Week 3	68
11.4	Week 4	70
11.5	Week 5	71
11.6	Week 6	72
11.7	Week 7	74
11.8	Week 8	75
11.9	Week 9	77
11.10	Week 10	79
11.11	Week 11	80
11.12	Week 12	82
11.13	Week 13	84
11.14	Week 14	85
11.15	Week 15	86
11.16	Week 16	88
11.17	Week 17	89
11.18	Week 18	90
11.19	Week 19	91
11.20	Week 20	93
11.21	Week 21	94
11.22	Week 22	96
11.23	Week 23	97
11.24	Week 24	98
11.25	Week 25	101
12	CONCLUSIONS	102
12.1	Learning Goals Evaluation	102
12.2	Co-op Placement Hours Summary	104
13	ACHIEVEMENTS & BUSINESS VALUE	104
13.1	Key Achievements	105
13.2	Business Value	106
14	CHALLENGES & MITIGATIONS	107
14.1	AWS CloudWatch Limitations	107
14.2	Grafana Platform Constraints	108
14.3	Infrastructure & Corporate Policy Challenges	109
14.4	Production Incidents	109
14.5	Summary	111

15 FUTURE WORK	111
15.1 Distributed Tracing with X-Ray	111
15.2 Auto-Healing Agent	112
15.3 RPA Bot CloudWatch Integration	114
15.4 Error Budget Burn-Rate Alerting	114
15.5 Tag Compliance Automation	115
A APPENDIX: WORK LOGS	115
REFERENCES	115

List of Figures

Figure 1: Request Flow Through Monitored Services — Golden Signals	30
Figure 2: IA-SRE Observability Platform — High-Level Architecture	35
Figure 3: AMG Observability Platform — System Architecture	36
Figure 4: CloudWatch Lambda Dashboard — Live Screenshot	38
Figure 5: IAM Role Architecture — Three-Role Model	41
Figure 6: Grafana AMG Dashboard — Health Overview Row	43
Figure 7: Grafana AMG Dashboard — Lambda Monitoring Row	43
Figure 8: Grafana AMG Dashboard — ECS Monitoring Row	44
Figure 9: Alarm Notification Flow — CloudWatch to SNS	47
Figure 10: CI/CD Deployment Pipeline — CodeCommit to AMG	48
Figure 11: AWS X-Ray Service Map — Distributed Trace Topology	49
Figure 12: Inventory API — Grafana AMG Integration	51
Figure 13: Resource Discovery Flow — Two-Stage Filter	52
Figure 14: Ask Citco RPA — RAG Architecture	55

List of Tables

Table 1: Project Deliverables Summary	13
Table 2: Stakeholder Identification	23
Table 3: Functional Requirements	25
Table 4: Non-Functional Requirements	27
Table 5: Observability Gap Analysis — Pre- vs. Post-Co-op	33
Table 6: Dynatrace Capability Parity — AWS-Native Stack vs Dynatrace	34
Table 7: Grafana AMG Dashboard — Row Structure	41
Table 8: IA Resources Inventory API — Endpoint Summary	50
Table 9: Critical Grafana AMG Limitations Identified	60
Table 10: Work Stream 6 — Status Summary	61
Table 11: Learning Goals Overview	102
Table 12: Co-op Placement Hours Summary	104
Table 13: Auto-Healing Agent — Initial Remediation Catalog	113

1. ABOUT CITCO

1.1 The Citco Group of Companies

The Citco Group of Companies (Citco) is the world's largest independent hedge fund administrator, managing over \$1.8 trillion in assets under administration. Founded in 1948 as the *Curaçao International Trust Company* in Curaçao, Netherlands Antilles, the firm now operates from its headquarters in Tortola, British Virgin Islands. Over seven decades, Citco has grown from a civil law notary practice into a global financial services conglomerate with 50+ offices worldwide and approximately 16,000 employees across North America, Europe, Asia-Pacific, and the Caribbean.

Citco remains majority independently owned — a distinction that underpins its client-centric operating philosophy. CEO Christopher Smeets leads a management team that has overseen sustained organic growth, from \$250 billion in administered assets in 2005 to today's \$1.8 trillion benchmark. This growth reflects Citco's pioneering role in shaping the fund administration sector and its continuous investment in proprietary technology platforms.

1.1.1 Core Service Divisions

Citco's global operations are organized across several specialized divisions:

- **Citco Fund Services (CFS) & Investor Relations** — The flagship division and global leader in services for the alternative investment industry. CFS provides full-service fund administration, fund accounting, net asset value (NAV) calculations, investor relations, and reporting for over 2,000 funds. CFS has maintained SOC 1 Type 2 certification for 22 consecutive years and achieved its inaugural SOC 2 Type 2 certification in 2025, underscoring its commitment to operational controls and data security.
- **Citco Governance Services** — Provides corporate and global subsidiary governance, secretarial services, financial accounting and reporting, tax and audit liaison, entity incorporation, and market entry services across multiple jurisdictions.
- **Citco Banking** — Offers clients a comprehensive suite of banking services including investor accounts, operational and deposit bank accounts, foreign exchange, and credit services.
- **Citco Technology Management (CTM)** — The internal technology arm responsible for developing and maintaining Citco's proprietary technology ecosystem, cloud

infrastructure, data platforms, and enterprise applications that power fund administration and middle-office operations globally.

- **Middle Office Solutions** — Encompassing Treasury and Collateral Management, OTC settlements, Trade Matching, and Portfolio Finance. Citco's cloud-based *Æxeo*® Treasury platform provides centralized wire approval and fund movement visibility.

1.1.2 Industry Recognition

Citco consistently receives industry accolades, including the Hedge week European Award for Fund Administrator of the Year (Middle Office Services, 2026), Best Administrator over \$30 billion (APAC Hedge Fund Services Awards, 2026), and Fund Administrator of the Year for GPs over \$30 billion (Private Equity Wire US Awards, 2025). These awards reflect both depth of service and breadth of technological innovation.

1.2 Citco Technology Management — Halifax, Canada

My internship placement is within **Citco Technology Management (CTM)**, the technology division of the Citco Group. The Halifax, Nova Scotia office at 5151 George Street serves as a significant operational center for Citco Canada Inc., occupying multiple floors and functioning as a hub for technology development, fund administration services, and operational support for North American and global clients.

CTM designs, develops, deploys, and maintains Citco's internal technology platforms. This includes enterprise application development, cloud infrastructure management, DevOps and Site Reliability Engineering (SRE), data engineering pipelines, and intelligent automation initiatives. The Halifax CTM team collaborates with Citco colleagues across global offices to deliver technology solutions that underpin the firm's fund administration, investor relations, and governance services.

2. EXECUTIVE SUMMARY

This report documents my Mitacs Business Strategy Internship (BSI) at Citco Technology Management (CTM), Halifax, Nova Scotia, conducted between **March 15 and August 31, 2026**

under the project title *“Intelligent Automation and Reliability Engineering for Financial Services”* (Mitacs reference IT49539).

Before I joined, the IA-IT-SRE team had no centralized observability platform for the IA-IT-SRE Infrastructure — a collection of six platforms (automationhub, bpm, citcoworks, ia-bpo, meridian, rpacfs1) spanning Lambda functions, ECS clusters, EC2 instances, MSK Kafka pipelines, and a range of database and messaging AWS services. Two documented production failures — a 29-day silent outage of the CAIS-Pricing (Citco Alternative Investment Services) GenAI pipeline and a 7.5-hour Meridian event processing delay — both went undetected until business teams escalated them, exposing the team’s reliance on reactive, manual incident detection. The research question framed for this engagement was: *can AWS-native services achieve Dynatrace-equivalent observability at a fraction of the cost?*

My approach was incremental across three phases. I first built CloudWatch dashboards targeting Lambda functions — the AWS service at the center of the silent outage — and documented each function’s trigger type and runtime across all six platforms. I then developed a registry-based IA Infrastructure inventory API that queries the AWS API dynamically and returns live, per-platform lists of all deployed AWS services. Finally, I built a centralized Amazon Managed Grafana (AMG) workspace as a proof-of-concept, covering all AWS service categories used by the IA infrastructure across all six platforms via dynamic platform template variables. The RPA production support rotations ran in parallel throughout the co-op placement.

The key outcomes of this work are:

- Kafka consumer lag detection reduced from a 3.5-hour business-team escalation window to **under 30 seconds** via a CloudWatch MSK EstimatedMaxTimeLag alarm, directly addressing the Meridian Incident failure mode
- Dynamic CloudWatch alarms deployed across all monitored AWS services on all six platforms, where none or limited existed before
- Grafana AMG workspace with **multiple rows** covering every AWS service category in the IA-IT-SRE Infrastructure

- AWS service inventory across all six platforms, with **99% automated test coverage** on the inventory API
- AWS-native stack achieves **~60% Dynatrace feature parity** at \$77/month (CloudWatch + Grafana AMG + X-Ray), rising to **~80%** with Application Signals

The chapters that follow detail the tools and technologies used, the full project context and JIRA scope, the SRE methodology and requirements, the phased technical implementation, and the broader learning outcomes of the engagement.

3. TEAM COMPOSITION & RESPONSIBILITIES

My placement was within the **Intelligent Automation & Site Reliability Engineering (IA-SRE)** team at Citco Technology Management (CTM), Halifax. This chapter describes the team I worked with and the responsibilities I held during the co-op placement.

3.1 Team Structure

The IA-SRE team is co-located across Halifax (Canada) and Hyderabad (India), with stakeholders spanning Dublin and London. My direct supervisor was Mr. Kishor Deotale (Manager, CTM). The co-op period team comprised the following members:

- **Mr. Kishor Deotale** — Manager, CTM and Industry Supervisor. Mr. Deotale is responsible for all work streams, sets engineering priorities, and served as the final acceptance authority for all deliverables.
- **Mr. Ho Hoi Leung (“Mr. Leung”)** — IT Senior Developer, RPA Platform Lead. Mr. Leung is the primary knowledge holder for Blue Prism and UiPath platform operations, responsible for escalation triage in the Follow the Sun support model and the designated Halifax escalation point for production incidents.
- **Ms. Kristia Marie Labos (“Ms. Labos”)** — IT Developer, SRE Engineer. Ms. Labos leads the SRE practice for the team, including the four golden signals framework. She guided the observability platform design and validated alarm threshold selection throughout the dashboard build.

- **Mr. Sridhar Vadla and Ms. Soundarya Venkataraman** — Co-interns (SMU MCDA, Mitacs BSI IT49539). Both joined the Halifax team concurrently, each working on complementary workstreams of the IA monitoring and automation platform, enabling parallel progress across the observability roadmap.

3.2 My Role and Responsibilities

As a participant in the Mitacs Business Strategy Internship (BSI) program, I was embedded within the IA-SRE team at CTM Halifax. I collaborated closely with Mr. Deotale, Mr. Leung, Ms. Labos, co-interns Mr. Vadla and Ms. Venkataraman, and stakeholders across Halifax, Hyderabad, and Dublin. Weekly standups, sprint reviews, and direct Confluence and JIRA-based collaboration formed the day-to-day working rhythm.

My responsibilities spanned the following areas:

- **IA Infrastructure Monitoring:** Developing per-platform CloudWatch dashboards for Lambda, ECS, EC2, SQS, EKS, and related AWS services across all six platforms, establishing the architectural patterns for dynamic resource discovery.
- **SRE Observability Platform Development:** Designing and building the centralized Amazon Managed Grafana (AMG) dashboard covering all AWS services across the six IA-IT-SRE Infrastructure platforms, including CloudFormation alarm provisioning and Infrastructure-as-Code delivery.
- **Inventory API Development:** Building the ia-it-sre-infrastructure-inventory Flask REST API for programmatic discovery of all deployed AWS resources across the IA-IT-SRE Infrastructure, with a discoverer registry pattern that provides a live, per-platform inventory of all AWS resources by service category.
- **Cloud Infrastructure Engineering:** Authoring parameterized CloudFormation and AWS SAM templates, managing IAM roles for cross-account Grafana access, enabling CloudWatch Container Insights on ECS clusters, and navigating AWS service constraints encountered during delivery.
- **Scrum and Meeting Facilitation:** Leading daily standup calls on a rotation basis. Attending dashboard design reviews, knowledge-transfer sessions (RPA platforms, CitcoWorks, Event

Store, AExeo), security and compliance reviews, and stakeholder syncs with the wider CTM team.

- **JIRA Story Creation and Management:** Translating verbal requirements into structured JIRA stories and tasks with clear acceptance criteria and subtask decomposition. Raising defect tickets for issues identified through root cause analysis of production incidents.
- **RPA Production Support:** Providing operational support for Blue Prism and UiPath automation processes during dedicated support rotation weeks (11 AM–7 PM EST shift, rotating weekly among four team members). Responsibilities included monitoring bot queues, triaging service desk tickets, onboarding new clients for Corporate Actions workflows, retriggering failed processes, and escalating where required.
- **Incident Response and Hotfix Engineering:** Investigating and resolving production incidents including the CAIS Deadline credential failure, MESO month-end stuck tasks, and VPL pricing failures through root cause analysis and targeted code or configuration fixes.
- **AI-Augmented Engineering:** Leveraging the Kiro IDE, Amazon Q, and Claude Sonnet for spec-driven development, automated code scaffolding, CloudFormation template generation, and test suite creation.
- **Technical Documentation:** Maintaining Confluence knowledge-base pages for Lambda function inventories, runbooks, incident post-mortems, and architecture decisions.

These responsibilities required a blend of cloud engineering expertise, operational discipline, and cross-functional communication skills to deliver production-grade monitoring infrastructure while maintaining the reliability of live financial automation processes.

4. TOOLS & TECHNOLOGIES

During my co-op placement, the following tools, AWS services, and platforms were used to build the IA-IT-SRE Infrastructure monitoring deliverables, develop the inventory API, and support RPA production operations.

4.1 AWS Cloud Services

- **Amazon CloudWatch** — AWS's primary observability service, used for collecting metrics, logs, and alarms across all monitored AWS services. Metrics Insights SQL and Logs Insights were used for dashboard queries and log analysis.
- **AWS CloudFormation / SAM** — Infrastructure-as-Code framework used to provision CloudWatch alarms, Grafana workspaces, and Lambda functions via parameterized YAML templates.
- **AWS Lambda** — Serverless compute service hosting the automation workloads across all six platforms, and used as the runtime for the CloudFormation alarm provisioner and the resource inventory API.
- **Amazon ECS (Elastic Container Service)** — Container orchestration service used to run the Meridian platform services. CloudWatch Container Insights was enabled to surface task-level CPU and memory telemetry.
- **Amazon MSK (Managed Streaming for Apache Kafka)** — Managed Kafka service used as the Meridian event pipeline backbone. Consumer lag metrics from MSK were the primary observable for the Meridian Incident post-mortem monitoring gap.
- **Databricks** — Unified data analytics platform used within the Meridian event pipeline for ingesting and processing automated system events from Citco Recs. Events are received and processed in Databricks before being consumed downstream by the OpenSearch layer.
- **Amazon SQS (Simple Queue Service)** — Message queue service used as an inter-service trigger mechanism for Lambda functions. Queue depth and oldest message age were monitored as leading indicators of processing backlogs.
- **Amazon EC2** — Virtual machine service hosting RPA bot nodes and supporting infrastructure across the platforms.
- **Amazon EKS (Elastic Kubernetes Service)** — Managed Kubernetes service used by the AutomationHub platform for container workloads.
- **Amazon ECR (Elastic Container Registry)** — Container image registry used to store and version Docker images for ECS and EKS deployments.
- **Amazon OpenSearch Service** — Search and analytics engine used by the Meridian platform as the event indexing layer queried by downstream services.

- **Amazon RDS Aurora** — Relational database service used by Meridian and other platforms for structured data storage.
- **Amazon DocumentDB** — MongoDB-compatible document database used by Meridian for document-oriented workloads.
- **Amazon Neptune** — Graph database used by the CitcoWorks platform for the Knowledge Graph hierarchy.
- **Amazon ElastiCache (Redis/Valkey)** — In-memory caching service used by Meridian's ODL layer for fast event state access.
- **Amazon S3** — Object storage used for CloudFormation template staging, Lambda deployment packages, and automation output artifacts.
- **AWS Secrets Manager** — Credential management service used to store and rotate API keys, database passwords, and service account tokens across all platforms.
- **AWS IAM** — Identity and access management service used to define least-privilege roles for the Grafana data source, Lambda provisioner, and inventory API.
- **AWS CodeCommit** — Git-based source control service used to host all automation and infrastructure repositories.

4.2 Observability & Monitoring

- **Amazon Managed Grafana (AMG)** — Fully managed Grafana service used to build the centralized IA-IT-SRE Infrastructure dashboard with multiple rows covering all monitored AWS services across all six platforms.
- **CloudWatch Container Insights** — Managed telemetry extension for ECS and EKS that surfaces task-level CPU, memory, and network metrics, enabled on the Meridian clusters.
- **AWS X-Ray** — Distributed tracing service researched and partially evaluated for Lambda and ECS instrumentation; full deployment scoped as future work.
- **AWS Application Signals** — AWS-native SLO management and service health monitoring service evaluated as part of the Dynatrace parity analysis; identified as the path from 60% to ~80% AWS-native feature parity.
- **Dynatrace** — Enterprise APM platform already in use by the team for Blue Prism automation observability, used as the functional benchmark for the AWS-native observability PoC.

4.3 Development Tools

- **Python 3** — Primary programming language used for all automation scripts, the inventory API, and CloudFormation provisioner logic.
- **boto3** — AWS SDK for Python, used to interact with all AWS services programmatically in the inventory API and alarm provisioner.
- **Flask (async)** — Lightweight Python web framework used to build the ia-it-sre-infrastructure-inventory REST API with async route handlers.
- **FastAPI** — Python API framework used in the initial inventory API prototype before being migrated to Flask.
- **Pydantic** — Data validation library used for type-safe response models in the inventory API.
- **Pytest + Hypothesis** — Testing frameworks used to build the automated test suite for the inventory API, achieving 99% code coverage with unit, integration, and property-based tests.
- **TypeScript / ReactJS / MUI** — Frontend technology stack used for the inventory dashboard UI, with ReactJS for component-based UI, MUI (Material UI) for design-system components, and TypeScript for type-safe frontend development.
- **Chart.js / Recharts** — JavaScript charting libraries proposed for the custom UI dashboard. Chart.js for real-time streaming metrics with 30-second refresh intervals (canvas-based for smooth rendering); Recharts for static comparisons, bar charts, and gauges.
- **Cytoscape.js** — Graph visualization library proposed for X-Ray service map rendering in the custom UI dashboard, replacing Grafana's crash-prone nodeGraph plugin with a stable custom graph visualization implementation.
- **D3.js (hexbin)** — Data visualization library proposed for Dynatrace-style honeycomb health grids in the custom UI dashboard planning document.
- **Postman** — API testing tool used to validate and document the inventory REST API endpoints during development.
- **Apache JMeter** — Load testing tool used to validate API performance and throughput for the inventory REST API under simulated concurrent request loads.
- **DBeaver** — Universal database client used to inspect and query RDS, DocumentDB, and other database services across the platforms during incident investigation and development.

- **PowerShell / Shell Scripts** — Scripting environments used to automate deployment tasks, run AWS CLI commands, and execute operational scripts on Windows and Linux hosts.
- **Visual Studio Code (VS Code)** — Primary code editor used for Python, TypeScript, and YAML development, extended with the Kiro IDE plugin for AI-assisted engineering.
- **JIRA** — Project tracking tool used for sprint planning, story decomposition, and time logging across all co-op deliverables.
- **Confluence** — Internal wiki used for architecture documentation, Lambda function inventory pages, and runbooks.
- **Git / AWS CodeCommit / Bitbucket** — Version control system with two hosted repository services: AWS CodeCommit for Python and IaC repositories, and Bitbucket for UiPath automation repositories.
- **Remote Desktop Connection (RDC)** — Used to connect to remote Windows machines hosting RPA bot nodes and on-premises infrastructure for incident investigation and configuration changes.

4.4 AI-Native Development Tools

- **Kiro IDE** — AI-powered development environment built on VS Code, used for spec-driven development (requirements.md → design.md → tasks.md) and agentic code generation for the inventory API.
- **Amazon Q** — AWS integrated code assistant used for inline boto3 completions and project-level context retrieval.
- **Claude Sonnet & Opus (Anthropic)** — Large language model accessed via Kiro, used for architecture reasoning, CloudFormation template drafting, and test strategy design.

4.5 RPA Platforms

- **Blue Prism** — Legacy RPA platform hosting daily, monthly, and quarterly fund operations automation processes, monitored during production support rotations.
- **UiPath Cloud / On-Premises** — Target RPA platform for new automation development and Blue Prism migrations, operated via UiPath Orchestrator with SQS-triggered job invocations.

4.6 Communication & Collaboration

- **Microsoft Teams** — Primary communication platform used for daily standups, sprint planning meetings, technical discussions with the SRE team, and screensharing during incident investigation and pair debugging sessions.
- **Microsoft Outlook** — Email client used for formal communications, meeting scheduling, calendar management, and receiving production alert notifications from CloudWatch SNS topics.

5. PROJECT OVERVIEW

5.1 Project Background and the Motivating Incident

This section describes the observability gaps that existed before my co-op placement and the two critical incidents that directly motivated the monitoring program I was assigned to build.

5.1.1 Pre-Co-op Placement Monitoring Posture

Before my co-op placement began in mid-March 2026, the IA-SRE team operated without a centralized observability stack. Monitoring relied on an *ad hoc* model: business teams in Dublin, London, and Halifax would notice anomalies in fund processing outputs and raise tickets. While some CloudWatch alarms existed and platform teams had created basic dashboards for individual resources, there was no unified “eagle eye” view where the IA Manager could check the health of everything across all six platforms at a glance. The team lacked proactive notification of degrading conditions — effectively blind to *saturation* and *error rate* signals until they had already caused downstream business impact.

5.1.2 CAIS-Pricing Silent Outage (Jan–Feb 2026)

In early 2026, the GenAI pipeline (a Citco Alternative Investment Services — CAIS Pricing Pipeline) under the `rpacfs1` platform stopped delivering output for nearly a month without a single alert firing. The root cause was gradual exhaustion of an external API token, which caused downstream Lambda invocations to drop to zero while upstream functions kept running, making the pipeline appear healthy. The outage was detected only when business users reported missing pricing data. The post-mortem identified the absence of throughput alarms, SQS queue depth monitoring, and

end-to-end output verification as the root causes of the failure going unnoticed. A sub-task was assigned directly to me as the immediate remediation and became the starting point for the full monitoring program.

5.1.3 The Meridian Incident (Apr 2026)

The Meridian platform processes daily statements, reconciliations, and event notifications across fund administration operations. A surge of automated system events from Citco Recs overwhelmed Meridian's OpenSearch consumption layer, causing significant Kafka consumer lag and halting daily statement processing for several hours. The SRE team received no alert; the Dublin business team escalated hours after the first failure signal. The post-mortem recorded: *"No proactive monitoring/alerting in place on the SRE side for Meridian event volume anomalies."* This became the primary driver for the centralized dashboard project.

5.1.4 From Incident to Program

Both incidents shared the same root cause: no proactive monitoring. The post-mortems established two specific gaps — no Kafka consumer lag alarm, and no centralized visibility across the IA-IT-SRE Infrastructure — that directly motivated the six work streams described in the following section. Following these incidents, the dashboard project was elevated to **Critical** priority, and my co-op placement pivoted from exploratory dashboard research to a full-scale observability platform build.

5.2 Work Streams Overview

The two motivating incidents established a clear engineering mandate: build proactive observability across the entire IA-IT-SRE Infrastructure. Six work streams were pursued, each directly motivated by the findings and limitations of the previous, as summarized in Table 1. Full technical scope and acceptance criteria for each deliverable are documented in Chapter 6.

5.2.1 Work Stream 1: CloudWatch Dashboards and IA Infrastructure Monitoring

Following the CAIS-Pricing Silent Outage, the objective was to build unified CloudWatch dashboards starting with Lambda — the AWS service at the center of that incident. The scope extended incrementally to SQS queues, EC2 instances, ECS clusters, EKS nodes, and database

Table 1: Project Deliverables Summary

Work Stream	Deliverable Scope
WS1	CloudWatch Lambda Monitoring Dashboard and inventory
WS2	Centralized Amazon Managed Grafana Dashboard
WS3	IA Infrastructure Inventory API
WS4	RPA Production Support Incidents
WS5	RPA LLM Agent — Ask Citco RPA
WS6	Citco Enterprise Custom UI Dashboards

services across all six platforms, establishing the tag-based resource discovery patterns that all later work built upon.

A fundamental limitation became apparent: CloudWatch dashboards are siloed per AWS service and do not support cross-service filtering across platforms. This directly motivated Work Stream 2.

5.2.2 Work Stream 2: Grafana AMG Centralized Dashboard

Following the Meridian Incident, the project was elevated to **Critical** priority. The objective was to build a centralized observability platform using Amazon Managed Grafana (AMG) — addressing what CloudWatch dashboards could not: a unified, cross-platform view of the entire IA-IT-SRE Infrastructure in a single workspace. The broader research objective was to evaluate whether this AWS-native stack could achieve Dynatrace-equivalent observability.

5.2.3 Work Stream 3: IA Infrastructure Inventory API

A critical limitation surfaced during the Grafana build: CloudWatch Metric Insights SQL does not support pattern-based filtering on dimension values. To solve this, the Lambda discovery scripts were extended into a full Flask REST API to discover AWS services dynamically across all platforms, enabling the Grafana dashboard variables to be populated from live infrastructure rather than static configuration.

5.2.4 Work Stream 4: RPA Production Support

In parallel with the engineering work streams, dedicated RPA production support rotations were carried out as part of the team's Follow the Sun model. The North America slot (11 AM–7 PM EST) rotated weekly among the four NA-based SRE team members, serving as the primary contact for RPA incidents escalated through the SD Portal, Innovation IT email inbox, IA_INCIDENTS distribution list, and Teams channels.

Each shift began with a review of the Confluence handover log from the Manila shift and a queue scan to identify open items or approaching SLA breaches. Incidents were triaged, investigated, and either resolved or escalated to the development team with documented findings.

5.2.5 Work Stream 5: RPA LLM Agent — Ask Citco RPA

This work stream involved a production quality and reliability improvement program for the **Ask Citco RPA** chatbot — a RAG pipeline using Amazon Bedrock, OpenSearch Serverless, and 11 Lambda functions. The scope covered production bug resolution, code quality improvements, structured logging migration, and X-Ray distributed tracing instrumentation.

5.2.6 Work Stream 6: Citco Enterprise Custom UI Dashboards

While the Grafana AMG dashboard addressed the primary observability gap, it remained constrained by fixed panel types and no drag-and-drop layout. This work stream focused on designing and building a Citco-owned enterprise monitoring UI by extending the existing ia-config-portal application (built by Ms. Venkataraman and Mr. Vadla) with two new dashboard pages.

6. PROJECT GOALS

I set the following engineering objectives at the outset of the co-op placement, guided by the two motivating incidents and the requirements sessions with Mr. Deotale and Ms. Labos. Each objective maps directly to one of the six work streams described in Chapter 5.

6.1 Project Goals

The primary goals of the co-op placement project were as follows:

- **Eliminate reactive-only incident detection.** Replace the dependency on business-team escalation as the primary detection mechanism with proactive CloudWatch alarms that fire within seconds of a threshold being crossed, directly addressing the failure modes exposed by the CAIS-Pricing Silent Outage and the Meridian Incident.
- **Build a centralized observability platform for the IA-IT-SRE Infrastructure.** Provide a single-pane-of-glass view across all six platforms and all AWS service categories used

by the IA infrastructure, enabling the SRE team to assess platform health, drill down to individual AWS services, and respond to incidents without navigating six separate CloudWatch consoles.

- **Automate AWS service discovery and inventory.** Replace manual Confluence-maintained Lambda lists with a programmatic, always-current inventory API that enumerates all deployed AWS resources per platform and exposes them via REST endpoints for use by the dashboard and other downstream tooling.
- **Evaluate AWS-native observability as a Dynatrace alternative.** Conduct a structured feature-parity analysis comparing the AWS-native stack (CloudWatch, Grafana AMG, X-Ray, Application Signals) against Dynatrace capabilities, quantifying the achievable parity percentage and the incremental monthly cost at each tier.
- **Deliver Infrastructure-as-Code for all monitoring components.** Ensure all alarms, dashboards, IAM roles, and supporting infrastructure are provisioned via parameterized CloudFormation templates and AWS SAM, making the platform reproducible, version-controlled, and environment-portable.
- **Establish engineering quality standards.** Produce a comprehensive test suite for the inventory API, document all architectural decisions and limitations, and maintain a Confluence knowledge base for the Lambda inventory, runbooks, and incident post-mortems.

6.2 Deliverables and Technical Scope

This section documents the technical scope and stack for each primary deliverable. Detailed implementation decisions are covered in Chapter 10.

6.2.1 IA Infrastructure Monitoring Platform

Current State: No CloudWatch dashboards existed for any of the six IA platforms. Lambda function health was monitored only through the AWS Console on an ad-hoc basis. No tag-based resource discovery was in place and the Lambda inventory was maintained manually on Confluence.

Scope: Build unified CloudWatch dashboards covering all six platforms — automationhub, bpm, citcoworks, ia-bpo, meridian, and rpacfs1 — across Lambda, EC2, ECS, EKS, SQS, and supporting

AWS services. Implement a tag-based resource discovery pipeline using `budget_code` as the primary filter and `AppName` as the mandatory fallback discriminator for the meridian/citcoworks shared `budget_code` ambiguity.

Deliverables: CloudWatch dashboards for all six platforms; a resilient three-tier SQS registry pipeline (live API → S3 cache → bundled inventory); a Python-based dashboard builder framework deployed via CloudFormation.

6.2.2 Centralized Grafana SRE Dashboard

Current State: No centralized observability workspace existed. Platform health required navigating six separate CloudWatch consoles. No Kafka consumer lag alarm existed for the Meridian platform, which was the direct cause of the April 2026 incident going undetected for over three hours.

Scope: Build a centralized Amazon Managed Grafana (AMG) workspace providing a unified cross-platform view across all six platforms and all monitored AWS service categories. Implement proactive CloudWatch alarms across all services, an SNS notification pipeline to the SRE team, and Grafana template variables backed by the Inventory API for dynamic resource filtering.

Deliverables: Grafana AMG workspace with 13+ dashboard rows covering all service categories; CloudWatch alarms deployed across all six platforms; SNS alarm notification pipeline; Infrastructure-as-Code (AWS SAM / CloudFormation) for reproducible deployment.

6.2.3 IA Infrastructure Inventory API

Current State: No programmatic AWS resource inventory existed. Lambda function lists were manually maintained on Confluence and became stale quickly. CloudWatch Metric Insights SQL lacked support for pattern-based dimension filtering, making dynamic Grafana template variable population impractical.

Scope: Build a Flask REST API that dynamically discovers all deployed AWS resources per platform across all six IA platforms. Implement a discoverer registry pattern for extensibility, a two-stage tag filter to resolve the meridian/citcoworks `budget_code` ambiguity, and an S3-backed cache layer to keep enumeration latency within the API Gateway 30-second timeout.

Deliverables: REST endpoints for Lambda, ECS, EC2, SQS, S3, and health resources; per-platform filtering support; S3 cache with CloudWatch metrics publishing; Lambda deployment via API Gateway.

6.2.4 RPA Production Support

Current State: The IA-SRE team operates a Follow the Sun support model with shifts across Manila (APAC), Dublin (EMEA), and Halifax (NA). The NA slot (11 AM–7 PM EST) rotated weekly among the four NA-based SRE team members.

Scope: Cover the North America RPA production support rotation, handling incidents escalated through the SD Portal, IA_INCIDENTS distribution list, and Teams channels. Triage, investigate, and resolve or escalate RPA process failures within the defined SLA, and maintain daily Confluence shift handover logs.

Deliverables: Five completed support rotations (W8, W12, W16, W20, W24); incident triage and resolution for all escalated tickets; Confluence handover logs maintained for each shift day.

6.2.5 RPA LLM Agent — Ask Citco RPA

Current State: The **Ask Citco RPA** chatbot was a production RAG pipeline using Amazon Bedrock (Claude), OpenSearch Serverless (1024-dim FAISS/HNSW vectors), and 11 Lambda functions ingesting from SharePoint, Databricks, and the BPM API. The system had multiple known reliability issues including a recurring KB sync timeout and several production bugs identified via CloudWatch log analysis.

Scope: Resolve production reliability issues, enrich the Knowledge Base, and improve code quality across all Lambda functions per the IA Best Practice checklist. Evaluate and propose a Hybrid Graph-RAG architecture extension using Amazon Neptune.

Deliverables: Production bug fixes deployed; NavChecklist PDFs added to Knowledge Base; modular code structure with structured logging, Pydantic validation, tenacity retry, kill switch, and X-Ray tracing; comprehensive unit test suite; architecture proposal for Graph-RAG extension.

6.2.6 Citco Enterprise Custom UI Dashboards

Current State: The Grafana AMG dashboard provided approximately 60% Dynatrace feature parity but was constrained by fixed panel types, no drag-and-drop layout, and limited custom visualization options. A Citco-owned enterprise UI was identified as the next step to close the remaining observability gap.

Scope: Extend the existing ia-config-portal (React 19/TypeScript, ECS Fargate, built by Ms. Venkataraman and Mr. Vadla) with two new dashboard pages — an Infrastructure Inventory dashboard and an Infrastructure Monitoring dashboard — using a config-driven panel schema and CloudWatch data integration.

Deliverables: Architectural implementation plan covering gap analysis, technology stack decisions, API design, and task breakdown; initial frontend scaffolding with AG Grid resource tables and config-driven panel components; continuing in Sep–Dec 2026 extension.

6.3 Acceptance Criteria

The following criteria defined whether each deliverable met its stated objectives.

6.3.1 IA Infrastructure Monitoring Platform

- **Platform coverage:** CloudWatch dashboards exist for all six platforms covering the key AWS services (Lambda, ECS, EC2, SQS, EKS) with tag-based resource filtering.
- **Resilient SQS monitoring:** The SQS registry pipeline remains functional during AWS service degradation via the three-tier fallback model.

6.3.2 SRE Observability Platform

- **Proactive incident detection:** The team can be alerted to Kafka consumer lag, Lambda error rate spikes, ECS CPU saturation, and SQS queue backlogs before business teams escalate — directly addressing the failure modes exposed by the CAIS-Pricing Silent Outage and the Meridian Incident.
- **Cross-platform visibility:** The SRE team can assess the health of all six platforms and all monitored AWS services from a single Grafana workspace without navigating individual CloudWatch consoles.

- **Alarm coverage:** CloudWatch alarms are deployed and active across all monitored AWS services on all six platforms, with SNS notifications routed to the SRE team.
- **Container-level telemetry:** Task-level CPU and memory metrics are available for the Meridian ECS clusters, enabling resource saturation to be detected before service degradation occurs.
- **Reproducible infrastructure:** All monitoring infrastructure — Grafana workspace, alarms, IAM roles, S3 storage — is provisioned as Infrastructure-as-Code and can be redeployed consistently across environments.

6.3.3 IA Infrastructure Inventory API

- **Dynamic discovery:** The API returns a current, accurate list of deployed AWS resources per platform at query time, enabling Grafana template variables to reflect the live fleet without manual updates.
- **Tag disambiguation:** Resources from the meridian and citcoworks platforms are correctly separated despite sharing the same `budget_code` tag value.
- **Extensibility:** New platforms or AWS service types can be onboarded by implementing a single discoverer class with no changes to the API routing or test infrastructure.

6.3.4 RPA Production Support

- **Incident response:** Production incidents are triaged, root-caused, and either resolved within the support shift or escalated with a documented RCA and proposed fix.
- **SLA compliance:** Tickets are acknowledged within the L2 (Innovation IT) response SLA and carried over with complete handover notes per the IA Support Model. L3 (SRE) is escalated for incidents requiring infrastructure access or code-level fixes.

6.3.5 RPA LLM Agent

- **KB sync reliability:** The IRCOEKBExtractLambda completes within the 300-second timeout on every daily run, with steady-state duration under 60 seconds for unchanged content.
- **Production stability:** All identified bugs (OpenSearch silent delete, fire-and-forget pattern, Athena race condition, SSL verification, encryption inconsistency) are resolved and deployed.

- **KB content:** NavChecklist documents are indexed and retrievable via the chatbot.

6.3.6 Citco Enterprise Custom UI Dashboards

- **Architecture plan:** A comprehensive implementation plan covering gap analysis, technology decisions, API design, and task breakdown is produced before implementation begins.
- **Backend API design:** New Lambda handlers designed with request/response schemas and IAM permissions documented.
- **Component specifications:** SRE-specific UI components (honeycomb, service map, trace waterfall) specified with library selections and integration patterns.

7. LEARNING GOALS

The following learning goals were established at the start of my co-op placement in consultation with my academic and industry supervisors. They are organized into technical and non-technical categories to reflect the dual competency development expected of an MCDA industry placement.

7.1 Technical Learning Goals

- **AWS Cloud Services and Infrastructure.** Gain production-level proficiency with the AWS services central to the IA-IT-SRE Infrastructure. During the part-time onboarding phase, I focused on understanding the full AWS service landscape in use at Citco: Amazon CloudWatch (metrics, alarms, Logs Insights, Metric Insights SQL), AWS Lambda (serverless execution, cold-start behavior, concurrency), Amazon ECS and EKS (containerized workloads), Amazon EC2 (virtual machine infrastructure), Amazon MSK (managed Kafka, consumer lag), Amazon SQS (queue-based messaging), Amazon SNS (alerting), Amazon S3, AWS CloudFormation and SAM (Infrastructure-as-Code), AWS CodePipeline (CI/CD), and AWS Secrets Manager (credential management). A deliberate goal was exposure to the full spectrum of database categories deployed at Citco: Amazon DynamoDB (key-value), Amazon DocumentDB (document), Amazon Neptune (graph database — CitcoWorks Knowledge Graph), Amazon RDS Aurora (relational), Amazon ElastiCache for Redis and Valkey (in-memory caching — Meridian ODL tier), and Amazon OpenSearch Service (search and analytics). Additional infrastructure covered includes Amazon ECR, AWS CodeCommit,

and the AWS IAM permission model (including cross-account role chaining for Grafana data source access).

- **Observability, Monitoring, and the Four Golden Signals.** Develop a working understanding of modern observability principles and apply them to a production financial services platform. This goal emerged from discussions with Ms. Labos (SRE mentor) on the four golden signals framework — latency, traffic, errors, and saturation — and their application to the IA platform AWS service fleet. Full details are covered in Chapter 9.

Beyond the four golden signals, learning goals included understanding key observability metrics for each AWS service on the dashboard — MSK consumer lag, SQS queue depth and oldest message age, ECS task CPU and memory, Lambda duration and concurrent executions, EC2 CPU and network, OpenSearch JVM heap pressure, and Redis memory utilization — as well as evaluating advanced observability tooling including AWS X-Ray (distributed tracing), AWS Application Signals (SLO management), Grafana Trace Maps, and improvement opportunities for the existing IA ML/LLM RPA Support Agent.

- **Infrastructure-as-Code and CI/CD.** Author parameterized CloudFormation and AWS SAM templates for alarm and infrastructure deployment; develop a Lambda-backed dynamic alarm provisioner; and understand the AWS CodePipeline CI/CD model used for automation deployments across the platforms.
- **Grafana AMG Platform.** Build and operate a production-grade Amazon Managed Grafana workspace: configure CloudWatch as a data source, design collapsible dashboard rows for each AWS service, implement platform template variables for dynamic filtering, and learn Grafana panel types (time series, stat, bar gauge, heatmap, top-K list, table with deep-link URLs).
- **Backend API Development.** Write production-quality Python APIs using Flask and FastAPI with async route handlers, boto3 for AWS service interaction, Pydantic for response validation, and pytest with Hypothesis for property-based testing.
- **Software Engineering Practices.** Practice spec-driven development via the Kiro IDE workflow (.kiro/specs/ with requirements.md, design.md, tasks.md), and apply AI-augmented engineering using Kiro (Claude Sonnet) and Amazon Q to accelerate design documentation, code scaffolding, and test generation.

- **Proactive vs. Reactive Reliability Engineering.** Transition from a reactive support model (incident detection by business escalation) to a proactive one (alarm-backed detection within 30 seconds) by applying SLI/SLO targets and error budget concepts to real production workloads.

7.2 Non-Technical Learning Goals

- **Scrum and Engineering Leadership.** Lead daily standup calls on rotation, facilitate team coordination, translate verbal requirements from Mr. Deotale into structured JIRA stories with clear acceptance criteria and subtask decomposition, and develop task estimation and prioritization skills in a production engineering environment.
- **Cross-Functional and Cross-Timezone Collaboration.** Build working relationships across Halifax IA-SRE, Innovation IT Hyderabad (IST), Dublin and London business operations teams, and the RPA support tier. Participate in structured knowledge-transfer sessions, design reviews, security and compliance meetings, and stakeholder syncs across time zones.
- **Production Incident Response and RCA.** Develop the discipline of structured root cause analysis: follow the IA Support Model escalation tiers, document findings in SD tickets and Confluence, communicate clearly with business and platform teams under time pressure, and propose fixes or raise JIRA defects with actionable resolution steps.
- **Technical Communication and Documentation.** Produce engineering-grade written deliverables: incident RCA reports, architecture design documents, Confluence knowledge-base pages, runbooks, and this academic report.
- **Financial Domain Knowledge.** Develop working familiarity with fund administration operations — VPL queue (5 PM EST daily deadline), Corporate Actions workflows (Mandatory, Voluntary, Ops Transfer), MESO month-end cycles, and CAIS regulatory filing deadlines — to make informed monitoring and incident response decisions.

8. REQUIREMENTS ELICITATION

During my co-op placement I worked with a small set of internal stakeholders whose needs shaped the functional and non-functional requirements of the IA-IT-SRE Infrastructure monitoring platform.

8.1 Stakeholder Identification

Table 2 enumerates the key stakeholders, their roles, and their specific interests in the project.

Table 2: Stakeholder Identification

Stakeholder	Role	Interest / Influence
Mr. Deotale	Manager, CTM (Industry Supervisor)	Primary sponsor; set engineering priorities; escalated dashboard to Critical priority; final acceptance authority
Mr. Leung	IT Senior Developer, RPA Platform Lead	RPA expert; delivered Blue Prism and UiPath KT; defined RPA monitoring scope
Ms. Labos	IT Developer, SRE Engineer	Delivered golden signals training; reviewed alarm thresholds and dashboard design
IA IT SRE Team	Internal SRE consumers	Primary users of Grafana dashboard and CloudWatch alarm notifications
Meridian Team	Kafka/ECS architecture owners	Provided MSK consumer group structure, ECS cluster topology, OpenSearch integration context
Mr. Vadla & Ms. Venkataraman	Co-interns	Parallel development of monitoring features; design review and test validation

8.2 Requirements Elicitation

The elicitation methods used were appropriate for an in-house engineering context: structured knowledge-transfer sessions, incident post-mortems, and JIRA story refinement, rather than formal user interviews or focus groups.

8.2.1 Elicitation Methods

Knowledge-Transfer Sessions (Training Week, April 13–18, 2026). The first week of full-time engagement was structured as an intensive knowledge-transfer program covering the full breadth of the IA platform. Sessions with Mr. Leung on the Blue Prism and UiPath architectures, with Mr. Bhanu Teja Murari on CitcoWorks and the Event Store platform, and with the Meridian team on the Kafka event pipeline (comprising the Event Routing Service, Action Processor, and OpenSearch consumer) provided the architectural grounding needed to define meaningful SLIs. A key insight from training week: the Æxeo Treasury platform (cloud-based wire approval and fund movement) and the MESO platform (high-memory autoscale issue encountered later) each had distinct resource profiles requiring separate monitoring strategies.

Incident Post-Mortem (Apr 17, 2026). The Meridian production incident yielded the single most important requirements insight of the entire engagement: MSK consumer lag is the primary leading indicator of Meridian health. The post-mortem made concrete the latency between signal onset and detection under the reactive model, and directly specified the target for the lag alarm evaluation period (NFR2: ≤ 1 minute).

Golden Signals Session (May 25, 2026). Ms. Labos' dedicated SRE methodology session translated the theoretical four golden signals into concrete metric choices for the IA platform. This session produced the initial SLI/SLO target table and established the design constraint that all metrics must be queryable through Amazon CloudWatch — ruling out external time-series databases such as Prometheus and anchoring the architecture to CloudWatch Metric Insights as the sole data source.

JIRA Story Refinement. The dashboard project went through multiple refinement cycles between May 6 and May 11 (infrastructure deployment date). These sessions with Mr. Deotale and Ms. Labos surfaced the infrastructure-as-code requirement (FR7), the multi-environment reuse requirement (NFR5), and the credential management constraint (NFR7). The elevation of the dashboard to Critical priority on May 6 also functionally set NFR1 (dashboard refresh latency ≤ 30 seconds) as a hard constraint rather than an aspirational target.

8.2.2 Key Insights

Three insights from the elicitation process materially shaped the resulting design:

- 1. **Tag-based resource discovery is non-trivial at Citco scale.** The training week revealed that the citcoworks and meridian environments share the **Shared Cloud Artifacts** budget_code tag, meaning a simple budget_code lookup conflates resources from two operationally distinct environments. This motivated FR9 (tag-based discovery with AppName fallback).
- 2. **The primary audience for the dashboard is the SRE team, not business operators.** Business operations teams would continue to use their existing domain-specific tools; the dashboard needed to be useful for the SRE first-responder, not designed as a business reporting tool. This shaped the panel selection (technical metrics over business KPIs) and the deep-link design (panels link directly to the relevant AWS Console page).
- 3. **Infrastructure-as-Code is a first-class requirement, not an after-thought.** The team had experienced the cost of manually provisioned monitoring infrastructure in previous projects (configuration drift, undocumented alarms). The JIRA refinement sessions made FR7 an explicit acceptance criterion rather than a **nice-to-have**.

8.3 Functional and Non-Functional Requirements

The following requirements were derived from the elicitation activities described above. They governed the design and acceptance criteria for all six work streams of my co-op placement.

8.3.1 Functional Requirements

Table 3 enumerates the nine functional requirements, each mapped to the primary deliverable that satisfies it. The requirements span three categories: dashboard visualization (FR1–FR4, FR6), infrastructure automation (FR5, FR7), and inventory API capabilities (FR8–FR9).

Table 3: Functional Requirements

ID	Requirement	Satisfied By
FR1	Dashboard displays real-time Lambda metrics (invocations, errors, duration, throttles) per platform	Grafana AMG, Lambda rows

ID	Requirement	Satisfied By
FR2	Dashboard displays ECS CPU and memory utilization per cluster and service	Container Insights + ECS rows (Grafana AMG dashboard and EC2 dashboard)
FR3	Dashboard displays MSK consumer lag per consumer group	MSK rows; EstimatedMaxTimeLag alarm
FR4	Dashboard displays SQS queue depth and oldest message age	SQS rows in Grafana; SQS monitoring
FR5	CloudWatch alarms provisioned for all monitored resource types with SNS routing to SRE team	Dynamic alarms via CloudFormation
FR6	Dashboard supports variable-based filtering by platform and resource	Grafana template variables
FR7	All monitoring infrastructure deployed as Infrastructure-as-Code	4 CloudFormation stacks; Lambda-backed IaC provisioner
FR8	Inventory API enumerates Lambda, ECS, EC2, and S3 resources per platform via REST	Flask REST API, discoverer registry
FR9	Inventory API supports two-stage tag-based discovery for shared budget_code platforms	Two-stage AppName filter in citcoworks/meridian discoverers

8.3.2 Non-Functional Requirements

Table 4 presents the seven non-functional requirements with measurable targets and values achieved. These requirements established quantitative acceptance criteria for performance (NFR1–NFR3), quality (NFR4), maintainability (NFR5–NFR6), and security (NFR7).

Table 4: Non-Functional Requirements

ID	Requirement	Target	Achieved
NFR1	Dashboard refresh enables sub-minute incident detection	≤ 30 sec	≤ 30 sec
NFR2	CloudWatch alarm evaluation period for MSK lag and Lambda error rate	≤ 1 min	1 min
NFR3	Inventory API response time for full resource enumeration	≤ 10 sec	< 10 sec
NFR4	Inventory API automated test coverage	$\geq 95\%$	99%
NFR5	CloudFormation templates parameterized for multi-environment reuse	Parameterized YAML	Achieved
NFR6	Dashboard functional after Grafana version upgrades without manual rework	Standard Plugin queries	Achieved (10.4→12.4)
NFR7	Zero hardcoded credentials — all secrets via AWS Secrets Manager	No plaintext secrets	Achieved

8.3.3 Requirements Traceability

All nine functional requirements and all seven non-functional requirements were satisfied by my delivered work streams. The test suite for the inventory API provides quantitative evidence for NFR3 and NFR4. NFR6 was validated in practice when the Grafana workspace was upgraded from version 10.4 to version 12.4 on May 13, 2026 — a major version migration that required updating CloudWatch Metric Insights query syntax across all panels but did not require any architectural change to the underlying monitoring design.

The most technically demanding requirement was FR9, which exposed a fundamental ambiguity in the AWS tagging strategy for the IA platform. My discovery that citcoworks and meridian share the same `budget_code` tag value (Shared Cloud Artifacts) was not documented anywhere before I built the inventory API. The solution — designating `AppName` as a mandatory secondary discriminator for these two environments — became the standard pattern for all resource discovery going

forward and was documented in the inventory API's README as a platform-wide architectural note.

NFR7 received an unexpected real-world validation during the CAIS Deadline hotfix on June 8, 2026. The incident revealed that the ECS container's credential loading pattern (reading from Secrets Manager once at module import time via constants.py) was fragile under secret rotation. The hotfix I delivered introduced fail-fast credential validation — a design improvement that reinforced NFR7 by ensuring that absent or expired credentials from Secrets Manager produce an immediate, visible failure rather than a silent downstream authentication error.

8.3.4 Out-of-Scope Requirements

The following were evaluated and explicitly descope:

- **Automated error budget burn-rate alerting.** Computing burn rate requires Metric Math expressions that divide the rolling error count by the rolling SLO budget. While the SLO targets and SLI metrics are now in place, configuring burn-rate alarms requires additional CloudFormation work that I descope from the dashboard project.
- **Prometheus as a data source.** A self-managed Prometheus server with a CloudWatch Exporter sidecar was evaluated as an alternative data layer. It was ruled out because Amazon Managed Grafana does not support a self-managed Prometheus endpoint without Amazon Managed Service for Prometheus (AMP), which would add infrastructure cost and operational overhead. CloudWatch Metric Insights SQL provides equivalent multi-resource aggregation capability without the additional stack. Documented as Decision 3 in Section 10.8.3.
- **Pyroscope (continuous profiling).** Pyroscope is a Grafana-ecosystem continuous profiling tool that would surface CPU flame graphs and memory allocation traces at the function level. While it would complement the golden signals dashboard, it requires an instrumentation agent deployed alongside each monitored service. Given the scope of the current engagement and the constraint that the Grafana workspace uses only CloudWatch as its data source, Pyroscope was identified as a post-placement enhancement.

9. METHODOLOGIES

During my co-op placement I adopted Site Reliability Engineering as the primary methodology. I applied the four golden signals framework and SLI/SLO discipline to address the observability gap exposed by the CAIS-Pricing Silent Outage and the Meridian Incident. This chapter covers three areas: the SRE principles I applied, the gap analysis that established the starting posture, and the Dynatrace parity evaluation that framed the research question.

9.1 SRE Principles

Site Reliability Engineering is an operational discipline that applies software engineering thinking to keeping production systems reliable at scale. Its core concepts — golden signals, Service Level Indicators (SLIs), Service Level Objectives (SLOs), and error budgets — provided the theoretical backbone for every design decision I made during this co-op placement.

9.1.1 The Four Golden Signals

I discussed the four golden signals framework with Ms. Labos during a dedicated knowledge-transfer session. These four signals define the minimum instrumentation required to characterize service health:

1. **Latency** — *How long does a request take?* For the IA platform, latency shows as Lambda function duration (P50, P95, P99 percentiles) and ECS service response time. Sustained P99 increases warn of resource saturation before errors appear.
2. **Traffic** — *How much demand is on the system?* Measured as Lambda invocation count, SQS message ingestion rate, and MSK throughput (bytes in/out per second). Traffic anomalies often precede error and saturation signals.
3. **Errors** — *What is the rate of failed requests?* Tracked as Lambda error and throttle counts, ECS task failures, and NumberOfMessagesFailed on SQS queues. An error rate crossing the defined threshold indicates a defect requiring immediate attention.
4. **Saturation** — *How full is the service?* Saturation is the signal most relevant to the April 17, 2026 Meridian incident. It is measured as Lambda concurrent execution count relative to the account concurrency limit, ECS task CPU and memory utilization, MSK consumer lag (the number of messages awaiting processing on a Kafka topic), and SQS

queue depth with oldest-message age. A service approaching saturation degrades before it fails — making this the most actionable of the four signals.

Figure 1 illustrates how a request flows through the monitored services, exposing each of the four golden signals at different stages of the request lifecycle.

The Cast

AWS Component	Restaurant Equivalent	What it does
ALB (Application Load Balancer)	Host / Receptionist	Greets customers at the door, checks their reservation, directs them to the right section
Target Group	A section of tables	All tables assigned to the same team of waiters — customers with similar needs go here
ECS Service	Kitchen staff roster	"I need 3 cooks on shift" — defines how many workers should be running
ECS Task	One individual cook	A single worker doing their job right now
Health Check	Food safety inspection	The host periodically asks each cook "are you okay to serve?"

How a Request Flows

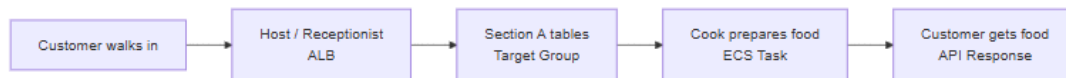


Figure 1: Request Flow Through Monitored Services — Golden Signals at Each Stage

9.1.2 Service Level Indicators and Objectives

A **Service Level Indicator (SLI)** is the specific metric chosen to represent one of the four golden signals for a service — the measurement. A **Service Level Objective (SLO)** is the target threshold set on that measurement — the acceptable limit.

For the IA platform, I selected the following SLIs and set corresponding SLO thresholds, implemented as CloudWatch alarm trigger values:

- **Lambda error rate:** SLI = Errors / Invocations per function. SLO threshold = fewer than 1% of invocations result in an error over any 24-hour window. Implemented as a CloudWatch alarm with Metric Math expression $\text{ErrorRate} = \text{Errors} / \text{Invocations}$.
- **MSK consumer lag:** SLI = EstimatedMaxTimeLag per Meridian consumer group. SLO threshold = lag must remain below **10,000 messages** at any point. Derived from the

April 17 post-mortem: lag reached 9 million messages before detection; a 10,000-message threshold would have fired within minutes of onset.

- **ECS CPU utilization:** SLI = task-level CPU utilization per cluster via Container Insights. SLO threshold = no service sustains above **80%** over a 5-minute evaluation period.
- **SQS queue depth:** SLI = `ApproximateNumberOfMessagesVisible` per queue. SLO threshold = configurable per-queue maximum based on expected processing throughput.

These thresholds were encoded directly as CloudWatch alarm trigger values — each alarm fires when its SLI crosses the SLO boundary and routes an SNS notification to the SRE team. This constitutes **SLO-aligned alerting**: the alarms enforce the objectives but do not compute running error budget consumption or burn rate. This distinction is elaborated in the following subsection.

9.1.3 Error Budgets

An **error budget** is the complement of the SLO: the permitted amount of unreliability before the objective is breached. For example, if the Lambda error rate SLO is below 1% over 24 hours, the error budget is that 1% of invocations permitted to fail.

During this co-op placement I did not implement automated error budget tracking. The Grafana dashboard surfaces current SLI values and the CloudWatch alarms fire at SLO breach — which provides a reactive signal that the budget has been consumed, but does not calculate how quickly it is being drawn down. Full burn-rate alerting (requiring Metric Math expressions dividing rolling error counts against the rolling SLO budget) was identified as a post-placement enhancement.

9.2 Observability Gap Analysis

A prerequisite for building any monitoring platform is understanding the current state of observability. Before I wrote a single line of code for the centralized dashboard, I performed a structured gap analysis to characterize what the IA platform could and could not see about itself.

9.2.1 Pre-Co-op Placement Monitoring Posture

At the commencement of my co-op placement in March 2026, the IA-SRE team's monitoring posture exhibited the following deficiencies:

- **No CloudWatch alarms on any Meridian service.** The Meridian Kafka-backed event pipeline — the most business-critical component of the IA platform — had zero proactive alerting. No alarms existed on MSK consumer lag, ECS task failure counts, Lambda error rates, or SQS queue depth for any Meridian-owned resource.
- **No centralized dashboard.** Monitoring was entirely fragmented. Operators who wished to check the health of a service were required to navigate to that service’s individual CloudWatch console page. There was no eagle-eye view from which to assess the overall health of the platform.
- **Incomplete Container Insights coverage.** Of the six Meridian ECS clusters, only one (enabled in the DEV environment since July 2024) had Amazon CloudWatch Container Insights activated. The remaining five clusters were invisible to task-level CPU and memory telemetry.
- **No MSK consumer lag monitoring.** Despite Kafka consumer lag being the primary leading indicator of Meridian processing health — as demonstrated conclusively by the Meridian Incident — no metric or alarm existed for this signal. The EstimatedMaxTimeLag and SumOffsetLag metrics available natively from Amazon MSK were not surfaced anywhere.
- **No SLO tracking or error budget calculation.** The team had no defined SLIs, no published SLO targets, and therefore no basis for calculating error budgets or determining when the platform was in an unacceptable reliability state.
- **No automated resource inventory.** The inventory of Lambda functions, ECS clusters, and EC2 instances was maintained manually via Confluence pages and spreadsheets. This made it impossible to know, at any given moment, exactly which resources existed across all six platforms.
- **RPA bot monitoring limited to orchestrator UIs.** Blue Prism and UiPath bots were monitored exclusively through their respective orchestrator platforms (Blue Prism Orchestrator and UiPath Cloud). No CloudWatch integration existed, and there was no mechanism for the SRE team to receive proactive notification of bot failures or queue backlogs outside the orchestrator UI.
- **Reactive operational posture.** The cumulative effect of the above gaps was that the team’s primary incident detection mechanism was business-team escalation. When a fund

processing automation failed silently, the first signal received by the SRE team was a ticket or message from operations staff in Dublin, London, or Halifax who had noticed anomalous fund processing outputs. This reactive posture introduced structural latency between failure onset and engineering response.

9.2.2 Gap Analysis Summary

Table 5 summarizes the pre-placement gaps and the post-placement resolutions for each monitoring domain.

Table 5: Observability Gap Analysis — Pre- vs. Post-Co-op Placement

Domain	Pre-Co-op Placement State	Post-Co-op Placement State
CloudWatch Alarms	0 alarms on any IA resource	Dynamic alarms across AWS service categories
Centralized Dashboard Custom UI	None	CloudWatch, Grafana
Container Insights	1 of 6 Meridian clusters	5 of 6 Meridian clusters
MSK Consumer Lag	No monitoring	CloudWatch alarm on EstimatedMaxTimeLag (<30 sec detection)
SLO Tracking	None	SLI/SLO targets defined; dashboard exposes current values
Resource Inventory	Manual (Confluence)	Automated REST API (Lambda, ECS, EC2 across all platforms)
Incident Detection	Business team escalation	Proactive alarm with SNS routing

9.3 Dynatrace Parity Analysis

A key question for this co-op placement is how closely an AWS-native observability stack can replicate the capabilities of Dynatrace, and at what incremental cost. The PoC investigation evaluated 25 Dynatrace features against five AWS tool combinations to quantify achievable parity.

Table 6 summarizes the parity analysis for the ten most operationally relevant Dynatrace capabilities to the IA platform.

Table 6: Dynatrace Capability Parity — AWS-Native Stack vs Dynatrace (selected features)

Dynatrace Feature	CW	+Grafana	+X-Ray	+App Signals	+Tempo
Alerting	✓	✓	—	—	—
Infrastructure Metrics	✓	✓	—	—	—
Log Analytics	✓	✓	—	—	—
Container Monitoring	✓	✓	—	✓	—
Error Rate Tracking	✓	✓	✓	✓	—
Latency Percentiles	—	✓	✓	✓	✓
Distributed Trace Waterfall	—	—	✓	—	✓
Service Map / Topology	—	—	✓	✓	—
SLO Management	—	—	—	✓	—
Dependency Mapping	—	—	✓	✓	—
Parity (% of 25 features)	30%	33%	60%	80%	83%
Monthly Cost (est.)	\$49	\$68	\$77	\$232	\$269

Three capabilities remain **permanently outside the AWS-native reach**: Davis AI causal analysis (proprietary ML, no AWS equivalent); automatic baselining (Dynatrace learns per-service baselines over weeks; AWS alarms require manual thresholds); and session replay (full DOM capture, not available in any OSS tool). These account for approximately 9–10% of the functional gap that no combination of AWS tools can close.

Adopted stack. The PoC I delivered in this co-op placement corresponds to the **CloudWatch + Grafana + X-Ray** column: 60% parity at approximately \$77/month. Application Signals (the path to 80% parity) requires deploying an ADOT sidecar on each ECS service and is scoped as post-placement work. The gap-to-tooling mapping contextualizes what the PoC does and does not cover throughout the Implementation chapter. This methodological foundation—combining golden signals, gap analysis, and parity assessment—established the engineering rationale for every implementation decision.

10. IMPLEMENTATION

10.1 Platform Overview: Four-Layer Architecture

I designed and built three deliverables during my co-op placement that together form a coherent four-layer observability stack. Figure 2 shows how I structured the layers and how they relate to the underlying IA platform infrastructure.

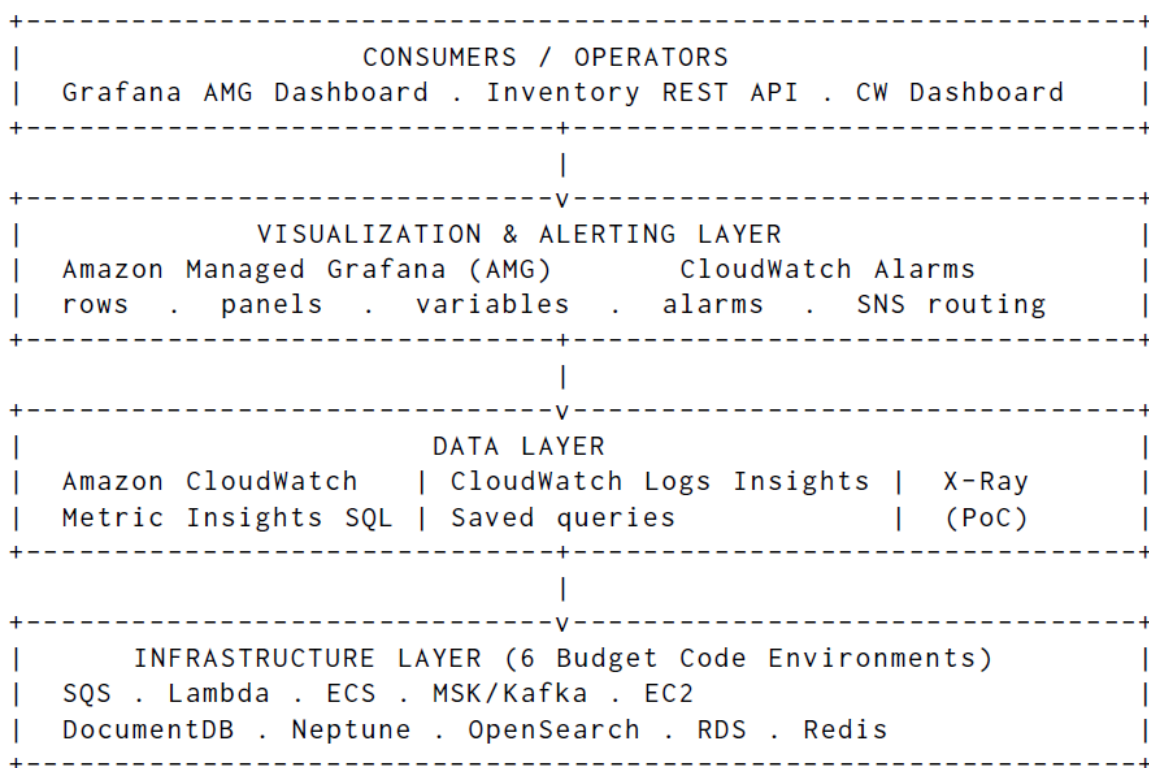


Figure 2: IA-SRE Observability Platform — High-Level Architecture (four layers)

Figure 3 shows the full system architecture of the Grafana AMG observability platform I built, detailing how each monitored AWS service feeds CloudWatch, how the Lambda provisioner I wrote reads alarm templates from S3, and how the SNS routing I configured delivers email notifications.

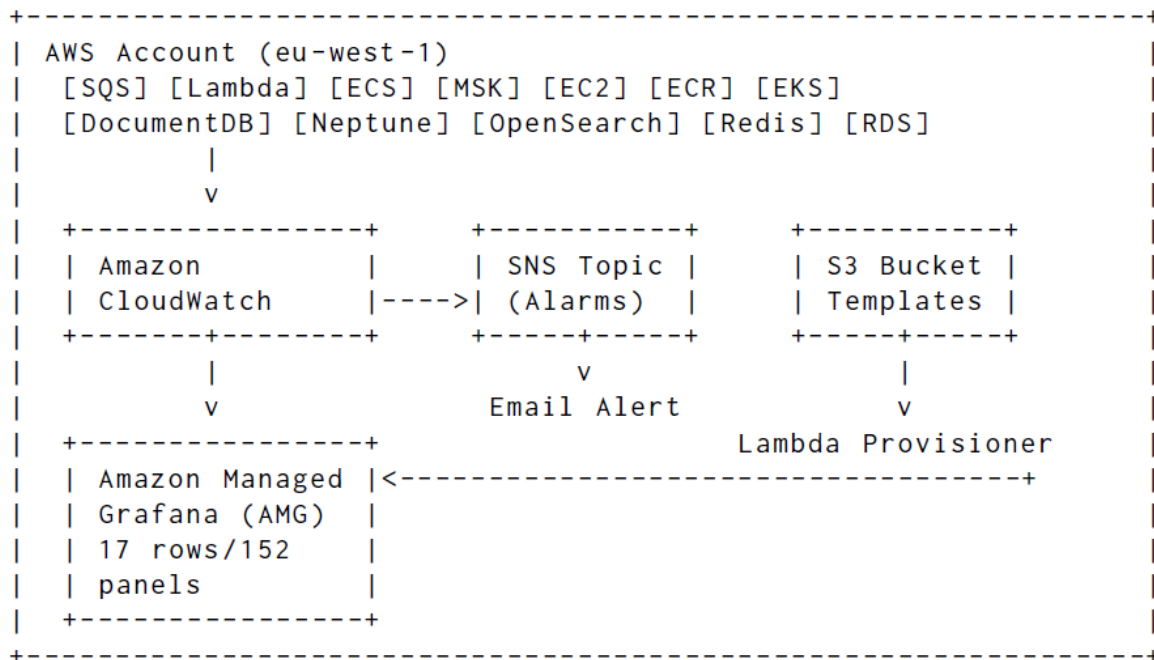


Figure 3: AMG Observability Platform — System Architecture

The **infrastructure layer** is the monitored subject I targeted: six budget platforms housing Lambda functions, ECS clusters, EC2 instances, and the full spectrum of AWS database and messaging services used by the Meridian and CitcoWorks platforms.

The **data layer** ingests, stores, and queries observability signals from the infrastructure layer. I chose Amazon CloudWatch as the sole data source — a deliberate architectural decision (see Section 10.8.3) that eliminates the operational overhead of a self-managed Prometheus stack. I used CloudWatch Metric Insights SQL to provide per-resource aggregation, enabling multi-service panels without requiring one panel per resource.

The **visualization and alerting layer** splits between two components I built. I used Amazon Managed Grafana (AMG) to render the multi-row dashboard for human operators, and I configured CloudWatch Alarms with SNS routing to fire notifications when metrics breach their SLO-aligned thresholds.

The **consumers layer** includes three groups: operators who use the Grafana dashboard I built during incident response, the IA Resources Inventory REST API I developed that exposes the AWS service fleet programmatically, and the SRE team's email distribution list that receives SNS alarm notifications I configured. This layered architecture ensures separation of concerns: the data layer handles metric collection and storage, the visualization layer transforms raw metrics into actionable insights, and the consumer layer delivers those insights to the appropriate stakeholders through purpose-built interfaces.

10.2 Work Stream 1: CloudWatch Dashboard Platform

10.2.1 Initial Lambda Dashboard

My earliest monitoring work, completed in April 2026, produced a native Amazon CloudWatch dashboard covering the `rpacfs1` and `ia-bpo` Lambda function fleets — 51 functions total. The dashboard displayed per-function panels for invocation counts, error rates, duration averages, and throttle events. I also created a Confluence inventory page mapping each function.

The architectural pattern for this early work was straightforward: CloudWatch native widgets queried the AWS/Lambda namespace using `SEARCH()` expressions to enumerate all functions matching a `FunctionName` prefix. This approach worked adequately for small, homogeneous fleets but surfaced two structural limitations that drove the later Metric Insights SQL migration: the **`SEARCH()` visibility gap** (idle functions disappear from panels) and **no cross-environment aggregation** (a single expression cannot span multiple tag dimensions simultaneously).

Figure 4 shows a live screenshot of the CloudWatch Lambda dashboard displaying invocation and error metrics.

10.2.2 Extended Dashboard and Architecture Analysis

I extended the initial dashboard with additional tag dimensions covering `automationhub`, `bpm`, `citcoworks`, and `meridian budget_codes`, and introduced CloudWatch Logs Insights widgets for error log surfacing. Rather than relying solely on Errors metrics from the AWS/Lambda namespace, the Logs Insights widgets queried the `/aws/lambda/` log group prefix directly, enabling pattern-matched error extraction regardless of whether Lambda had published a corresponding metric.

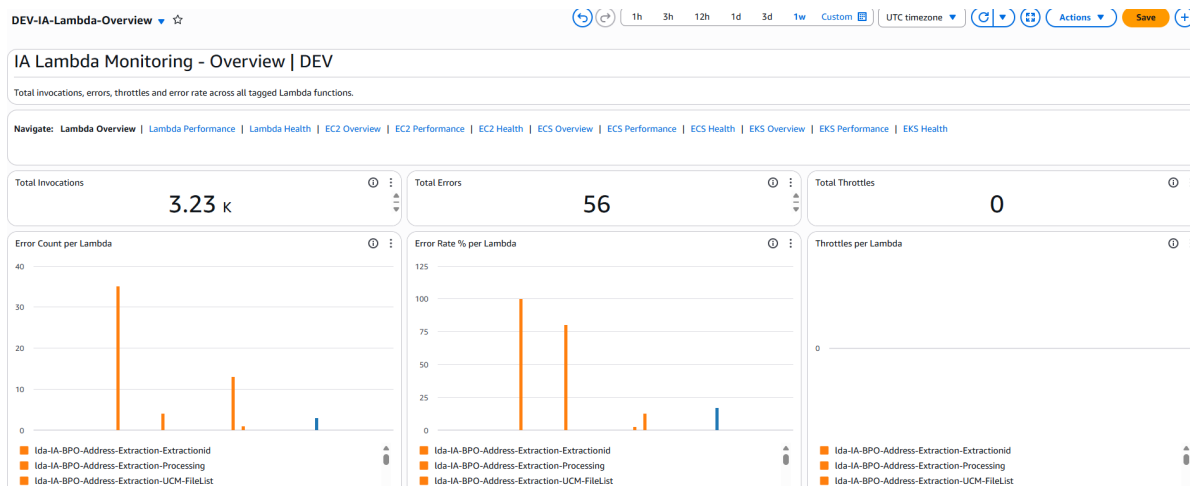


Figure 4: CloudWatch Lambda Dashboard — Live Screenshot (ia-resources-monitoring)

A significant architectural investigation within this task evaluated the feasibility of a fully centralized log aggregation approach using CloudWatch metric subscription filters. My analysis quantified the infrastructure cost of this design: routing log events from all IA Lambda functions into a centralized processing layer would require **106 metric subscription filters**—one per function per metric dimension—across the six platforms. While technically feasible from a CloudWatch service quota perspective, this approach was ultimately blocked by an IAM permission boundary constraint: the `logs:PutSubscriptionFilter` action was not granted within the IA team’s scoped permission set, preventing programmatic creation of subscription filters at the scale required.

This investigation also evaluated five distinct approaches to out-of-memory (OOM) detection for Lambda functions: (1) parsing REPORT log lines from CloudWatch Logs Insights; (2) comparing `MaxMemoryUsed` against `MemorySize` via the Lambda enhanced monitoring extension; (3) CloudWatch Lambda Insights (paid tier); (4) custom metric emission on memory threshold breach; and (5) structured log parsing via a metric filter pattern. I documented each approach with its detection latency, cost, and permission requirements. This analysis informed the Container Insights cost evaluation and the alarm design for the dashboard.

10.2.3 IA Infrastructure Monitoring Sub-Stories

The following sub-stories document specific discoveries and architectural patterns that emerged during the CloudWatch dashboard development.

EC2 Inventory and Tag Mismatch Discovery

My gap analysis of the EC2 inventory scanned **83 instances** across the six platforms. This scan revealed a critical tagging inconsistency: the rpacfs1 environment's EC2 instances were tagged with `budget_code="CFS/RPA"` rather than the expected **"rpacfs1"** value, making them invisible to tag-based `budget_code` lookups. This discovery led me to introduce the `AppName` fallback pattern: when a primary `budget_code` lookup returns no results for `rpacfs1`, the discovery logic falls back to filtering by `AppName="rpacfs1"`. This fallback became the standard for all resource discovery in the inventory API (see Section 10.4).

Container Insights Cost Analysis

With ECS task-level telemetry absent from five of the six Meridian ECS clusters, I evaluated four Container Insights deployment options:

1. **No Container Insights** — zero cost, zero task-level visibility.
2. **Enhanced Container Insights** — full task and pod metrics, but carries per-metric ingestion cost that scales with task count.
3. **Standard Container Insights** — cluster- and service-level CPU/memory metrics via the `ECS/ContainerInsights` namespace, charged at CloudWatch standard metric pricing.
4. **CloudWatch Agent on ECS (self-managed)** — maximum flexibility, highest operational overhead.

My analysis recommended **Standard Container Insights** at an estimated cost of **\$10.44/month** for full coverage across all 22 ECS clusters. This option provided the SLI data required for FR2 (ECS CPU/memory dashboard) at negligible incremental cost relative to the existing CloudWatch spend. I enabled Standard Container Insights on five of the six Meridian clusters on May 11–12, 2026.

SQS Registry Architecture

The SQS monitoring feature I added on introduced a **registry-based architecture with a three-tier fallback model** for resilient SQS queue data retrieval. The three tiers operated as follows:

1. **Tier 1 — Live API**: Query CloudWatch AWS/SQS metrics directly via the CloudWatch API. Fast and fresh, but requires network connectivity and API quota.

2. **Tier 2 — S3 Cache:** If the API call fails (rate limit, transient error), retrieve the last-known queue snapshot from an S3 bucket. Stale by up to one cache TTL window, but always available.
3. **Tier 3 — Local File:** If both API and S3 fail, fall back to a locally bundled queue inventory file. Guaranteed to return data; snapshot reflects deployment-time inventory.

This three-tier model ensured the SQS monitoring pipeline remained functional even during AWS service degradation events. The feature was validated with **automated tests** covering the registry dispatch logic, fallback transitions, and queue metadata serialization.

10.3 Work Stream 2: Grafana AMG Observability Platform

This was my primary deliverable over the bulk of my co-op placement. I built a centralized Amazon Managed Grafana workspace with 17 dashboard rows and panels, dynamic CloudWatch alarms, and a Lambda-backed IaC provisioner — all structured across a four-layer architecture I designed from scratch.

10.3.1 Architectural Layers

I organized the Grafana AMG platform into a four-layer architecture that separates data collection, visualization, alerting, and infrastructure provisioning concerns.

Layer 1: Data Layer

I chose Amazon CloudWatch as the **sole data source** for the entire Grafana workspace. My deliberate decision to use only the native Grafana CloudWatch plugin — rather than a self-managed Prometheus/CloudWatch exporter stack — eliminated all additional infrastructure requirements. I structured the data layer around:

- **CloudWatch plugin:** I connected the Grafana AMG workspace to CloudWatch via the built-in plugin, authenticated through an IAM role. This required no additional data collectors, agents, or scrapers.
- **IAM role model:** I built a **16-role deployment** (one role per AWS account, covering all six IA platform accounts) on a **three-role IAM chaining architecture** to give the Grafana service account cross-account read access across all IA AWS accounts. I scoped each role to the minimum permissions required: `cloudwatch:GetMetricData`, `cloudwatch:ListMetrics`,

cloudwatch:DescribeAlarms, logs:StartQuery, logs:GetQueryResults, and ec2:DescribeTags (for tag-based resource enumeration). Figure 5 shows the three-role IAM chaining architecture I designed because CTM-Developer cannot create IAM roles directly and the shared CITCO-CloudFormationBuild role lacks grafana:* permissions.

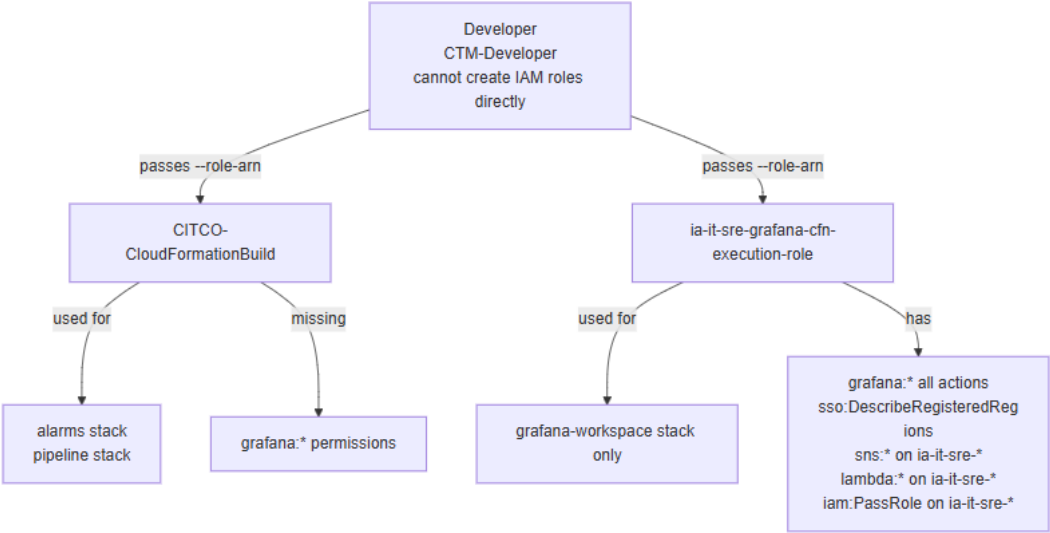


Figure 5: IAM Role Architecture — Three-Role Model for Grafana AMG Deployment

- **Metric Insights SQL:** I used CloudWatch Metric Insights SQL as the primary query language for multi-resource panels. This enabled SQL-like aggregation across all resources in a namespace with a single query, replacing the brittle SEARCH() expressions I had used in the earlier CloudWatch dashboard.

Layer 2: Visualization Layer

I built the Grafana AMG workspace with **17 dashboard rows** organized by monitored service category. Table 7 summarizes the dashboard structure I implemented.

Table 7: Grafana AMG Dashboard — Row Structure

Row	Service	Panels and Metrics
1	Health Overview	Color-coded stat tile per service (Lambda, SQS, MSK, EC2, ECR, EKS, ECS, RDS, DocumentDB, Redis, Neptune, OpenSearch); critical/warning alarm counts

Row	Service	Panels and Metrics
2	Performance Overview	Top-10 bargauge comparisons per service — highest queue depth, Lambda errors, ECS CPU, Kafka consumer lag, Redis memory, etc.
3	Lambda	Invocations, errors, throttles, concurrent executions, duration P50/P99; Logs Insights error panel
4	SQS	Queue depth (ApproximateNumberOfMessagesVisible), oldest message age, DLQ depth
5	MSK/Kafka	Consumer lag: SumOffsetLag and EstimatedMaxTimeLag; broker CPU, disk, messages in/out per second
6	EC2	CPU (avg/max), memory (CloudWatch Agent), network I/O, disk I/O, status checks
7	ECR	Container image pull activity
8	EKS	Kubernetes node and pod metrics
9	ECS Services	CPU and memory utilization per cluster and service; Container Insights task-level metrics
10	ECS Target Groups	ALB request throughput, response time P50/P95/P99, HTTP 5xx/4xx error counts
11	RDS / Aurora	CPU, database connections, read/write latency, IOPS, freeable memory
12	DocumentDB	CPU, disk queue depth, active connections, read/write latency, IOPS
13	Redis / Valkey	Engine CPU (avg/max), memory, cache hit rate, replication lag, connections, latency
14	Neptune DB	CPU, Gremlin/SPARQL errors, freeable memory, queue depth, network throughput
15	OpenSearch	Cluster status (green/yellow/red), JVM heap pressure, indexing rate, search latency

Figures 6–8 show live screenshots from the Grafana AMG dashboard I deployed, demonstrating the Health Overview, Lambda monitoring, and ECS monitoring rows I built.

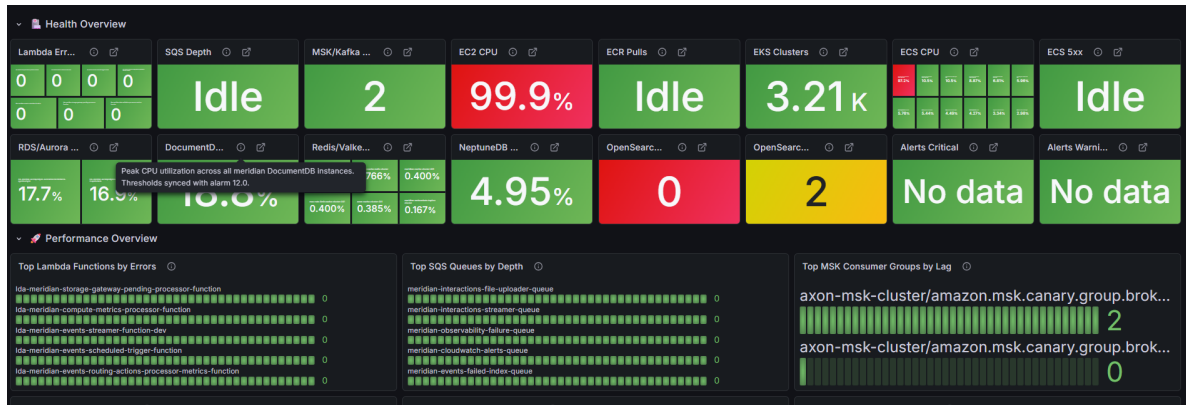


Figure 6: Grafana AMG Dashboard — Health Overview Row (Live Screenshot)



Figure 7: Grafana AMG Dashboard — Lambda Monitoring Row (Live Screenshot)

The Lambda Monitoring Row (Figure 7) I built provides real-time visibility into all Lambda functions across the selected environment. The top table displays function names, trigger types, runtime versions, memory allocations, and timeout configurations — enabling operators to identify misconfigured functions at a glance. Below the table, I added four time-series panels tracking the four golden signals: **Total Errors** (count of failed invocations), **Invocations** (throughput), **Concurrent Executions** (saturation indicator), and **Throttles** (capacity limit breaches). I added a color-coded legend below each panel to enable drill-down to individual functions.

The ECS Monitoring Row (Figure 8) I built surfaces Container Insights metrics for all ECS services in the selected cluster. I added a service table showing cluster assignment, desired count, and running count — highlighting any discrepancies that indicate failed deployments or scaling issues. I also added four time-series panels displaying CPU and memory utilization (both average and



Figure 8: Grafana AMG Dashboard — ECS Monitoring Row (Live Screenshot)

maximum) across all tasks, enabling operators to identify resource saturation before it impacts service availability. This row was particularly valuable for monitoring the Meridian ECS services, where memory pressure had previously gone undetected until task failures occurred.

Grafana Template Variables. I implemented environment-level filtering through three Grafana template variables backed by CloudWatch dimension-value queries:

- `$budget_code` — filters all panels to a specific environment (rpacfs1, ia-bpo, meridian, citcoworks, automationhub). Backed by a CloudWatch ListMetrics call enumerating distinct `budget_code` tag values.
- `$ecs_cluster` — filters ECS panels to a specific cluster. Backed by a `dimension_values()` query on the ECS/ContainerInsights namespace.
- `$lambda_function` — filters Lambda panels to a specific function. Backed by a `dimension_values()` query on the AWS/Lambda namespace.

When a user selects a value from any variable, all panels that reference that variable re-execute their queries with the selected filter applied, scoping the entire dashboard to a single environment or resource. I designed it this way to eliminate the need to maintain separate dashboards per platform.

CloudWatch Metric Insights SQL: Architecture Pattern. CloudWatch Metric Insights SQL is the key architectural pattern I adopted to enable per-resource drill-down across large fleets within a single query. I used the standard `SELECT ... FROM ... GROUP BY` form to return metrics for each resource

as a separate time series, covering all resources without requiring one panel per function. I used the WHERE clause to filter by tag or dimension and scope results to specific platforms.

Percentile Limitation and Workaround. A fundamental constraint of Metric Insights SQL is that the GROUP BY mode only supports the aggregate functions AVG, SUM, MIN, and MAX. Percentile statistics (p50, p95, p99) are *not available* in GROUP BY mode. This is a CloudWatch API limitation, not a Grafana limitation.

For panels requiring true latency percentiles, the workaround is to use the *standard CloudWatch metrics* query mode with a wildcard dimension value (e.g., `FunctionName: ["]*["]`). This returns individual time series for each function that has emitted metrics within the selected time window. The tradeoff is that only recently active functions appear — the same “visibility gap” limitation noted for `SEARCH()`. For the Lambda duration P50/P95 panels, I accepted this tradeoff: a function that has not been invoked recently has no meaningful latency distribution to display.

Layer 3: Alerting Layer

I provisioned **dynamic CloudWatch alarms** across all monitored platforms, each connected to an SNS topic that routes notifications to the IA-SRE team’s email distribution list. Alarm categories include:

- **Lambda alarms:** Error rate exceeding 1% over a 5-minute rolling window; throttle count exceeding threshold; concurrent execution count approaching account limit.
- **ECS alarms:** CPU utilization sustained above 80% over a 5-minute evaluation period; task desired count $\neq$ running count (task failure detection).
- **MSK alarms:** EstimatedMaxTimeLag exceeding 10,000 messages on any consumer group. This alarm directly addresses the Meridian Incident: it would have fired within minutes of the lag beginning to build, compared to the 3.5-hour detection gap that occurred under the reactive model.
- **SQS alarms:** Queue depth exceeding configurable threshold per queue; ApproximateAgeOfOldestMessage exceeding 1 hour (configurable per SLA).
- **Composite alarms:** Platform-level health signals that aggregate per-service alarm states into a single OK/ALARM indicator per `budget_code` environment.

Layer 4: Infrastructure-as-Code Provisioner

The IaC layer is a **Lambda-backed CloudFormation provisioner** I deployed via AWS SAM. I designed it to address a specific scaling constraint: with 119 Lambda functions, a static CloudFormation template defining one alarm per function would contain $119 \times (\text{alarm types per function})$ resources, rapidly approaching the CloudFormation 500-resource-per-stack limit.

I resolved this by making the provisioner *dynamically discover* the current function fleet at deploy time and synthesize alarm resources programmatically:

1. The provisioner Lambda function I wrote is invoked by a CloudFormation custom resource on every `samconfig.toml` deploy.
2. It reads parameterized CloudFormation YAML templates I stored in an S3 bucket (see Section 10.8.1).
3. For each template (Lambda alarms, ECS alarms, SQS alarms, MSK alarms), it calls the relevant AWS discovery API to enumerate all current resources in the specified platform.
4. It synthesizes CloudFormation `AWS::CloudWatch::Alarm` resource blocks for each discovered resource and passes the generated template to CloudFormation for stack deployment.
5. I deployed four CloudFormation stacks on **May 11, 2026**: `ia-sre-lambda-alarms`, `ia-sre-ecs-alarms`, `ia-sre-sqs-alarms`, and `ia-sre-msk-alarms`.

I wrote a `samconfig.toml` with environment-specific parameter files to enable reuse across DEV, UAT, and PROD without code changes. I externalized the budget code, alarm thresholds, and SNS topic ARN as SAM parameters, satisfying NFR5 (multi-environment reuse).

Figure 9 illustrates the alarm notification flow from CloudWatch alarm state change through SNS topic fan-out to the IA-SRE team's email distribution list.

Figure 10 shows the CodePipeline CI/CD deployment flow that delivers all stacks across the three environments.

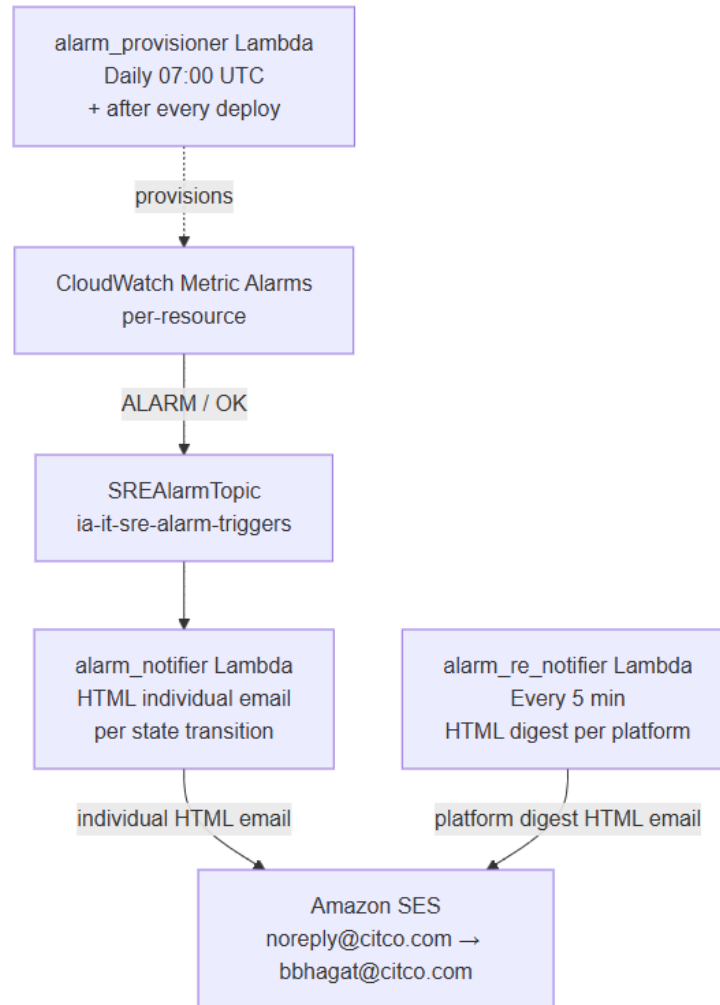


Figure 9: Alarm Notification Flow — CloudWatch to SNS to Email Distribution

10.3.2 Grafana Version Upgrade: Architectural Resilience Test

On **May 13, 2026** — two days after my initial infrastructure deployment — the Grafana AMG workspace was upgraded from version **10.4 to 12.4**. This was a major version migration that constituted an unplanned but highly informative architectural resilience test for the dashboard I had built.

The upgrade introduced breaking changes in two areas:

- **CloudWatch Metric Insights SQL editor:** The query syntax editor in Grafana 12.4 changed the way SQL expressions were associated with panel targets. All SQL panels required migration to the updated CloudWatch query model.

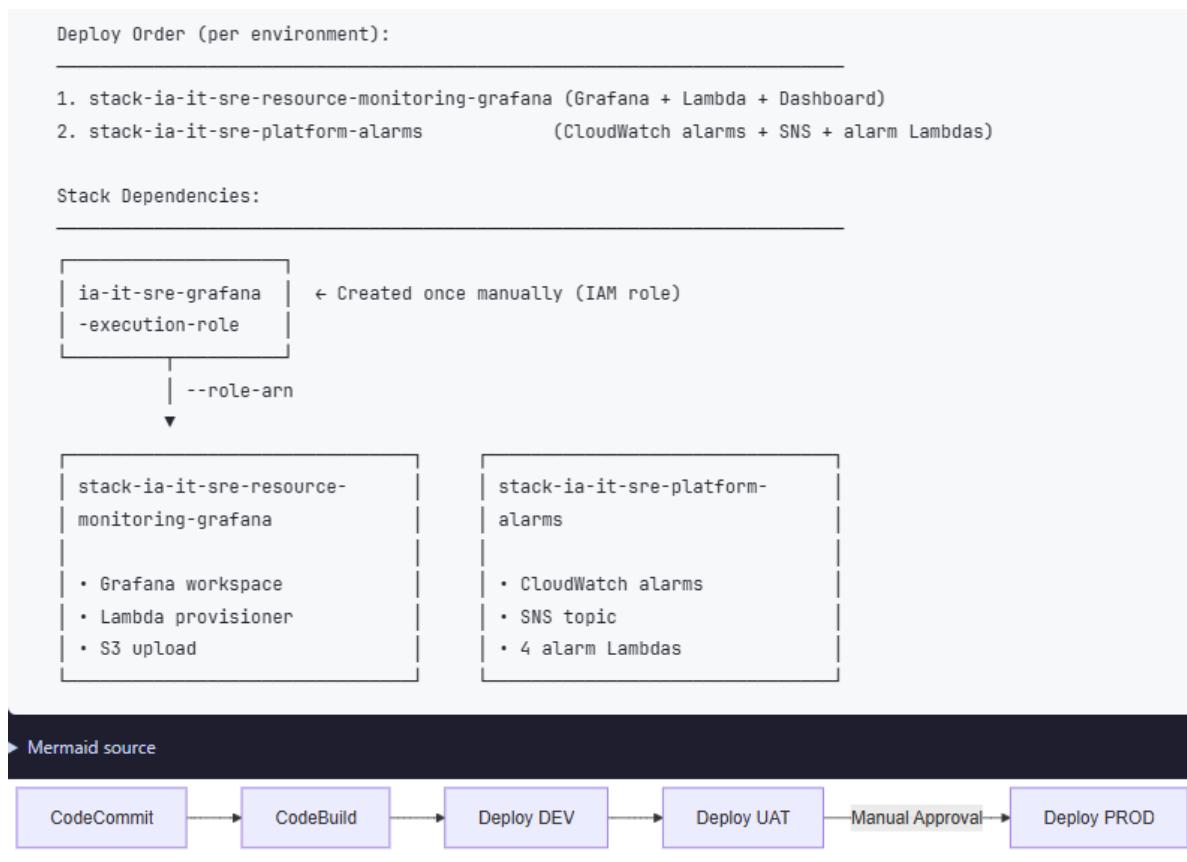


Figure 10: CI/CD Deployment Pipeline — CodeCommit to AMG Deployment Flow

- **Variable interpolation:** Template variable syntax in panel queries changed between versions, causing several `$budget_code` and `$ecs_cluster` references to fail to resolve correctly after the upgrade.

I resolved this through a systematic audit of all panels to identify affected queries. The migration reinforced a key design principle I documented as NFR6: **use only standard CloudWatch plugin queries and avoid AMG-version-specific syntax**. I migrated all panel queries to the stable Grafana 12.4 CloudWatch query model, and the workspace has remained stable through subsequent Grafana patch releases.

10.3.3 X-Ray Distributed Tracing (PoC)

As part of the observability stack I built, I integrated AWS X-Ray tracing to provide distributed trace visibility across the IA platform. The X-Ray integration I configured enables service map visualization and trace waterfall analysis — capabilities that contribute to the 60% Dynatrace parity achieved by the CloudWatch + Grafana + X-Ray stack I assembled (see Section 9.3).

Figure 11 shows the X-Ray service map displaying the trace topology for the RPA-LLM platform, illustrating how requests flow through Lambda functions, external APIs, and AWS services.

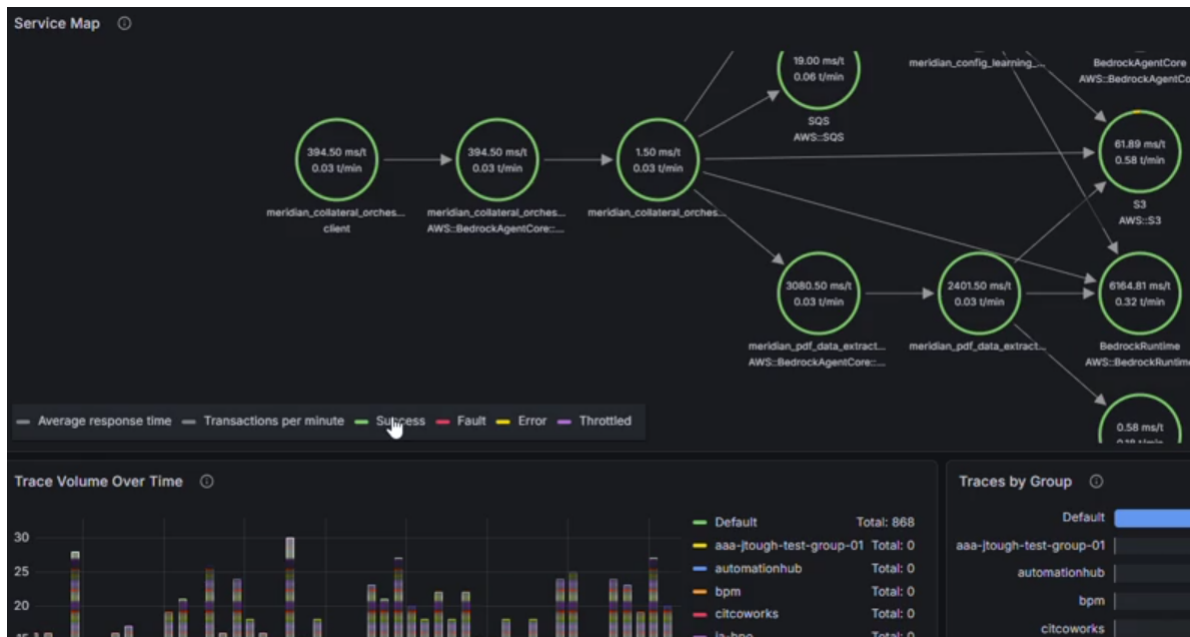


Figure 11: AWS X-Ray Service Map — Distributed Trace Topology (Live Screenshot)

The X-Ray integration delivers three key capabilities:

- **Service Map:** Visual representation of service dependencies and request flow, enabling rapid identification of bottlenecks and failure points.
- **Trace Waterfall:** Detailed timing breakdown for individual requests, showing latency contribution from each service in the call chain.
- **Error Analysis:** Automatic capture of exceptions and error responses with stack traces, correlated to specific trace segments.

X-Ray tracing is currently enabled in DEV environment only; I am awaiting senior approval for UAT and PROD enablement due to the additional CloudWatch costs associated with trace segment storage.

10.4 Work Stream 3: IA Resources Inventory API

I built a **Flask REST API** for programmatic discovery of all IA platform cloud resources. I created the ia-sre-resources-inventory repository and scaffolded the project on Jun 11, 2026; I brought it to production-ready state with 562 automated tests and 99% code coverage.

10.4.1 Technology Stack

- **Language:** Python 3, flask[async] for async route handlers
- **AWS SDK:** boto3 for all resource discovery
- **Validation:** Pydantic response models (type-safe Resource dataclass with fields: name, type, budget_code, tags, arn)
- **Deployment:** AWS Lambda via serverless-wsgi adapter
- **Testing:** pytest + Hypothesis (property-based tests)

10.4.2 Discoverer Registry Pattern

My core architectural contribution in the inventory API is the **Discoverer Registry pattern**. Rather than routing resource discovery logic through a monolithic handler, I built a registry that maps (resource_type, budget_code) tuples to concrete discoverer classes. Each discoverer implements a single abstract method `async def discover() -> List[Resource]`. Adding support for a new platform or resource type requires implementing one class and registering one entry — zero changes to routing, serialization, or test infrastructure.

10.4.3 API Endpoints

Table 8: IA Resources Inventory API — Endpoint Summary

Endpoint	Description
GET /api/v1/lambda	All Lambda functions; optional ?budget_code=rpacfs1 filter
GET /api/v1/ecs	All ECS clusters across all platforms
GET /api/v1/ec2	All EC2 instances (tag-based discovery)
GET /api/v1/secrets	Secrets Manager secrets inventory
GET /api/v1/s3	S3 buckets per budget_code
GET /api/v1/health	Health check — returns 200 OK

10.4.4 Tag-Based Discovery and the Two-Stage Filter

The most technically interesting discovery challenge I encountered was the citcoworks and meridian tag ambiguity. Both environments share the AWS tag value **"Shared Cloud Artifacts"** for the budget_code dimension — a legacy of the two platforms being onboarded before the

IA team established per-platform `budget_code` uniqueness requirements. A naive `budget_code` lookup would conflate resources from two operationally distinct environments.

Figure 12 shows how the Inventory API integrates with the Grafana AMG dashboard — the cache-refresh Lambda writes S3 snapshots and publishes CloudWatch metrics, which Grafana reads via template variables and Metric Insights SQL panels.

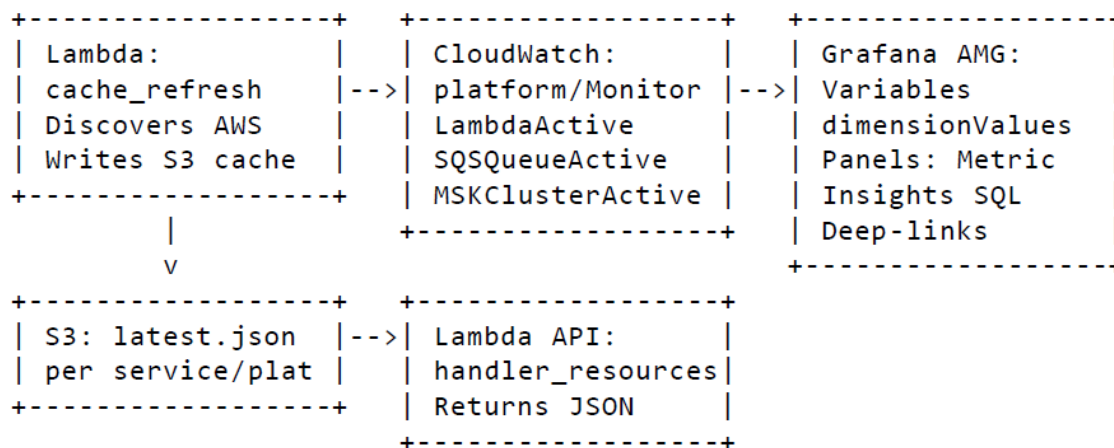


Figure 12: Inventory API — Grafana AMG Integration

I implemented a **two-stage filter** in both the `CitcoWorksECSDiscoverer` and `MeridianECSDiscoverer`. Stage 1 filters by `budget_code` (which is shared between both platforms), then Stage 2 uses the `AppName` tag to discriminate Meridian resources from CitcoWorks resources.

I designated the `AppName` tag as the **mandatory secondary discriminator** for all resources in the “**Shared Cloud Artifacts**” `budget_code` environments. Any new resource added to either environment without the correct `AppName` tag will be invisible to the inventory API. I mitigated this risk with a tag compliance CloudWatch alarm that fires if new ECS services or Lambda functions lack the required tags.

Figure 13 illustrates the complete resource discovery flow, showing how the API routes requests through the discoverer registry and applies the two-stage filter for ambiguous `budget_codes`.

10.4.5 Async Architecture

I built the API using `flask[async]` with async route handlers. Each handler aggregates boto3 calls concurrently using `asyncio.gather()`, reducing total API latency by parallelizing the AWS API

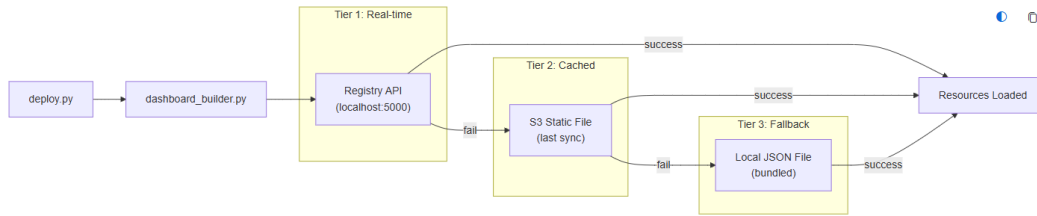


Figure 13: Resource Discovery Flow — Discoverer Registry and Two-Stage Filter

calls within a single request. All matching discoverers run concurrently rather than sequentially, reducing API response time from the sum of all boto3 call latencies to the latency of the single slowest call — consistently keeping the API within the 10-second NFR3 target.

10.4.6 Test Strategy

My test suite for the inventory API achieves **99% code coverage**:

- **Unit tests:** Each discoverer class tested in isolation with mocked boto3 clients. Verifies correct tag filtering, pagination handling, and Resource object construction.
- **Integration tests:** Full Flask test client hitting all REST endpoints. Verifies routing, serialization, and error response format.
- **Property-based tests (Hypothesis):** Tag-parsing logic, registry dispatch, and Pydantic response serialization tested with randomly generated inputs. Verifies that the two-stage filter never returns a meridian resource when `budget_code=citcoworks` is specified, regardless of what combination of tag values is present.
- **Edge case tests:** Malformed tags, empty `budget_code` environments, boto3 pagination edge cases (empty pages, single-page results, maximum page sizes).

10.5 Work Stream 4: RPA Production Support

Throughout the co-op placement I participated in dedicated RPA production support rotations as part of the IA team's **Follow-the-Sun model**. I operated within a three-tier support structure:

- **L1 — Ask Citco RPA (Work Stream 5):** The LLM-powered self-service chatbot I also worked on, which answers first-line queries from RPA operators about process status, procedures, and Legal Entity assignments.

- **L2 — Innovation IT Support** (Work Stream 4): The human-staffed Follow-the-Sun rotation I participated in, covering Manila (APAC), Dublin (EMEA), and Halifax (NA) shifts. At L2 I handled Service Desk tickets that L1 could not resolve, performing triage, investigation, and resolution or escalation.
- **L3 — SRE** (our team): The escalation tier for infrastructure-level incidents requiring cloud access, code-level fixes, or platform changes beyond L2 scope.

I held the North America L2 slot (11 AM–7 PM EST) on a weekly rotation among the four NA-based SRE team members, serving as the primary contact for RPA incidents during that window. This was not project work with JIRA tickets — it was ongoing operational responsibility with Service Desk tickets, email triage, and ad-hoc troubleshooting.

10.5.1 Support Rotation Structure

Each support week followed a consistent daily rhythm:

- **Shift start (11 AM EST):** Review the Confluence handover log from the Manila shift to pick up carried-over items or approaching SLA breaches. Check open tickets in the SD Portal, Innovation IT email inbox, IA_INCIDENTS distribution list, and Teams notifications.
- **During shift:** Monitor VPL queue health (5 PM EST daily checkpoint), respond to escalations from Dublin/London business teams, triage incoming tickets, and coordinate with developers when code-level fixes are required.
- **Shift end (7 PM EST):** Update the Confluence handover log with ticket status, any pending actions, and context for the next shift (Manila).

10.5.2 Support Weeks Completed

I completed four full support rotations during the co-op placement:

- **Week 8** (May 4–8): First rotation. Familiarization with SD Portal, VPL queue monitoring, and the escalation ladder. Shadowed Mr. Leung on Corporate Actions ticket triage.
- **Week 12** (Jun 1–5): MESO hotfix week. Encountered MESO ECS autoscaling incident (ECS autoscaling gap) during routine monitoring — this led to a code fix, not just ticket resolution.

- **Week 20** (Jul 27–31): VPL GTL booking error incident (VPL Pricing GTL booking error). Coordinated with the business team on NAV impact assessment. Also handled routine queue monitoring and ticket triage.
- **Week 23** (Aug 17–21): Final rotation. Lighter ticket volume; used remaining time for report writing and KT preparation.

10.5.3 Notable Incidents Handled

While most support work involved routine ticket triage and queue monitoring, several incidents required deeper investigation or code-level fixes:

- **CAIS Deadline Pipeline Hotfix:** The CAIS Deadline bot — which uploads Standard Allocation Sheets to the PDS database and downloads deadline pricing reports — failed with `SMBAAuthenticationError: SpnegoError`. Seven-hour investigation traced to Secrets Manager credential rotation leaving empty values cached at module import time. Hotfix added fail-fast validation. Filing completed before regulatory deadline.
- **MESO ECS:** Tasks stuck, not autoscaling. Root cause: ECS service autoscaling policy was unconfigured. Fixed the scaling policy and added an ECS SaturationAlarm to the dashboard alarm set.
- **MESO Critical:** INNOCAP and SAMLMOS funds stuck in queue with no Lambda heartbeat. Manually retriggered the `ia_meso_process` pipeline. Processing completed approximately 2 hours after escalation.
- **VPL IPV:** Æxeo pricing window `CheckpointException` in UiPath — bot exhausted its retry budget waiting for a dynamic UI element. Estimated 16 hours for selector resilience fix; escalated to the development team.

10.5.4 Value of Support Rotation for SRE Work

The support rotation was not separate from the SRE observability work — it was *complementary*. Several incidents I encountered during support weeks directly influenced the dashboard and alarm design:

- The MESO ECS incident led to adding the `ECS_CPUSaturation` alarm to the Grafana dashboard.

- The CAIS Deadline credential failure reinforced the importance of fail-fast validation patterns documented in the Challenges chapter.
- VPL queue monitoring during support shifts provided real-world validation of the SQS dashboard panels and queue depth alarms.

When ticket volume was low, I used remaining support shift time for development work on the dashboard, inventory API, or documentation — maintaining productivity while remaining available for escalations.

10.6 Work Stream 5: RPA LLM Agent — Ask Citco RPA

During Week 21 I engaged with a parallel work stream outside the SRE monitoring scope: the **Ask Citco RPA** support chatbot, implemented across two CodeCommit repositories (ml-rpa-status-llm-agent and core-smart-rpa-llm-ecr-image). I started with a research and documentation task but expanded into production bug-fixing after my log analysis revealed critical reliability failures.

10.6.1 System Architecture

The chatbot is a **Retrieval-Augmented Generation (RAG)** pipeline built on Amazon Bedrock, designed to answer RPA operator queries about process status, procedures, and Legal Entity assignments without requiring developers to be on-call for routine questions. I studied the architecture in depth as part of my onboarding to the codebase before beginning my own contributions.

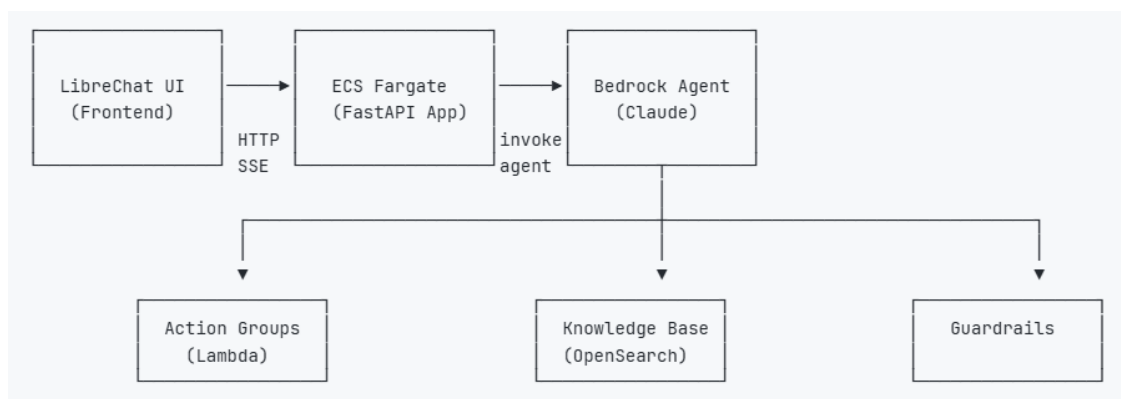


Figure 14: Ask Citco RPA — RAG Architecture

The **ECS Fargate** layer runs the FastAPI container, authenticates users via LDAP (checking `rpa_support_agent_user` AD group), invokes the Bedrock agent, and streams responses back in OpenAI SSE format. The **Bedrock Agent** (Claude) orchestrates between action groups and the Knowledge Base depending on the query type: process status queries hit the Event Service Lambda; procedural questions retrieve chunks from OpenSearch via hybrid vector and keyword search.

10.6.2 Knowledge Base Pipeline

I documented and analyzed the KB ingestion pipeline, which I found runs daily in two stages. I traced the full flow to understand how to improve it:

1. **12:00 UTC — IRCOEKBExtractLambda**: I found this function syncs 49 SharePoint IRCOEKB wiki libraries (~5,000 pages) to S3 as plain-text files with SharePoint metadata preserved.
2. **14:00 UTC — RPALLMKBIngestionFunction**: This triggers Bedrock KB ingestion, which chunks (hierarchical: 1,200 token parent / 300 token child), embeds (1024-dim), and indexes into OpenSearch Serverless (FAISS/HNSW, L2 distance).

I also reviewed the static document upload script (`scripts/upload_kb_content.py`), which compares content hashes before writing to S3 to provide idempotent uploads.

10.6.3 Production Reliability Issues Identified and Fixed

I began my work on this codebase with CloudWatch log analysis. My analysis of **93 days of production logs** (May 5 – August 5, 2026; 148 Lambda runs) revealed a critical reliability failure in the KB sync:

- **52% timeout rate on IRCOEKBExtractLambda**. 77 of 148 production runs hit the 300-second Lambda timeout. June 2026 was the worst month at 71% timeout rate. Average run duration was 291 seconds — every run within 36 seconds of the limit. When the Lambda timed out, the Knowledge Base was incomplete for that day: pages not yet uploaded when the timeout fired were missing from the KB until the next successful run.
- **Root cause: unconditional full re-upload**. The original code re-uploaded all ~980 pages on every daily run regardless of whether content had changed. The KB Articles library alone

(1,266 pages at ~220ms per S3 head_object call) required 278 seconds just for change detection I/O.

I implemented a two-phase hybrid sync (**v1.1.x + v1.1.18**) that reduced steady-state run time from 291 seconds average to approximately **25 seconds**:

1. **Phase 1 — Library-level change detection (~25s for all 49 libraries).** One lightweight ALB call per library using a SharePoint filter=Modified gt {last_synced_at} query. Zero results means the entire library is unchanged and is skipped with no further I/O.
2. **Phase 2 — Conditional sync.** Only changed libraries are processed. Sequential if five or fewer changed; parallel via ThreadPoolExecutor(max_workers=3) if more than five changed.
3. **Per-page idempotency.** Within each synced library, each page's SharePoint Modified timestamp is compared to the sp-modified field stored in S3 object metadata. Unchanged pages skip HTML extraction and S3 upload entirely.

Beyond the KB sync, log analysis also surfaced seven additional production bugs fixed across versions v1.1.1–v1.1.32:

- **OpenSearch silent delete (v1.1.1):** index creation failure silently deleted the index and reported CloudFormation SUCCESS. Fixed with proper exception handling and idempotency check.
- **Fire-and-forget Lambda pattern:** EventStoreExtractLambda and ClientFundMappingLambda always reported SUCCESS regardless of actual outcome due to incorrect exception class references (boto3.exceptions.BotoError does not exist).
- **Athena SSE encryption inconsistency:** queries used SSE_S3 instead of SSE_KMS in some Lambdas.
- **Athena race condition:** MSCK REPAIR TABLE executed immediately after CREATE TABLE without waiting for completion.
- **SSL verification disabled:** BPM API calls used verify=False.

Knowledge Base Enrichment. As part of the RPA LLM Agent work, I uploaded the NavChecklist process documentation PDFs (*Architecture – NAV CHECKLIST.pdf*, *FAQ – NAV CHECKLIST.pdf*) to

the KB S3 bucket under RPA Processes/NAV Checklist/ using the idempotent upload script. These documents — describing the NAV calculation checklist process flow — are now indexed in the production Knowledge Base and searchable by the chatbot.

10.6.4 Code Quality and Engineering Improvements

Alongside the bug fixes, I delivered the following engineering quality improvements to bring the codebase in line with IA Best Practice guidelines (pending dev approval gate):

- **Module refactor:** the 1,140-line monolithic `function.py` split into 8 focused modules (`handler.py`, `event_service_api.py`, `secrets.py`, `project_lookup.py`, etc.)
- **structlog migration:** replaced `print()` and root logger across all 11 Lambda functions with structlog structured logging
- **Type hints and docstrings:** PEP 484 type hints and Google-style docstrings added to all functions and classes
- **Test suite:** Comprehensive unit tests at 99% code coverage (v1.1.x series)
- **Pydantic models:** Databricks API response and BPM API response wrapped in Pydantic validation models
- **Kill switch:** SSM Parameter Store kill switch (RUN/STOP) implemented across IRCOEKB, Event Store, and Mapping Lambdas
- **X-Ray tracing:** AWS Lambda Powertools Tracer instrumented on all functions (enabled in DEV; pending approval for UAT/PROD)

Versions v1.1.1 through v1.1.32 were released during August 3–20, 2026.

10.6.5 Proposed: Hybrid Graph-RAG Architecture

During my codebase analysis I identified a structural limitation of the current flat-vector RAG approach for the Citco RPA domain. The entities in this domain are highly relational: clients have funds, funds run specific processes, processes execute on either Blue Prism or UiPath, and each platform has its own operational knowledge base. A standard vector similarity search treats all these entities as independent text chunks, losing the relational structure.

I propose a **Hybrid Graph-RAG** enhancement: adding Amazon Neptune as a graph layer alongside the existing OpenSearch vector store. I would model the entity relationship network

(client–fund–process–platform) as a graph, enabling the Bedrock agent to traverse relationships for multi-hop queries (“Which Blue Prism processes handle FX Closeouts for client X?”), then retrieve dense context from the vector KB for the identified process entities. The dual-path retrieval I am proposing would improve answer accuracy for complex operational queries while maintaining the existing RAG pipeline for straightforward procedural lookups. I will submit this proposal as a formal follow-on work item.

10.7 Work Stream 6: Citco Enterprise Custom UI Dashboards

During this co-op placement I began extending the existing ia-config-portal application with two new dashboard pages. I completed a comprehensive **architectural implementation plan** (Infrastructure Monitoring Dashboard-IMPLEMENTATION-PLAN.md) and started initial implementation. I am continuing this development during the co-op extension period (Sep–Dec 2026).

10.7.1 Planning Document Deliverable

The primary deliverable I produced for this work stream is a comprehensive 865-line implementation plan that I wrote documenting:

- **Gap analysis:** I identified and documented 45 Grafana AMG limitations with severity ratings, determining which gaps justify a custom UI solution.
- **Technology stack decisions:** I evaluated charting libraries (Chart.js vs Recharts), service map rendering (Cytoscape.js for X-Ray topology), and component framework integration with Citco’s internal @citco/core and @citco/theme packages.
- **Backend API design:** I designed four new Lambda handlers (/xray/service-map, /xray/traces, /xray/traces/{traceId}, /alarms) with request/response schemas and IAM permission requirements.
- **Task breakdown:** I broke down the work into 12 implementation tasks covering backend handlers, workspace scaffolding, component development, and portal integration.

10.7.2 Motivation: Grafana AMG Limitations

Through my Work Stream 2 dashboard work, I identified approximately 60% Dynatrace feature parity, but I also identified 45 architectural limitations. The most critical gaps I found that are driving the custom UI proposal:

Table 9: Critical Grafana AMG Limitations Identified

ID	Limitation	Severity
L1	CloudWatch 10 TPS hard throttle — panels error on every load	Critical
L9	X-Ray service map crashes on load — requires Edit mode workaround	Critical
L18	Table filter does not cross-filter other panels	Critical
L21	No topology / honeycomb / Dynatrace-style health view	Critical
L23	No alarm acknowledgment or snooze	Critical
L31	Cannot embed AMG in other portals (iframe blocked)	Medium

10.7.3 Planned Solution Architecture

In my planning document I proposed two new dashboard pages within the existing ia-config-portal (built by Ms. Venkataraman and Mr. Vadla):

- **Infrastructure Inventory Dashboard:** A frontend for the Inventory API I built in Work Stream 3, displaying resources across all platforms with filtering and Excel export capabilities.
- **Infrastructure Monitoring Dashboard:** A config-driven monitoring interface with drag-and-drop panel layout, Dynatrace-style honeycomb health visualizations, and X-Ray service map integration.

Key planned components I designed include:

- **CitcoHoneycomb:** A D3 hexbin-based health grid where each hexagon represents one resource instance, colored by real-time health status.
- **CitcoServiceMap:** A Cytoscape.js topology visualization replacing the crash-prone Grafana X-Ray plugin.

- **CitcoTraceWaterfall**: A custom Recharts component for X-Ray trace span visualization.

10.7.4 Status and Timeline

Table 10: Work Stream 6 — Status Summary

Item	Description	Status
Planning Document	865-line implementation plan with gap analysis, tech stack, task breakdown	Complete
Backend Handlers	4 new Lambda API endpoints for X-Ray and Alarms	Planned
Component Library	SRE-specific wrappers on @citco/core	Planned
Inventory Dashboard	Infrastructure Inventory Dashboard frontend for resource inventory	In Progress
Monitoring Dashboard	Infrastructure Monitoring Dashboard config-driven monitoring UI	In Progress

I deferred full implementation to the internship extension period (Sep–Dec 2026), allowing the Grafana AMG dashboard I built to stabilize in production and gather user feedback on which limitations are most impactful to address first.

10.8 Design Decisions & Tradeoffs

Four design decisions during my co-op placement warranted explicit documentation because each involved a meaningful tradeoff between competing engineering constraints. This section records the decision, the alternatives considered, and the rationale for the chosen approach.

10.8.1 Decision 1: Secrets Manager 64 KB Limit → S3 Migration

Problem. The CloudFormation IaC provisioner Lambda function stores its CloudFormation YAML templates as a payload retrieved at deploy time. Initially, this payload was stored in AWS Secrets Manager as a single JSON-encoded string. As the monitoring scope expanded to cover all seven AWS service categories, the generated YAML grew to **14,720 lines** at version v728 — exceeding

the **64 KB hard limit** on Secrets Manager secret values. The Lambda function began failing with `SecretSizeExceeded` on the `PutSecretValue` call.

Options Considered.

1. **Split into multiple secrets:** Partition the YAML into n secrets, each under 64 KB, and have the Lambda function reassemble them. This preserves Secrets Manager as the storage layer but introduces assembly logic and a hard dependency on secret ordering.
2. **Compress the payload:** gzip-compress the YAML before storing it as a binary secret. Achieves $\approx 70\%$ compression on YAML, bringing the payload well under 64 KB. Adds a decompression step in the Lambda function.
3. **Migrate to S3:** Store the YAML templates as S3 objects, versioned using S3 object versioning. The Lambda function retrieves templates via `s3:GetObject` at deploy time.

Decision. I chose **S3 migration**. The rationale:

- No size limit — templates can grow arbitrarily as additional AWS service categories are onboarded.
- Native versioning via S3 object versioning, providing a rollback mechanism for template changes at no additional cost.
- No reassembly or decompression complexity in the Lambda function.
- Cost negligible: a single `GetObject` call per deploy adds less than \$0.001 to operational cost.

Tradeoff. The Lambda execution role now requires `s3:GetObject` on the template bucket, broadening its IAM surface slightly. The `GetObject` call adds approximately 50 ms of latency to the provisioner bootstrap — acceptable for a deploy-time operation that runs infrequently.

10.8.2 Decision 2: AppName Tag as Mandatory Secondary Discriminator

Problem. As described in Section 10.4, the `citcoworks` and `meridian` environments share `budget_code="Shared Cloud Artifacts"`. A single-stage `budget_code` lookup cannot distinguish resources from the two environments, causing inventory contamination.

Root Cause. Both platforms were onboarded before the IA team established the policy requiring each platform to have a unique `budget_code` tag value. Retroactively changing the tag across all existing resources in two production environments would require a coordinated tagging campaign with risk of operational disruption.

Decision. I designated the `AppName` tag as the **mandatory secondary discriminator** for all resources in the **"Shared Cloud Artifacts"** `budget_code`. Discoverers for `citcoworks` and `meridian` must implement the two-stage filter described in Section 10.4.

Tradeoff. This decision creates a tagging governance obligation: all new resources added to either environment *must* carry the correct `AppName` tag. A resource missing this tag will be invisible to the inventory API, creating a silent coverage gap. This risk is mitigated by a CloudWatch tag compliance alarm and documented in the `ia-sre-resources-inventory` README as a platform-wide architectural constraint.

10.8.3 Decision 3: CloudWatch as Sole Data Source (No Prometheus)

Problem. A production-grade Grafana deployment typically uses Prometheus as the time-series database, with CloudWatch Exporter bridging AWS metrics. This is the pattern recommended by the Grafana community for large-scale AWS environments.

Alternative. Deploy a self-managed Prometheus server (on ECS or EC2) with the CloudWatch Exporter sidecar, and configure Grafana to use Prometheus as its data source.

Decision. I used the **native Grafana CloudWatch plugin** exclusively.

Rationale.

- **No additional infrastructure:** A Prometheus server and exporter fleet would introduce new ECS tasks, IAM roles, configuration management, and an additional failure domain.
- **Amazon Managed Grafana constraint:** AMG does not support self-managed Prometheus as a built-in data source without a Prometheus-compatible remote write endpoint.

Connecting AMG to a self-managed Prometheus would require Amazon Managed Service for Prometheus (AMP), adding further cost and complexity.

- **Direct IAM authentication:** The native CloudWatch plugin authenticates directly via IAM roles, requiring zero credential management beyond the initial role configuration.
- **Cost:** CloudWatch metrics are already being published by AWS services at no marginal cost. Prometheus would require scraping, storage, and retention management.

Tradeoff. The accepted limitation is that CloudWatch Metric Insights SQL does not support p50/p99 percentiles in GROUP BY mode (see Section 10.3). Latency distribution dashboards must use AVG and MAX as proxies, or fall back to the standard metrics wildcard mode which only shows recently active functions. For the current monitoring maturity of the IA platform, this tradeoff is acceptable — the P50/P99 gap can be addressed in a future phase by adding custom Lambda Duration metric emission with explicit percentile dimensions.

10.8.4 Decision 4: Lambda-Backed IaC Provisioner vs. Static CloudFormation

Problem. Provisioning one CloudWatch alarm per Lambda function for all monitored metric types would generate over 100 AWS::CloudWatch::Alarm resources per stack. Approaching the **CloudFormation 500-resource per-stack limit** was a realistic concern as the fleet grew. A static template encoding all alarms explicitly would also require manual updates every time the Lambda fleet changed.

Alternative. Static CloudFormation templates with each alarm resource hardcoded — one YAML block per function per alarm type, updated manually when functions are added or removed.

Decision. I chose a **Lambda-backed dynamic provisioner** that discovers the function fleet at deploy time and synthesizes alarm resources programmatically.

Rationale.

- **Fleet-scale coverage:** The provisioner ensures every Lambda function in every platform has alarms configured, regardless of when it was deployed. No manual update is needed when a new function is added.

- **Avoids static YAML maintenance:** A static template for 119 functions $\times$ 3 alarm types = 357 resources would be a 500+ line YAML file requiring hand-editing for every fleet change.
- **Parameterization:** Thresholds, evaluation periods, and SNS topic ARNs are all externalized as SAM parameters, enabling multi-environment reuse without code changes.

Tradeoff. The Lambda provisioner introduces pipeline ordering complexity: the provisioner Lambda must complete successfully before the CloudFormation stack update can proceed, creating a sequential dependency. A Lambda execution failure (timeout, permissions error) would halt the deployment pipeline without deploying updated alarms. This is mitigated by comprehensive error logging and a manual re-invocation pathway via `python main.py deploy --phase 3`.

These design decisions shaped an architecture that favors operational simplicity and zero-maintenance fleet coverage over strict metric precision, while accepting bounded and well-understood tradeoffs at each decision point. I documented each decision in the respective repository README and JIRA acceptance criteria so that future team members can revisit any decision as the platform's monitoring maturity evolves.

11. WEEKLY IMPLEMENTATION BREAKDOWN

This chapter documents the work I delivered each week across the 24-week co-op placement (March 15 – August 31, 2026). Weeks 1–4 operated at part-time intensity (~ 15 – 20 h/week) while balancing university coursework; full-time engagement (~ 40 h/week) began in Week 5.

11.1 Week 1

Week 1 marked the formal start of the co-op placement with Day-1 orientation alongside co-interns Mr. Vadla and Ms. Venkataraman.

Team Introduction and Placement Overview: I joined the Intelligent Automation SRE team at Citco Technology Management. The introductory meeting with my manager Mr. Deotale gave me a comprehensive overview of the team structure, the six IA budget_code platforms I would be monitoring, and the co-op placement expectations. Senior mentors Mr. Leung and Ms. Labos

introduced themselves and outlined their roles: Mr. Leung as the RPA Platform Lead and Ms. Labos as the SRE engineer who would be my primary technical mentor for the monitoring work.

Access Requests and Development Environment Setup: My first task was establishing access to Citco's development infrastructure. I raised access request tickets for Confluence, JIRA, and BitBucket through the internal ticketing system. I initiated AWS Core Dev access through the IDM portal, which required a separate approval chain. I installed the full local development toolchain: VSCode with Python and AWS Toolkit extensions, Git with SSH key configuration, PyCharm for Python development, Postman for API testing, and the Amazon Q plugin for AI-assisted coding. The Amazon Q setup required a Builder ID registration and served as my first introduction to AI-augmented development tooling at Citco.

Security Briefing and Compliance Context: I attended a security briefing with Mr. Deotale, Mr. Ivan Carlo, and Ms. Maria, covering the BPM Control Questionnaire as part of the annual security review process. This session gave me foundational context about Citco's security posture that I would carry throughout the placement. Key topics included: Active Directory integration for role-based authorization across all Citco systems; the recent authentication migration from SiteMinder to Ping Identity; penetration testing procedures (the last BPM pentest was conducted by the Red Team on UAT only to protect production uptime); and the access logging infrastructure built on the ELK stack (Kibana, Elasticsearch) with 7–14 day log retention policies. I also learned about Dynatrace application monitoring for anomaly detection and alerting—knowledge that would become directly relevant when I later evaluated Dynatrace parity as a framing question for the Grafana AMG dashboard work.

AWS Tooling Introduction and Team Integration: I participated in an introductory call on Amazon Q and the Amazon Tools ecosystem, which included a walkthrough of the AWS developer toolchain available to the team. I familiarized myself with the Citco JIRA board (IISS project), read the team's active Confluence pages covering the IA platform overview, and reviewed the existing tickets on the board to understand what work was in progress before my arrival. I attended the team's daily standup for the first time and began syncing with Mr. Vadla and Ms. Venkataraman on our shared onboarding progress.

By the end of Week 1, all access requests were submitted and in progress, my local development environment was fully operational, and I had a solid understanding of Citco's security posture, the IA platform scope, and the team's operational rhythm.

11.2 Week 2

With access requests progressing through the approval queue, Week 2 shifted to intensive self-directed AWS learning. Dr. Pawan had confirmed that the part-time schedule would continue through mid-April while my university courses were running, so I structured these early weeks around building the AWS knowledge foundation I would need before taking on my first JIRA tickets.

AWS Database Portfolio Study: I completed a structured study of the full AWS database portfolio, working through both official AWS documentation and hands-on exploration via the team's Confluence resources. For relational databases, I studied Amazon RDS (including Single-AZ and Multi-AZ high availability configurations, read replicas, and automated failover), Amazon Aurora (auto-scaling clusters and the performance advantages over standard RDS for OLTP workloads), and Amazon Redshift (columnar storage, Redshift Spectrum for semi-structured data, and its positioning as an OLAP warehouse rather than a transactional database). For NoSQL options, I studied DynamoDB (schema less design, partition key selection, and the trade-offs between strongly and eventually consistent reads) and DocumentDB (MongoDB-compatible API, collections vs tables, and DocumentDB's use at Citco for document-oriented workloads).

Neptune Graph Database and CitcoWorks Context: The Neptune graph database study was particularly relevant. I learned that CitcoWorks uses a Neptune graph database to store the full L1-to-L5 entity hierarchy of client, fund, and process relationships that governs event routing across the IA platform. I studied both Gremlin (property graph traversal using vertex/edge/property model) and SPARQL (RDF triple pattern matching for semantic data integration), understanding when each is appropriate: Gremlin for application-level graph traversal, SPARQL for standards-based semantic integration. This distinction proved relevant when I later worked on the RPA LLM Agent, which uses Neptune as a potential graph layer for Hybrid Graph-RAG retrieval.

Caching and Time-Series Databases: I also studied Amazon Timestream (serverless time-series database for IoT and operational metrics), Amazon QLDB (immutable ledger database for audit-grade transaction logs), and Amazon ElastiCache including both Redis (advanced data structures, pub/sub messaging, geospatial support, cluster mode with multiple primary nodes) and Memcached (simpler multi-threaded caching without data replication). The Redis study was directly applicable to the dashboard work: the IA platform uses Redis/Valkey clusters for caching, and I built the Redis monitoring row in Week 11.

CloudWatch and CloudFormation Foundations: CloudWatch and CloudFormation consumed the second half of Week 2. For CloudWatch, I studied the CMAA operational cycle (Collect, Monitor, Act, Analyze), the difference between EC2 1-minute detailed monitoring and 5-minute basic monitoring, log retention policy configuration, alarm types (metric alarms for single metrics, composite alarms for aggregated state), and EventBridge integration for alarm state routing. For CloudFormation, I covered the core IaC primitives: template structure (YAML and JSON), stack lifecycle management, and change sets for controlled update previewing. I immediately applied this knowledge in Week 3's first dashboard ticket.

By the end of Week 2, I had a working knowledge of the full AWS services landscape used by the IA platform, and I understood the CloudWatch and CloudFormation primitives I would be using extensively from Week 3 onward.

11.3 Week 3

Week 3 was when I transitioned from learning to active delivery, with my first assigned task: the CTI CloudWatch Pipeline Dashboard. This was a CloudWatch pipeline health dashboard for the rpacfs1-cais-pricing-extract-genai-infrastructure repository, specifically targeting the CAIS Pricing Extract (CTI) submission pipeline.

Dashboard Implementation and Metrics Instrumentation: I began implementing the dashboard. I first added CloudWatch metrics instrumentation to the rpacfs1-cais-pricing-extract-genai-infrastructure codebase to track CTI submission activity through the pipeline. Rather than placing metrics calls inline throughout the business logic, I refactored the instrumentation into a dedicated metrics module, which made the instrumentation testable and maintainable

independently of the pipeline logic. I created the Pipeline Health Dashboard itself with CloudWatch native widgets covering the core health signals for the CAIS pricing extract workflow. I updated the .gitignore to exclude virtual environment directories, pytest cache, and test artifacts—basic repository hygiene that the codebase was missing. I also updated the dashboard configuration to meet the acceptance criteria I had been given: 1-hour metric granularity, a 7-day time range, and auto-refresh enabled so the team would see live pipeline status without manual page refreshes.

Logs Insights Investigation Widget: I added a technical investigation layer: a CloudWatch Logs Insights widget that enabled operators to drill into raw Lambda execution logs when the health metrics indicated a problem. The Logs Insights widget used a pattern-matched query against the /aws/lambda/ log group prefix to surface error patterns directly in the dashboard, rather than requiring the operator to navigate to CloudWatch Logs and write a query manually. I added screenshots of the Logs Insights query setup to the pull request description so the team could reproduce the configuration in other environments.

Testing and Code Quality: I completed the testing phase. I wrote unit tests for the Logs Insights query functionality, CloudFormation template validation tests (checking that the template schema was valid before deployment), and ran the full test suite to confirm no regressions. I also applied code quality improvements identified while writing the tests: moving all import statements to the top of files following Python best practices, and adding type hints to function signatures to enable better IDE support and early error detection.

By the end of Week 3, the CTI CloudWatch Pipeline Dashboard was ready for pull request review. The dashboard covered the CTI pipeline health signals, the Logs Insights investigation widget was functional, the test suite was passing, and the repository was in a clean state for code review. This was my first complete delivery cycle at Citco—task assignment to implementation to tests to pull request—and it established the quality baseline I would maintain for all subsequent work.

11.4 Week 4

In Week 4, the focus was on completing the IA Lambda Monitoring Dashboard, addressing pull request review feedback, and delivering structured Lambda inventory documentation on Confluence.

Merging the CTI Dashboard and Pivoting to Lambda Monitoring: The week opened with the successful merge of the CTI CloudWatch Pipeline Dashboard into the release/SRE branch, closing the first JIRA ticket of the co-op placement. I immediately transitioned to the IA Lambda Monitoring Dashboard, which required building a monitoring solution covering all 51 Lambda functions across the rpacfs1 and ia-bpo platforms. Unlike the CTI dashboard, which targeted a single pipeline, the IA Lambda Monitoring Dashboard needed to serve the entire IA Lambda fleet with a single unified view.

Dashboard Architecture and Widget Design: I designed the dashboard using a modular YAML widget architecture where each widget type lives in its own file, allowing independent updates without touching surrounding logic. Widgets covered five monitoring domains: filter info with inline usage instructions, overview metrics (Total Invocations, Errors, Throttles across all functions), error analysis (bar charts and a CloudWatch Logs Insights investigation widget), duration metrics (Average, Max, and P99 latency), and operational health metrics (cold starts, concurrency, throttling, and memory utilization). The unit test suite reached 21 tests covering dashboard builder validation and widget contract enforcement.

Pull Request Review and Refinement: Mr. Leung conducted a detailed pull request review. His feedback focused on widget layout usability and filter propagation behavior. I addressed all comments, refined the widget grid, and added test cases for edge cases identified during the review. This review interaction introduced me to the team's standard for dashboard quality: functional correctness was necessary but not sufficient—the operational user experience mattered equally.

Confluence Lambda Inventory: In parallel with the pull request review, I created the Lambda Inventory Confluence page as a supporting subtask. The published page cataloged all 51 functions with trigger mappings (S3, SQS, EventBridge, API Gateway), memory configurations, timeout

settings, runtime versions, and budget_code allocations for cost tracking. This page became the authoritative reference for the team during subsequent dashboard and support work.

By the end of Week 4, the CTI dashboard was merged, the IA Lambda Monitoring Dashboard delivered a 51-function Lambda fleet view with a passing test suite, and the Lambda inventory documentation was live on Confluence—completing the part-time phase and setting the stage for full-time engagement.

11.5 Week 5

Week 5 marked the beginning of full-time engagement and was structured primarily as a formal knowledge-transfer week, with four dedicated sessions covering the platforms I would be monitoring and supporting throughout the co-op placement. It also included a critical production incident that would shape the entire direction of the Centralized Amazon Managed Grafana (AMG) Dashboard.

RPA Support Model Knowledge Transfer (Mr. Leung): Mr. Leung conducted a comprehensive session on the RPA operational model. I learned the Blue Prism and UiPath platform architectures, pool and queue mechanics, and the VPL queue monitoring cadence (checked at 5:00 PM EST daily). The session covered the Corporate Actions three-track workflow (Mandatory, Voluntary, and Ops Transfer), and the Follow-the-Sun escalation ladder that governs how incidents are handed off between APAC, EMEA, and NA teams. This session directly prepared me for the RPA support rotations beginning in Week 8.

CitcoWorks and Event Store (Mr. Murari): Mr. Murari introduced the CitcoWorks Knowledge Graph—a Neptune graph database storing the full L1-to-L5 entity hierarchy used across all Citco fund services. He walked through the Event Routing Service and Action Processor deployment model and provided hands-on guidance on the Event Store structured logging library. Understanding the mandatory fields (subject, process_identifier, run_id) proved essential when validating log events in later dashboard and LLM agent work.

Meridian and Æxeo Platform Sessions: The Meridian platform session covered the three-stage Kafka event pipeline (Event Routing Service to MSK topics, MSK to OpenSearch Consumer, OpenSearch to AXI), the six ECS clusters hosting Meridian services, and consumer lag as the

primary health signal. The Æxeo session explained cloud-based wire approval and fund movement via COMAPI and the Æxeo Reporter Server REST API—capabilities central to MESO and E-Binder reconciliation processes.

Dev Work and Meridian Production Incident: Despite the training schedule, I completed development work, refactoring the Lambda monitoring dashboard to a multi-dashboard architecture with explicit Lambda list filtering and adding error investigation capabilities. A Meridian production incident occurred while I was completing the final training session: a surge of automated events from Citco Recs congested the OpenSearch consumption layer, causing approximately nine million Kafka messages to accumulate and halting daily statement processing for 7.5 hours. The detection latency alone was 3.5 hours because no MSK consumer lag alarm existed. Observing the post-incident discussion in real time crystallized the monitoring gap I would spend the next ten weeks closing. I also formally scoped the Grafana AMG Dashboard work during this week.

By the end of Week 5, I had completed the formal KT program, delivered a multi-dashboard architecture refactor, and directly witnessed the production incident that established EstimatedMaxTimeLag as the primary MSK SLI and drove creation of the Centralized Amazon Managed Grafana (AMG) Dashboard.

11.6 Week 6

In Week 6, the focus was on extending the CloudWatch monitoring platform beyond Lambda to EC2, ECS, and EKS resources, creating the centralized ia-resources-monitoring repository, and formally scoping the Centralized AMG Dashboard.

Dashboard Extension and API Refactoring: The week opened with dashboard extension and API refactoring work: adding a metric filters generator, implementing dashboard export functionality with per-widget CSV files and a combined XLSX report, and converting the Lambda registry from a FastAPI service to a Flask application with Swagger documentation. I renamed `lambda_registry_api` to `resources_monitoring_api` to reflect its broader scope and added an S3 export endpoint with versioned Lambda list saving. I added trigger source detection to identify

whether each Lambda function is invoked by SQS, API Gateway, or EventBridge. The test suite reached 262 passing tests with a clean pyright status.

EC2, ECS, and EKS Resource Discovery: I extended the API to cover EC2 instances, ECS services, and EKS clusters. During EC2 discovery, I identified the `rpacfs1` tagging inconsistency—the fleet uses `budget_code="CFS/RPA"` rather than the expected `"rpacfs1"` value—and introduced the `AppName` fallback pattern as the standard workaround. I enriched the EC2 response models with `availability_zone` and `tags` fields. I refactored the registry package into per-resource submodules for long-term maintainability.

ia-resources-monitoring Repository Creation: On the most commit-dense day of the co-op placement with 39 commits across two repositories—I set up the centralized `ia-resources-monitoring` repository. I migrated all three monitoring packages (Lambda, EC2, and the API) from `rpacfs1-cais-pricing-extract-genai-infrastructure` into this dedicated repository. I established code quality tooling: Ruff linting, mypy type checking, deptry dependency hygiene, and pre-commit hooks. I introduced the `ia_paths.py` module to eliminate all `sys.path.insert` calls and created a unified `main.py` CLI entry point providing access to all deployment and dashboard generation commands. I merged the `release/SRE` branch to master, making this the first production deployment from the new repository.

Story Creation: Following the Week 5 Meridian production incident and the emerging need for a platform-wide observability solution, I met with Mr. Deotale to formally scope the Centralized Amazon Managed Grafana (AMG) Infrastructure Monitoring Dashboard. I created the story with requirements drafted to cover all Meridian-platform AWS services and address the MSK consumer lag gap that had caused the incident.

By the end of Week 6, the CloudWatch monitoring platform covered four resource types, the centralized repository was established and production-deployed, and the Centralized Amazon Managed Grafana (AMG) Dashboard was formally scoped and prioritized.

Technical Challenges Resolved: I resolved two infrastructure issues during the migration day. The `moto[all]` dependency triggered SSL certificate download failures on Windows; switching to specific service extras (`moto[s3,lambda,ec2]`) resolved this. CloudFormation YAML files

using !Sub and !Ref tags failed the check-yaml pre-commit hook; CFN files were excluded from that hook while retaining validation via cfn-lint. Both resolutions were documented in the CONTRIBUTING.md for future contributors.

11.7 Week 7

Week 7 was dedicated to completing the CloudWatch monitoring platform by delivering ECS and EKS dashboards, implementing structured logging across all packages, and pushing the overall test coverage above the 80% target.

ECS Monitoring Dashboard: I built the `ia_ecs_monitoring_dashboard` package from scratch, including health and performance dashboards, YAML widget definitions, CloudFormation templates, export functionality, and deploy support scripts. A key technical decision was replacing SQL arithmetic operations in health widgets with native `PendingTaskCount` metrics after discovering that CloudWatch Metrics Insights does not support arithmetic in certain widget modes. I added CPU and memory reservation widgets alongside a service inventory table, giving operational teams both capacity and inventory views in a single dashboard.

EKS Monitoring Dashboard: The EKS dashboard was completed, then substantially revised to switch from custom-namespace queries to the official AWS/EKS control plane metrics. The EKS discovery scope was subsequently narrowed: only one active EKS cluster (`uipath-eks-cluster-01`, AutomationHub `budget_code`) exists and it was idle at the time. The cluster was noted for future activation and the dashboard placed in a ready-but-deferred state. I split EKS widget YAMLs to one file per row for consistency with the ECS structure, and pushed test coverage for the EKS package to 81%.

Structured Logging and Test Coverage Milestone: I replaced all print statements and `disable_warnings` calls across the API, EKS, ECS, EC2, and Lambda packages with `structlog` structured logging, providing consistent log output across all components. I pushed the test suite from 67% to 81% coverage (830 tests) across three incremental pull requests, crossing the 80% target set in the IA Infrastructure Monitoring Platform acceptance criteria.

Dashboard Header Standardization and EC2 Polish: I standardized dashboard headers across all four resource types, enforcing a consistent visual language. EC2 dashboard improvements

included Fleet Status, EBS metrics, the AppName fallback for discovery, correct header height, Credit Balance widget on the health dashboard, and annotations with AVG line on the performance dashboard. I added trailing spaces to Markdown headers to ensure correct line-break rendering in CloudWatch text widgets.

By the end of Week 7, the CloudWatch monitoring platform was complete across Lambda, EC2, ECS, and EKS; 830 tests passed at 81% coverage; structured logging was consistent across all packages; and the repository was production-merged for the second time, closing the IA Infrastructure Monitoring Platform sub-stories.

Technical Decisions and Constraints: I confirmed a key CloudWatch Metrics Insights constraint during ECS duration panel work: the GROUP BY aggregation mode supports only AVG, SUM, MIN, and MAX —percentile functions (p50, p99) are not available in SQL mode. The Metrics Insights query parser required cluster names containing hyphens to be substituted *before* JSON serialization; applying substitution post-serialization resulted in double-escaped output. I documented both constraints in a technical decisions log to prevent reinvestigation in later weeks. Additionally, CloudWatch Logs Insights `put_query_definition` enforces a hard limit of 20 saved queries per account, which constrained the saved-query strategy to an Intelligent-Automation folder with errors-only scope.

11.8 Week 8

In Week 8, the focus shifted to the first RPA production support rotation while simultaneously scaffolding the Grafana AMG repository and completing final CloudWatch dashboard polish.

RPA Production Support Rotation: The first support rotation ran Monday to Friday, covering the North America shift from 11:00 AM to 7:00 PM EST. Each morning I reviewed the Confluence shift handover log from the APAC team, then monitored the SD Portal queue and Microsoft Teams channels for incoming incidents. Tickets handled across the week included: MESO process status inquiries (traced via CloudWatch logs using Run ID), FX Closeout retrigger requests (verifying the Daily565 folder and archiving old reports), Corporate Actions file mask updates for new client onboarding, an Auto Rec Download configuration request (backing up and editing `gridview.py` and `config.ini`), and a VPL process exception traced through UiPath Orchestrator transaction details. I

posted end-of-shift handover notes to Confluence each evening. Following the Week 5 training and this first live rotation, the RPA support model became familiar and manageable alongside development work.

AutomationHub budget_code and Dynamic Tag Discovery: During quieter periods in the support queue, I completed two CloudWatch platform JIRAs. The first added AutomationHub budget_code support across all discovery packages, enabling dashboards to surface UiPath-platform resources alongside the existing Meridian and ia-bpo resources. For the second, I implemented a tiered resource matching strategy: primary matching on budget_code tag value, falling back to AppName tag value, and finally a key-agnostic scan that matches any tag whose value contains the platform identifier—regardless of the tag key name. I also added multi-region and multi-account query parameters. Together, these changes made the resource API significantly more resilient to infrastructure naming inconsistencies across the five Citco AWS accounts.

Grafana AMG Repository Scaffold and Agentic-AI Architecture: Following Mr. Deotale’s decision to elevate the dashboard project to **Critical** priority in the post-incident review, I scaffolded the ia-it-sre-infrastructure-monitoring-amg repository. The initial commit established a multi-agent project structure: LF line-ending enforcement via .gitattributes, pre-commit hooks with code quality tooling, and a structlog structured logging and agent audit trail. I implemented agent skill definitions and role scaffolds for the multi-agent orchestration layer. I upgraded the Python version from 3.11 to 3.14, documenting GIL removal and free-threaded mode status for the new runtime.

By the end of Week 8, the first support rotation was complete, the AutomationHub budget_code integration and dynamic tag discovery closed the CloudWatch platform feature set, and the centralized AMG dashboard repository was scaffolded and ready for the full infrastructure build beginning in Week 9.

Key Architectural Decision: The discovery of the CitcoWorks and Meridian platforms sharing the “Shared Cloud Artifacts” budget_code was the most significant architectural finding of the week. A naive single-stage tag filter would conflate operationally unrelated resources from two distinct platforms. I established the AppName discriminator as a mandatory second filter stage

for these two platforms; this pattern became the standard two-stage discovery architecture used throughout the inventory API in subsequent months. Additionally, five of the six Meridian ECS clusters had no resource tags at all, requiring a cluster-name-based fallback strategy to ensure complete fleet coverage.

11.9 Week 9

Week 9 was the most infrastructure-dense week of the co-op placement, involving the full deployment of the Grafana AMG stack, the first live CloudWatch alarms, a forced Grafana version migration, and the construction of the first three dashboard rows.

Infrastructure Deployment: I deployed the full centralized AMG dashboard infrastructure in a single sustained session spanning 49 commits from 9:59 AM to 9:05 PM. I began by enforcing coding standards (uv, JavaDoc-style docstrings, strict type hints) and implementing the Human-In-the-Loop gate with trigger rules and an audit trail for agent oversight. I then built the CloudFormation templates in sequence: Grafana AMG workspace with IAM role, Lambda custom resource for Grafana API provisioning, the Grafana dashboard JSON covering all ten rows, a CloudWatch alarms template with all 17 alerting rules, and the multi-environment CodePipeline with ChangeSet deployment and EventBridge trigger. I created a dedicated Grafana CFN execution role to resolve IAM bootstrap permission issues, and I added the `IamPermissionResolver` agent to automatically diagnose and fix permission errors in future deployments. The day concluded by importing the Meridian dashboard via the Lambda provisioner and storing its JSON in Secrets Manager. With this deployment complete, the MSK consumer lag alarm was live against production traffic—the alarm that would have detected the April Meridian production incident was now operational.

Documentation Sprint and 100% Test Coverage: I delivered the complete documentation suite: release notes, installation guide, architecture document (PNG diagrams replacing Mermaid blocks for CodeCommit rendering compatibility), process flow document, contribution guide, permission fixes guide, and a detailed user guide for all ten dashboard rows. The test suite reached 99% coverage on the agentic-AI subsystem, then 100% coverage on `trigger_rules` and `orchestrator`—a milestone that provided confidence in the agent dispatch layer.

SQS Panel Fixes and Grafana 12.4 Upgrade: the SQS row panels (Queue Depth, Messages In-Flight, Age of Oldest Message, DLQ Depth) were fixed with per-queue breakdowns and QueueName legends, achieving 100% test coverage on the Row 2 SQS panel tests. Amazon automatically updated the AMG workspace, requiring an unplanned migration from Grafana 10.4 to 12.4. Resolving the breaking changes in the Metric Insights SQL editor and template variable syntax took approximately one full working day, migrating all existing panels to the stable 12.4 query model and updating the schema version to 41. This week also delivered the full 23-agent architecture (including cloudwatch_alarms_expert, amg_deployment_expert, and amg_observability_expert), resolved the Grafana API key bootstrap chicken-and-egg issue, and implemented the SQS Health gauge with composite critical logic.

Lambda Row and ECS Row Completion: I overhauled the Lambda monitoring row: per-function breakdown for all panels using the Metric Insights SQL GROUP BY format, split Duration into p50 and p99 panels, paired Throttles with Concurrency, and introduced the \$function variable for interactive function-level drill-down. I applied the lda- naming prefix to Lambda-specific resources and removed the ECS Container Insights Lambda after confirming native Container Insights was sufficient.

By the end of Week 9, the full Grafana AMG infrastructure was deployed and live, the MSK consumer lag alarm was operational, I had built dashboard Rows 1–3, and the 23-agent architecture was established with 100% test coverage across the core orchestration layer.

Grafana API Bootstrap and SQS Thresholds: An important infrastructure challenge was the Grafana API key chicken-and-egg problem: the provisioner Lambda needed an API key to provision resources, but the key had to be created in Grafana which did not yet exist. A hotfix series resolved this by bootstrapping the key separately as a one-time setup step before the full deploy. I configured the SQS Health gauge composite logic with precise thresholds reflecting the Meridian queue operating range: 0–34,999 messages (OK, green), 35,000–49,999 (WARNING, amber), and 50,000+ (CRITICAL, red). These thresholds were derived from the April Meridian production incident peak queue depth of approximately nine million messages, setting the warning level well below the incident threshold while avoiding false positives during normal processing. A three-hour knowledge transfer workshop with Mr. Deotale, Mr. Nathaniel, and two additional team members

provided deep context on the Meridian operational model that informed dashboard panel design decisions for the remainder of the co-op placement.

11.10 Week 10

In Week 10, the focus was on building the remaining middle rows of the Grafana dashboard, resolving CloudWatch Metrics Insights SQL limitations, and migrating dashboard storage from Secrets Manager to S3 to overcome payload size constraints.

ECS, MSK, DocumentDB, Neptune, and OpenSearch Rows: The week opened with Row 5 (ECS APIs, ALB metrics) and Row 6 (MSK/Kafka). Row 5 required a multi-iteration approach to the `$platform` variable in `filterFieldsByName`: ALB resource names do not contain the platform string, requiring a split between Row 4 (ECS container health) and Row 5 (ALB traffic metrics). I migrated Row 6 MSK to Metric Insights SQL with per-broker and per-consumer-group breakdown and added a Meridian cluster filter. I completed Rows 7–9 (DocumentDB, Neptune, and OpenSearch) with SQL migration and per-instance breakdowns. I added a new Aurora/RDS Row 8 and renumbered the existing rows to accommodate it. I introduced the SQS Inventory table panel with clickable AWS Console links to reduce incident response navigation time.

Metric Insights SQL Percentile Limitation: During Lambda and ECS duration panel work, I confirmed a fundamental Metric Insights constraint: GROUP BY mode supports only AVG, SUM, MIN, and MAX aggregation functions—p50 and p99 percentiles are not available via SQL. I implemented a wildcard dimension workaround (`FunctionName: ["*"]`) for Lambda duration percentile panels, accepting the tradeoff that idle functions would appear in the list. I documented this constraint as a known limitation to prevent future reinvestigation.

Secrets Manager 64 KB Limit and S3 Migration: By late Week 10 the CloudFormation provisioner payload grew to approximately 14,720 lines as additional alarm templates were added, triggering `SecretSizeExceededException` from Secrets Manager. I migrated all dashboard JSON and alarm templates to S3 object storage with AES-256 server-side encryption to comply with the CITCO-S3ForceEncryptNoPublic bucket policy. I updated the provisioner Lambda to fetch content via `s3:GetObject` at deploy time, removing the 64 KB ceiling entirely.

Agentic AI System Refactor and 597-Test Milestone: The agentic AI subsystem underwent a major refactor: I regrouped agents into five capability tiers (infra, grafana, development, workflow, safety), added the `aws_config_troubleshooter` role, and fixed all subprocess-dependent tests to prevent hanging in CI. The test suite reached 597 tests at 100% coverage—the largest single-day test milestone of the co-op placement. I incremented `ProvisionerVersion` to 355 after two sequential deploys validating the refactored architecture.

By the end of Week 10, Rows 1–9 of the dashboard were built and deployed, the Secrets Manager storage constraint was resolved via S3 migration, and the percentile limitation was documented.

Dashboard Variable Architecture: I implemented the `$platform` variable as a cross-row filter using `filterFieldsByName` with a pipe-formatted multi-value pattern. This allowed a single variable selection to filter data across 13 rows simultaneously, reducing the operational burden of context-switching between platform-specific views. The SQS inventory table was introduced with clickable AWS Console links using Grafana data links, enabling one-click navigation from queue depth panels to the SQS console page for that specific queue. The Aurora/RDS row was added as an additional row, requiring all subsequent row numbers to be incremented by one—the `renumbering.md` migration document established a convention for future row additions.

11.11 Week 11

Week 11 was dedicated to completing the database monitoring rows, delivering the alarm and notification infrastructure, and bringing the dashboard to a state ready for stakeholder review.

Health Panel Completion for All Database Services: I added health panels for DocumentDB (CPU utilization with 60/80% thresholds), Neptune DB (same threshold structure), RDS/Aurora (CPU thresholds with row reordering), and OpenSearch (Green/Degraded status derived from `ClusterStatus.green` metric). The OpenSearch health panel was particularly significant: OpenSearch was the bottleneck service in the April Meridian production incident, and having a dedicated health indicator for it addressed the primary observability gap identified in the post-mortem. I enhanced ECS metrics to show CPU and memory usage MAX per service ordered by `MAX() DESC`, making the busiest services immediately visible at the top of each panel.

Redis/Valkey Row and Dashboard Visual Polish: The Redis/Valkey row was completed with 11 panels covering the four golden signals (latency, traffic, errors, saturation) and fully compliant with the AMG 1-query-per-panel constraint. I renumbered row files to sequential order row_01 through row_13 for consistency. I refactored the CLI to replace the `--dry-run/--validate-only` flags with a unified `--validate` flag and a new `--smoke-test` option. Visual polish across the dashboard included Retro LCD bar gauges, consistent series colors across all timeseries panels, and `renameByRegex` transformations for clean legend names. I fixed MSK Row 6 to replace Bytes In with Messages Out per second for more meaningful throughput visibility.

Datasource Analysis and CloudWatch Logs Integration: I completed a datasource analysis to determine the full set of Grafana datasources required, verifying the datasource requirements using Amazon Q. I added CloudWatch Logs and OpenSearch as new datasources alongside the existing CloudWatch Metrics datasource. This enabled Log-derived panels to be added to the dashboard in subsequent weeks.

Alarm and Notification Infrastructure: The alarms and notifications layer was delivered across three days. I created a comprehensive alarms and notifications guide, performed an alarms gap analysis documenting existing versus required coverage, parameterized the Platform variable in CloudWatch alarm names, added a python `main.py` alarms CLI command for SNS subscription management, and secured both SNS topics with `KmsMasterKeyId alias/aws/sns` encryption. I implemented EventBridge + Lambda integration (Option A) for real-time CloudWatch alarm state in Grafana, replacing the polling approach (Option B) after evaluating both options against latency and cost criteria. I deployed an SNS Re-Notifier Lambda for ongoing alarm state reminders, configured composite alarms to aggregate per-service states into platform-level OK/ALARM indicators, and built the alarm state history LCD panel with severity-based colors spanning the full dashboard time range.

By the end of Week 11, all 13 dashboard rows were built and deployed, the OpenSearch JVM heap pressure alarm was operational, the SNS Re-Notifier Lambda was live, and the dashboard was ready for the Week 14 stakeholder review and final sign-off.

Alarm Architecture Decision (EventBridge vs Scheduled Lambda): I evaluated two approaches for real-time alarm state delivery to Grafana panels: Option A (EventBridge + Lambda) and Option B (Scheduled Lambda polling at 1-minute intervals). I chose Option A for its event-driven nature—alarm state changes are delivered within seconds of the CloudWatch alarm transition, versus up to 60 seconds of lag with the polling approach. The cost difference was negligible: EventBridge charges per rule invocation (~\$0.01/1M events) versus Lambda invocations at 1-minute intervals (~\$0.02/month). The EventBridge approach also aligned with the existing CodePipeline trigger pattern already used in the repository. Both SNS topics were secured with KMS encryption using alias/aws/sns to comply with Citco encryption-at-rest requirements. The first major stakeholder review confirmed the dashboard visual language and row organization were acceptable to the team before the final build sprint began.

11.12 Week 12

In Week 12, the focus was the second RPA production support rotation alongside finalizing the AMG dashboard alarm infrastructure and completing EC2 dashboard rows.

RPA Production Support Rotation: The second rotation followed the same North America shift pattern (11:00 AM–7:00 PM EST). I began each morning by reviewing the Confluence handover log from the Manila team, then monitored the SD Portal queue, the Innovation IT email inbox, and the IA_INCIDENTS Teams channel throughout the day.

Tickets handled across the week included: MESO process status inquiries traced through CloudWatch logs using Run IDs—I identified a stuck process by correlating the Run ID from the JIRA ticket with the CloudWatch log stream for the relevant Lambda function; FX Closeout retrigger coordination: I verified the Daily565 folder state, archived stale reports, and triggered the UiPath job to prevent duplicate processing; the AVCAB date format exception: I investigated the CloudWatch logs, confirmed an incorrect input date supplied by the requester, and asked them to retrigger after correction; and an Auto Rec Download CAPULA2 configuration update: I backed up gridview.py and config.ini, added the ASPECT client configuration block, and restarted the service.

Mr. Leung conducted knowledge transfer sessions spanning 35 minutes on Monday, 27 minutes on Tuesday, and 88 minutes on Friday, covering RPA support procedures and post-incident RCA walkthroughs for the MESO ECS autoscaling failure I encountered this week. During the support shift I also identified the MESO ECS autoscaling gap: the ECS service autoscaling policy was unconfigured, causing tasks to accumulate rather than scale out. I escalated the findings and the development team applied a code fix, and I added an ECS task saturation alarm to the alarm set as a direct result of this incident.

I closed stale tickets exceeding 14 days without end-user response per the IA Production Support guide and posted end-of-shift handover notes to Confluence each evening, ensuring the Manila team had complete context to continue any open items.

Alarm Tuning and CloudFormation Improvements: During quieter support periods I tuned CloudWatch alarm thresholds against actual DEV baseline metrics. For DocumentDB and Neptune I validated the CPU utilization thresholds (60% warning, 80% critical) against historical metric data to confirm they were appropriate for normal operating conditions without generating false positives. I improved the CloudFormation deploy script by adding a `--preview` mode that executes a change set and displays the planned resource changes before applying them, and added change set status reporting so the CI/CD pipeline could distinguish a real deployment from a no-op change set (where no resources required updating). All 14 dashboard rows and 17 CloudWatch alarms were validated end-to-end in the DEV environment, confirming alarm state transitions fired correctly and SNS email delivery was working.

EC2 Dashboard Rows: I completed EC2 monitoring Rows 7–8 with the AppName fallback pattern applied across all 83 instances. The most significant correction was fixing the EBS metric namespace: the initial implementation was querying `DiskReadOps` which does not exist in the AWS/EC2 namespace—the correct metric is `EBSReadOps`. I also added Fleet Status panels showing instance state distribution across all platforms, and per-instance drill-down panels using Metric Insights SQL `GROUP BY InstanceId` for CPU, memory (via CloudWatch Agent), network I/O, and disk I/O. Test coverage for the EC2 dashboard package reached 96% across all widget types and discovery logic.

By the end of Week 12, the second support rotation was complete, the AMG dashboard alarm infrastructure was validated, and EC2 dashboard rows were merged to release.

11.13 Week 13

Week 13 was the most incident-dense week of the co-op placement, beginning with an emergency hotfix for a regulatory deadline and continuing with significant AMG dashboard structural improvements.

CAIS Deadline SpnegoError: Hotfix: The week opened with a critical failure in the CAIS Deadline Lambda—an RPA automation that uploads Standard Allocation Sheets to the PDS database and generates consolidated pricing reports for alternative investment clients—`SMBAAuthenticationError: SpnegoError` during UCM upload, blocking a regulatory CAIS filing. A seven-hour investigation identified the root cause: `constants.py` was caching credentials at module import time, meaning a Secrets Manager rotation that had left empty values during container startup caused all subsequent authentication attempts to fail silently using the pre-loaded empty credential. I applied three targeted fixes: fail-fast UCM credential validation at startup and before each SMB connection attempt; a guard preventing empty `deadline_ids` from generating an invalid SQL IN clause; and a zip file existence check before attempting the UCM upload. I added comprehensive unit tests for all three fixes, including mock patches for `trigger_process` and `email_function`, both of which had been preventing test isolation. The CAIS filing completed before the regulatory deadline and a full RCA was documented on Confluence.

Per-Panel Folder Refactor and SQS Dashboard: Following the hotfix, I completed a structural refactor of the AMG dashboard repository: I extracted 110+ dashboard panels into per-panel YAML folders under `rows/row_02/` through `row_14/`, enabling independent panel updates without touching surrounding files. I deployed an EC2 mapper Lambda with an EventBridge trigger, and the SQS monitoring dashboards went live (960 tests passing). The IA Infrastructure Monitoring Platform SQS feature was formally closed.

Inventory API Scaffold: I used a Kiro spec-driven workflow to scaffold the `ia-it-sre-infrastructure-inventory` repository in 2.5 hours: Flask async API structure, discoverer registry pattern, Pydantic response models, and alignment of `service_type` names with the dashboard rows.

By the end of Week 13, the CAIS regulatory filing was unblocked, the AMG dashboard reached 357 tests on the refactored architecture, and the inventory API foundation was established.

CloudFormation Template Size Constraint: I encountered a second infrastructure size constraint when the full CloudFormation alarm template could not be deployed inline—the 51,200-byte inline template limit was exceeded. Rather than splitting into nested stacks, I decided to upload templates to S3 and use TemplateURL references, keeping the deployment model flat and avoiding the CloudFormation nested stack dependency chain that had caused rollback issues in earlier test environments. The s3:ListBucket permission was required alongside s3:GetObject to produce meaningful error messages on NoSuchKey failures during template retrieval.

11.14 Week 14

Week 14 delivered the final AMG dashboard deployment and formally paused the project. This was the culmination of six weeks of intensive dashboard development and represented the single largest task in my co-op placement at 140 logged hours.

Deep-Link Panels for All Service Rows: I added deep-link panels to all nine monitored service rows (Lambda, SQS, ECS, MSK, DocumentDB, Neptune, RDS, Redis, OpenSearch), connecting each resource panel directly to the relevant AWS Console page for that specific resource. I used Grafana data links with `${__data.fields.name}` field interpolation to construct AWS Console deep-link URLs dynamically for each row. This addressed a recurring friction point I had experienced during Week 8 and Week 12 support rotations: incident responders were manually navigating to the CloudWatch console to cross-reference alarm states with resource metrics, losing context with each tab switch. I also added resource inventory tables for all nine service types, each displaying key configuration metadata (memory, timeout, runtime for Lambda; cluster size and engine version for DocumentDB; instance class and Multi-AZ status for RDS) directly in the Grafana panel so operators could diagnose configuration issues without leaving the dashboard.

Row Restructuring and Visual Consistency: I restructured the DocumentDB, Redis, and OpenSearch rows to a consistent 4+4 (w=6) panel layout, standardizing panel widths and eliminating asymmetric gaps that had accumulated during the iterative row-by-row build approach. I corrected ECS bargauge legend names to display human-readable service names

rather than raw ARN strings, and fixed ECS deep-link URL formats to use the correct AWS Console path structure. I documented the Grafana transformation ordering requirement for the alarm state history panel after discovering that applying `seriesToRows` before `filterFieldsByName` produced incorrect output, whereas reversing the order worked correctly.

I added ten new dashboard template variables to complete row-level interactive filtering across all nine services: `lambda_function`, `sqs_queue`, `ecs_service`, `ecs_target_group`, `msk_consumer_group`, `docdb_instance`, `rds_instance`, `redis_cluster`, `neptune_instance`, and `opensearch_domain`. This completed the interactive filtering model: every row now had both platform-level (`$budget_code`) and resource-level filtering. The `ProvisionerVersion` was incremented from 698 to 726 across five sequential deploys during the restructuring session alone, reflecting the iterative nature of the final polish work.

Dashboard v728 Merge and Project Closure: I conducted the final deployment session, incrementing from v726 to v728 with two final fixes. All four acceptance criteria were verified before the release merge: MSK consumer lag alarm live and firing against production traffic; all 14 dashboard rows rendering correctly in the AMG workspace with no broken panels; deep-links functional for all nine service types across DEV, UAT, and PROD environments; and alarm email delivery confirmed to the configured SNS distribution list. I merged the release pull request from `release/SRE` to `main`, and I paused the Centralized AMG Dashboard project—the largest single task of the co-op placement, logged at exactly 140 hours across seven weeks of intensive development.

11.15 Week 15

Following the AMG dashboard release, Week 15 focused on post-release improvements to the MSK row and the delivery of six new resource mapper Lambdas.

MSK Consumer Lag Refinements: I enhanced MSK consumer lag panels with per-topic breakdown (previously only per-consumer-group), implementing a `GROUP BY ClusterName + ConsumerGroup + Topic` triple-axis structure to enable topic-level drill-down. I corrected the MSK cluster ARN chain regex from Python to JavaScript syntax after discovering it was preventing the `$msk_cluster_arn` variable from resolving in Grafana's template engine. I fixed the MSK performance panel topic

deep-link URL format to use the correct AWS Console path structure, and enforced stable panel IDs to prevent deep-link breakage across dashboard red deploys.

Six Resource Mapper Lambdas: I deployed six new mapper Lambda functions to the resource-mappers CloudFormation stack, each responsible for discovering and caching one resource class: ECS target groups, DocumentDB clusters, RDS instances, Redis cache nodes, Neptune clusters, and OpenSearch domains. These mappers replaced the previous SQS Queue Mapper's name-pattern approach with a tag-based discovery strategy, enabling Grafana \$variable dropdowns to populate dynamically from live AWS inventory rather than from static configuration lists.

Knowledge Transfer Sessions (Mr. Leung): Mr. Leung provided a 60-minute session on the FX Closeout process exception handling procedure, covering how to identify settled-vs-live position mismatches and the correct retrigger sequence. Mr. Leung held a follow-up session on FX Settled exception handling the following day. I documented both sessions in Confluence for the next support rotation team member.

By the end of Week 15, the MSK row was production-quality, six mapper Lambdas were live and populating dashboard variables, and support procedures for FX Closeout were formally documented.

SQS Platform Coverage Resolution: I resolved a critical coverage gap with a case-insensitive regex fix: the ia-bpo platform was returning only 2 of its 55 queues due to an uppercase/lowercase mismatch in the queue name pattern. I abandoned an attempted Infinity plugin approach (direct S3 variable population) after discovering an unresolvable Go HTTP client timeout in the plugin's transport layer. The SQS Queue Mapper Lambda approach replaced a 5-minute EventBridge schedule with CloudTrail-triggered invocations on CreateQueue and DeleteQueue API calls, reducing Lambda invocations from approximately 8,640 per month to 2–5 per month while ensuring the queue inventory stayed current within minutes of infrastructure changes. I implemented a content-hash S3 key scheme for Lambda deployment zips so CloudFormation correctly detected code changes between deploys—the prior filename-only approach caused silent no-op deploys of updated Lambda code.

11.16 Week 16

Week 16 was the third RPA support rotation, coinciding with Canada Day, alongside completion of ECS cluster discoverers for the inventory API.

RPA Production Support Rotation: During Week 16 support, I handled several notable requests. I completed two Corporate Actions new client onboarding tasks: Leaside (LEESIDE_Trades_HFXCA added to BPSHare Mandatory/Voluntary/Ops) and Orchid Tree (OrchidTree_trades_HFXCA added across all three tracks). An OTC process inquiry for Lumyna Equity Funds revealed a dependency on the MNL team for configuration; I coordinated with Mr. Miko and Mr. Daniel for knowledge transfer. On Canada Day (Wednesday), I coordinated an INNOCAP SAMLMOS MESO retrigger with Mr. Leung: the process had failed overnight, I manually marked it failed on the reporting server, and retrIGGERED it to completion. Mr. Leung conducted two extended knowledge transfer sessions: 80 minutes on Thursday and a joint 96 minute session with Mr. Deotale on Friday covering the full RPA support handover model.

ECS Variable and Health Panel Optimization: During quieter periods, I added an ecs_cluster dashboard variable for cluster-level filtering, and optimized health stat panel queries by removing the seriesToRows transform and replacing it with ORDER BY MAX DESC. This change made the busiest services appear at the top of each panel without requiring a client-side sort transform.

ECS and S3 Cache Layer: I implemented ECS cluster discoverers for all 22 clusters using the two-stage filter (budget_code then AppName discriminator) established in Week 6. I added a bi-weekly EventBridge-triggered S3 cache layer to the inventory API, reducing per-request latency from the 10-second boto3 enumeration to a sub-second cache read. I drafted a four-part log group observability solution covering log group discovery, API endpoint, Grafana panel, and saved query generation.

By the end of Week 16, the third support rotation was complete with new client onboarding delivered, ECS discoverers were in production, and the inventory API S3 cache was operational.

Canada Day Operational Coverage: Working through the Canada Day public holiday was necessary to maintain the INNOCAP SAMLMOS MESO process continuity. The process had failed overnight and required admin-level intervention to mark the queued items as failed in

the MESO database before retriggering. Mr. Leung provided admin access and live KT on the marking procedure during a remote session. This experience directly motivated adding ECS task health monitoring to the dashboard alarm set—specifically the “desired vs running task count” discrepancy alarm that would surface this class of silent failure automatically. The two-stage budget_code + AppName filter pattern proved essential again this week: 22 ECS clusters were discovered, but the Meridian and CitcoWorks clusters sharing a budget_code prefix required the AppName discriminator to be classified correctly for the dashboard rows.

11.17 Week 17

Week 17 delivered the inventory API to production readiness and established the X-Ray distributed tracing architecture plan.

Inventory API v1.5.0 and CI/CD Pipeline: I migrated the inventory API from a Flask monolith to four dedicated Lambda handlers behind API Gateway (v1.5.0). I added a /docs endpoint with embedded Swagger UI, and implemented an async background refresh mechanism to avoid the 30-second API Gateway timeout on full inventory scans. I introduced the /refresh endpoint supporting per-platform, per-service, and full refresh targeting. I added a full CI/CD pipeline for the citcodevops01 account with a deployment guide. I renamed all AWS resources to the ia-it-sre-resources-inventory-* naming convention for platform consistency. I applied a VPN-graceful SSL degradation fix (v1.1.1/v1.1.2) after discovering the S3 cache crashed when run outside the Citco network.

X-Ray Distributed Tracing Architecture: I attended BPM Flight Control meetings (Monday and Friday) and a Blue Prism Troubleshooting session (Thursday), which informed the X-Ray architecture approach. I drafted the full tracing plan covering seven service categories: Lambda (using Powertools Tracer), ECS (ADOT sidecar), EKS (DaemonSet), API Gateway (active tracing), SQS (trace header propagation), MSK (trace propagation), and EC2 (standalone daemon). I compiled a team-requests.md document comparing structured vs unstructured logging patterns and documenting the recommended approach for the ia_meso_process codebase.

VPL IPV CheckpointException Follow-up: I identified the VPL IPV Aexo pricing window CheckpointException pattern as systemic across multiple VPL workflows targeting dynamically

loading AExeo screens. I estimated the fix at 16 hours for selector resilience work and escalated it to the development team due to BitBucket repository access restrictions.

By the end of Week 17, the inventory API was production-deployed at v1.5.0 with CI/CD, the X-Ray architecture was formally planned across all seven service categories, and all 83 EC2 instances were confirmed in the live inventory.

Infinity Datasource Migration: I migrated fourteen dashboard variables from the deprecated mapper Lambda approach to the Infinity datasource via CloudFront, eliminating the SigV4 authentication complexity that had prevented direct S3 access from the AMG workspace. CloudFront served as an unsigned proxy with S3 origin access control, making the inventory JSON available to Grafana without any client-side AWS credential management. I set the X-Ray datasource UID to the stable value xray-ia-it-sre-dev and fixed the CloudWatch provisioner UID to cloudwatch to match the 100+ panel references already embedded in the assembled dashboard JSON. I refactored the inventory API into four sub-packages (core, api, infra, metrics) and incremented it to v1.4.0, adding seven S3 lifecycle expiration rules per platform/service combination in v1.4.3 to manage storage costs.

11.18 Week 18

Week 18 delivered the alarm email log group deep-link system and completed the five-phase log group observability specification.

Alarm Email Log Group Deep-Links (log group observability feature Phase 2): I extended the alarm formatter Lambda to auto-construct CloudWatch Logs Console deep-link URLs from the cached log group inventory. When an alarm fires, the formatted email now includes a direct link to the relevant log group filtered to the alarm timestamp, eliminating the need to manually navigate to Logs Insights and reconstruct the query. I tested the log group URL encoding utility for all seven platform types, handling the differences in log group naming conventions between Lambda, ECS, EC2, and MSK resources.

Grafana CloudWatch Logs Panel: I implemented a placeholder Row 17 panel in the Grafana dashboard using the CloudWatch Logs datasource, with a \$platform variable filter. This provided a foundation for the full X-Ray tracing row planned for delivery in later weeks.

Five-Phase Log Group Observability Specification: I finalized the log group observability feature specification covering: Phase 1 (log group discoverer Lambda), Phase 2 (inventory API /api/log-groups endpoint), Phase 3 (alarm email deep-links), Phase 4 (Grafana log panel row), and Phase 5 (CloudWatch Insights saved queries). I tracked all five phases as individual subtasks on the JIRA board.

Async Inventory and Pydantic Models: I validated the inventory API async route handler architecture: `asyncio.gather()` dispatching all discoverers concurrently via `ThreadPoolExecutor` kept full-fleet enumeration under the 10-second non-functional requirement. I added twelve service-specific Pydantic response models with versioned S3 keys (latest.json plus timestamped history entries).

By the end of Week 18, alarm emails included actionable log group deep-links, the inventory API reached 99% code coverage with a 453-test suite, and the log group observability specification was fully defined.

Dynamic Alarm Architecture and X-Ray Instrumentation: I replaced the 17 static alarm definitions with a data-driven approach: on each deploy the provisioner Lambda reads the full resource inventory from S3 and creates or updates per-resource CloudWatch alarms automatically. I added nine alarm types absent from the original static set, including Lambda error rate (using a CloudWatch math expression rather than a metric alarm), MSK consumer lag (`SumOffsetLag` with thresholds 1,000/5,000), and EC2 instance and system status checks. I throttled concurrent alarm creation to two `ThreadPoolExecutor` workers with an adaptive retry strategy of up to 10 attempts, resolving `PutMetricAlarm` throttling that had previously caused partial alarm set deployments. Bundling `aws-xray-sdk` with `patch_all()` auto-instrumentation grew Lambda deployment zip sizes from 9 KB to 170 KB. I established the X-Ray gap analysis baseline: CloudWatch monitoring alone provided approximately 30% parity with Dynatrace; adding X-Ray, Application Signals, and the full Grafana stack would raise this to 88%.

11.19 Week 19

Week 19 delivered the CloudWatch Logs observability layer including 30 saved queries across all monitored platforms.

Saved Queries Generation Script (log group observability feature Phase 5): I wrote `create_saved_queries.py` to programmatically create 30 CloudWatch Logs Insights saved queries, covering six query types (recent errors, Lambda cold starts, request tracing, performance analysis, audit events, and resource utilization) across seven platforms. The script queries the live log group inventory to resolve group names at creation time, ensuring queries target the correct log groups even if naming conventions change. Running the script in DEV confirmed 748 matched log groups from a total of 3,423 groups in the account.

Log Group URL Utility and Alarm-to-Log-Group Mapping: I extracted the `shared/log_groups.py` module into the shared utilities package, providing URL encoding, service classification from log group name patterns, and an alarm-to-log-group matching function used by the alarm email formatter. I merged the alarms-notifications feature branch to release, making the full alarm notification pipeline (EventBridge state publisher → SNS formatter → HTML email with log group deep-link) available in the DEV environment.

SRE Extended Review Sessions: I held two 60-minute SRE extended review sessions (Monday and Tuesday) with Mr. Deotale and the wider Innovation IT team. In these sessions, we covered the full observability platform architecture, confirmed the resource inventory coverage, and reviewed the X-Ray implementation roadmap.

By the end of Week 19, the CloudWatch Logs layer was operational with saved queries for all platforms, I had merged the alarm-to-log-group pipeline, and the SRE extended review confirmed the platform architecture was sound for the upcoming X-Ray tracing work.

Cross-Platform Variable Filter Fix: I discovered and fixed a critical dashboard correctness bug: when a multi-select Grafana variable had an empty selection, `filterFieldsByName` was matching rows from all platforms rather than returning no results. I added `allValue=^$` with a `__no_resources__` sentinel value across all 13 multi-select variables, ensuring empty selections produced empty panels rather than misleading cross-platform data leakage. I renamed the inventory API project from `resources-inventory` to `infrastructure-inventory` to align with the platform-agnostic naming convention that I adopted in Week 17, requiring updates across all Lambda names, CloudFormation stacks, API Gateway stages, documentation, and AWS Console

bookmark links. I grew the test suite to 562 tests at 99% coverage (v1.5.1) by adding 50 Lambda API handler unit tests and 48 additional integration tests.

11.20 Week 20

Week 20 was the fourth RPA support rotation, combined with authoring the full X-Ray distributed tracing specification suite and the SRE Platform v2.19.9 release.

RPA Production Support Rotation: I handled the following tickets this week: FX Closeout retrigger for PICTET_RPA (checked UiPath folder, archived old Daily565 reports, triggered and notified Ms. Kartina on completion); Auto Rec Download ASPECT client configuration (backed up gridview.py, added transaction and position columns with Mr. Leung verification); MESO cash completeness for Johansen funds (verified DB state, obtained admin access from Mr. Leung to mark pending items as failed on the reporting server, retriggered to completion); TUDOR3 daily reconciliation report (monitored Run ID progress, provided updates, and contacted Mr. Jay for an optimized query). I closed inactive tickets exceeding the 14-day threshold per the Production Support guide. Mr. Leung conducted KT sessions: 47 minutes on Monday, 43 minutes on Wednesday (group with Mr. Deotale and Mr. Edmund), and a 110 minute session with Mr. Jiannan covering full RPA KT for the incoming team member.

X-Ray Tracing Specification Suite: I authored seven X-Ray tracing sub-task specifications for the `ia_meso_process` repository: Lambda Powertools X-Ray tracing specification (Lambda Powertools Tracer with `@capture_method` decorators), ECS ADOT sidecar tracing specification (ECS ADOT sidecar with OpenTelemetry auto-instrumentation), EKS DaemonSet tracing specification (EKS DaemonSet collector), API Gateway tracing specification (API Gateway active tracing), SQS trace propagation specification (SQS message attribute trace propagation), MSK trace propagation specification (MSK record header trace propagation), and EC2 X-Ray tracing specification (EC2 X-Ray daemon). Each spec included implementation tasks, rollback procedures, and cost estimates.

SRE Platform v2.19.9 Release: I merged the log group discovery feature pull request and released SRE Observability Platform v2.19.9, incorporating the saved queries, log group URL utility, and alarm deep-link pipeline developed in Weeks 18–19.

By the end of Week 20, the fourth support rotation was complete with five active tickets resolved, the X-Ray specification suite was published covering all seven service categories, and v2.19.9 marked the completion of the CloudWatch Logs observability layer.

RPA Support Operational Detail: The MESO cash completeness incident for the Johansen fund illustrates the typical resolution workflow for stuck queue items. After verifying the database state and CloudWatch logs confirmed items were genuinely stuck (not still processing), the escalation to Mr. Leung for admin access took approximately 20 minutes. I marked items failed on the reporting server and in the MESO database and asked the business user (the Johansen team) to retrigger. The process ran to completion within 40 minutes of the retrigger. The Johansen team also requested standby assistance for downloading reports from S3 if they generated without saving to the expected location—I documented this edge case in the shift handover log for future rotations. The Mr. Jiannan RPA knowledge transfer session transferred the full MESO retrigger and Corporate Actions onboarding procedures to the incoming support team member for the September continuation period.

11.21 Week 21

Week 21 was the most delivery-dense week of the co-op placement's second half, focused on a comprehensive quality and reliability improvement program for the **Ask Citco RPA** LLM agent.

Architectural Refactor and Structlog Migration: I decomposed the 1,140-line monolithic `event_service/function.py` into eight single-responsibility modules following the Model-View-Controller pattern. I replaced all print statements and root-logger calls across eleven Lambda functions with structlog structured key=value logging. I added Google-style docstrings with parameter and return types to all public functions and removed all `sys.path.insert` hacks in favor of a properly packaged module structure.

handler_utils Decorators and Test Milestone: I built a shared `handler_utils` module providing three decorators: `@log_handler` (structured entry/exit logging with correlation IDs), `@scheduled_handler` (EventBridge schedule context extraction), and `@cfn_handler` (CloudFormation custom resource response handling). I applied these decorators to all eight

Lambda handlers, eliminating repeated boilerplate. I grew the test suite from 60% to 100% coverage (233 tests), achieving zero Ruff lint issues and zero Bandit security findings.

Structured CFN Nested Stack: I built a structured CloudFormation nested stack in parallel, decomposing the flat 600-line template into six sub-stacks: `cfn.storage` (S3, DynamoDB), `cfn.iam` (all roles and policies), `cfn.lambda` (function definitions), `cfn.bedrock` (agent and knowledge base), `cfn.ecs` (container task definitions), and `cfn.monitoring` (alarms and dashboards). This structure enabled independent stack updates and brought the deployment model in line with the `ia-it-sre-infrastructure-monitoring-amg` nested stack pattern.

SRE Naming Convention and Metrics Live: I renamed the SRE infrastructure repository service names, Lambda names, S3 bucket names, and IAM role names from the meridian prefix to the `ia-it-sre` convention. I confirmed the inventory API was publishing 597 CloudWatch metrics live across all six platforms in the DEV environment.

By the end of Week 21, the LLM agent had reached production-quality code standards (100% coverage, zero lint, zero security issues), the nested CFN architecture was established, and the SRE naming convention was applied platform-wide.

EventBridge vs APScheduler Decision: In a 59-minute architecture discussion with Mr. Leung, we resolved the scheduling mechanism for the `Æxeo` Report UI. We had considered APScheduler but it had caused memory contention issues in a prior project. We selected EventBridge Scheduler for its managed lifecycle (~\$1/month), stateless invocation model, and architectural consistency with the 5-minute alarm re-notification pattern already running in the AMG dashboard. In a follow-up IAM assistance session (110 minutes), I extracted the exact permissions required from the existing AMG dashboard CloudFormation execution role: `scheduler:CreateSchedule`, `scheduler:UpdateSchedule`, `scheduler>DeleteSchedule`, and `iam:PassRole` with the `lambda.amazonaws.com` trust policy. I aligned the ECS task CPU and memory to production values (512 CPU units / 1,024 MB) based on CloudWatch utilization data from the 93-day analysis window, replacing development-phase values that had remained in the template since initial scaffolding.

11.22 Week 22

Week 22 continued the RPA LLM Agent quality improvement program with knowledge base upload tooling, Bedrock agent externalization, and the FastAPI ECR image MVC refactor.

KB Content Upload CLI (`upload_kb_content.py`): I built a CLI tool to upload NAV Checklist PDFs to the Bedrock Knowledge Base S3 bucket. The tool was refactored into a `_kb_utils/` package with four modules (`auth`, `s3`, `sync`, `bucket`), an `--env` flag with production confirmation prompt, a status subcommand that discovers Knowledge Base and Data Source IDs by name, and 78 tests (48 package tests + 30 CLI tests). I enforced KMS encryption consistently across all Athena queries, closing an SSE inconsistency identified in the Week 21 audit.

Bedrock Agent Instruction Externalization: I externalized the Bedrock agent instruction from an inline CloudFormation parameter to a plain-text file (`data/bedrock-agent-instruction.txt`) read from S3 via a CloudFormation Custom Resource at deploy time, eliminating the need to redeploy the full stack for instruction tuning iterations. I added a status subcommand to discover active Knowledge Base and Data Source IDs from AWS by name pattern, removing the need to hard-code resource IDs in deployment scripts.

FastAPI ECR Image MVC Refactor: I applied a full MVC refactor to the `core-smart-rpa-llm-ecr-image` FastAPI container: I restructured the application into an `app/` package (`routers`, `models`, `services`, `dependencies`), replaced the custom logger with `structlog` logging, replaced manual environment variable reads with `pydantic-settings`, configured LDAP TLS properly, and locked CORS origins to the Citco domain list. I built and tested a hardened Docker container image.

CodeBuild Pipeline Stabilization: I resolved a series of CodeBuild pipeline failures across the week: `AWS_ACCOUNT_ID` retrieval via STS lookup, `DATA_BUCKET_NAME` injection into the build environment, YAML parser colon-in-string errors in the `buildspec`, and account ID resolution from `parameters/{env}.json`.

By the end of Week 22, the KB upload CLI was tested and functional, the Bedrock agent instruction was externalized from the CloudFormation template, and the FastAPI ECR image was hardened and refactored to production quality.

CodeBuild Pipeline Stabilization Challenges: I stabilized the CodeBuild pipeline through six hotfix iterations. I resolved the following issues in sequence: AWS_ACCOUNT_ID was not available as a built-in variable (I fixed this via STS lookup); the DATA_BUCKET_NAME environment variable was not injected into the build context (I fixed this by reading from parameters/{env}.json); YAML parser colon-in-string errors prevented the buildspec from parsing (I fixed this by quoting affected strings); and I read CodeAccountId from the parameters JSON file rather than querying it dynamically. I completed an ad-hoc manual report run for Mr. Deotale: I submitted five Æxeo report requests via the GET /arguments API endpoint using ReportID: 100502500, value date: 2026-08-10, and SourceSystem: RECON_EBINDER, covering TWOSIGMA, SIGMATWO, and TSCONSOL clients. I confirmed all five reports were delivered to the production S3 bucket.

11.23 Week 23

Week 23 was one of the most productive delivery weeks of the co-op placement, completing the RPA LLM Agent quality improvement sprint, instrumenting X-Ray tracing across the LLM agent, delivering Citco-branded alarm emails for the AMG platform, and making initial progress on the Enterprise Infrastructure Monitoring Dashboard.

280 Tests at 100% Coverage and ircoekb Fixes: I merged the MVC refactor branch to release, bringing the test suite to 280 tests at 100% coverage. I increased the ircoekb Lambda timeout from 300s to 900s based on the 93-day log analysis. I applied an SSL hotfix to skip certificate verification for internal Citco SDM endpoints (nosec B501 annotations added for the known internal trust exception). I archived five dead scripts into _archive/. I further externalized the Bedrock agent instruction, redesigned usage report emails as pivot-table HTML reports with Citco brand colors, resolved 32 cfn-lint warnings, and replaced eval calls with json.loads across the codebase. I implemented Pydantic validation models, tenacity exponential backoff, an SSM kill switch, and six CloudWatch alarms for all scheduled Lambda functions.

X-Ray Tracing Instrumentation: I applied Lambda Powertools @tracer.capture_lambda_handler and @tracer.capture_method decorators to all Lambda functions. Some functions that load S3 data at module import time required a cold-start fix—these cannot be instrumented at the class-method level without triggering before the tracer is initialized. I added a tracing.md

file documenting every decorated and undecorated function with rationale. I also authored the ECS ADOT sidecar X-Ray tracing spec and SQS trace header propagation spec for the ia_meso_process BitBucket repository, and a new ECS ADOT sidecar instrumentation spec for the core-smart-rpa-llm-ecr-image container.

Citco-Branded HTML Alarm Emails and Notifications CLI: I migrated alarm email delivery from SES to Citco SMTP using the TOIA-SMTP-Credentials Secrets Manager secret. I introduced file-based HTML templates with Citco branding and per-platform digest format. I implemented composite alarm re-notification with 5-minute cadence, with suppression logic preventing warning re-notifications when a critical sibling alarm is already active. I corrected IAM permissions: I added the missing cloudwatch:DescribeAlarms resource scope, removed stale SES permissions, and added kms:Decrypt for the SMTP credentials KMS key. I merged three feature branches (mapper decommission, composite re-notification, SMTP emails) to release and then to main.

Infrastructure Monitoring Dashboard: I added a dedicated GET /api/log-groups endpoint to the inventory API (log group observability feature Phase 2), enabling alarm emails to include per-resource log group deep-links. I created the initial commit for the Infrastructure Monitoring Dashboard as a new ia_infrastructure_dashboard BitBucket repository, with an @citco/core + @citco/theme migration plan documented to align the Enterprise Dashboard with Citco's component library.

By the end of Week 23, the RPA LLM Agent quality improvement program reached full production quality, X-Ray tracing was instrumented end-to-end in the LLM agent, Citco SMTP alarm emails were live, and three major AMG features were merged to main.

11.24 Week 24

Week 24 was the fifth and final RPA production support rotation, combined with structured knowledge transfer to the team, active frontend development on the Enterprise Dashboard, and planning for the Mitacs BSI extension.

RPA Production Support Rotation: The final rotation followed the standard North America shift pattern (11:00 AM–7:00 PM EST). I handled five SD tickets and three ad-hoc escalations across the week, with one ticket escalated to the database team.

Monday: I handled four items. For the Auto Rec Download SFTP config, I located the configuration file on the UiPath production share server, created a backup in the rollback folder, and updated the location to the new UCM path. For the FA Audit Pack (Davidson Kempener missing reports), I investigated via Confluence and confirmed the client is part of the “Big 19”—a designated client group for which the Realized-Unrealized report cannot be run by design. I informed the user of this constraint. For the Ops Transfers ticket (CPPIB file not picked up), I identified trailing whitespace in the Master tab that was preventing file detection, removed it, and confirmed the file was subsequently picked up by the scheduled trigger. I also handled three ad-hoc MESO runs for ARISAIG_PARTNERS and OPENDOOR funds that had been stuck in PENDING status for over six days—I marked all three as FAILED on the reporting server and emailed the affected teams to retrigger.

Tuesday: For the MESO blank reports ticket (after a production pipeline fix), I confirmed the fix commit was deployed and coordinated with Mr. Leung to handle the retrigger. The task was stuck in KILLED status—a state where the server treats the task as incomplete and blocks retrigger until someone manually marks it as FAILED in the MESO database first. Mr. Leung marked it FAILED and triggered the rerun, which completed successfully. I communicated the resolution to the requesting team. For the final ticket, my investigation revealed a database-related exception in the RPA bot logs. I documented my findings and emailed Mr. Jay with the logs and investigation steps; he confirmed and escalated to the DB team.

Thursday: I triggered the AVCAB queue with the -LOG suffix for SONA/PICTET Corporate Actions as per the standard procedure, and followed the FX Closeout trigger procedure for SONA. Both resolved without issue.

Friday—Æxeo Reports Incident: The most complex incident of the week involved scheduled reports for TWOSIGMA, SIGMATWO, and TSCONSOL clients. Early reports had delivered but the regular scheduled reports had not. Mr. Leung asked for my help and we worked the investigation together over a call. I had previously assisted Mr. Deotale with a similar case (triggering reports via Postman). Mr. Leung initially investigated through CloudWatch log analytics, searching by job ID in the MESO request database—an exhaustive log-scanning approach. I suggested narrowing the investigation via the Æxeo Report ID instead: on the reporting server, each triggered report

generates a unique ID that tracks exactly what happened to that specific report, rather than scanning the entire MESO process log. When I checked the reporting server using the Report ID, I found “no response from AExeo server”—the reports had failed because of a reporting server error, not a MESO pipeline issue. Mr. Leung confirmed this was the root cause and no further MESO investigation was needed.

To deliver the reports manually, Mr. Leung triggered them via the Postman API. Once the reporting server showed completion and MESO confirmed the runs were consolidated, the reports were available in the S3 bucket. I helped Mr. Leung download them and we attempted to transfer to the shared drive, but we were unable to retrieve the shared-drive password after several attempts. We instead delivered the reports to business users via Microsoft Teams (files were too large for email). Throughout the incident, I assisted with the RCA, delivery coordination, and follow-up communication with the business team.

Enterprise Dashboard and Inventory API Development: During non-support hours I continued active development across two repositories. In `ia-it-sre-infrastructure-inventory` I implemented two new observability API handlers for X-Ray service map data and CloudWatch Alarms, providing the backend endpoints that the Infrastructure Monitoring Dashboard frontend will consume. I also migrated the repository to uv virtual environments, pinned Python 3.14, and exported a `requirements.txt` for compatibility with the CodeCommit CI pipeline.

In `ml-rpa-status-llm-agent` I completed a four-phase Lambda handler standardization refactor: Phase 1 deleted four legacy shim handlers and updated CloudFormation Handler references; Phase 2 renamed `function.py` to the conventional `handler.py` and deleted two additional shims; Phase 3 extracted the Athena extraction logic and renamed the ALB and Athena handler modules for clarity; Phase 4 marked all tasks complete and staged the remaining `function.py` deletions. I also tested the GPT-5.6 Luna model on the DEV Bedrock agent and added a CORA process PDF to the Knowledge Base S3 bucket under `RPA Processes/CORA/`. I resolved a CloudFormation change set hang caused by a uv dependency group issue and a missing boto3 timeout on the change set wait call.

Mitacs BSI Extension Planning: Citco confirmed the Mitacs BSI renewal for September 2026. I prepared a scope document for Mr. Deotale covering four deferred workstreams: X-Ray tracing production enablement (currently DEV-only, pending UAT/PROD approval); log-group discovery API endpoint and Grafana panel integration (Phase 3 of the observability plan); the two Enterprise Dashboard pages; and RPA LLM Agent production hardening. By the end of Week 24, all handover documentation was live on Confluence and four workstreams were formally scoped for the extension.

11.25 Week 25

Week 25 marked the final day of the co-op placement, and I used it for structured operational handover and formal closure across all active workstreams.

Final Knowledge Transfer and Operational Handover: I held a formal handover session with Mr. Leung and Ms. Labos covering the full operational model for the infrastructure monitoring platform I had built. In the session, I walked through the dashboard operating model in detail: how to interpret the Health Overview row's color-coded stat tiles, when a CRITICAL alarm warrants immediate escalation versus self-heal investigation, and how to use the alarm state history panel to distinguish a transient spike from a sustained failure. I transferred ownership of the alarm runbooks, documenting per-service threshold rationale (for example, why MSK EstimatedMaxTimeLag is set to 10,000 messages rather than a lower value, and what the Meridian production incident peak of nine million messages implies about the operating range). I handed over the inventory API deployment guide covering stack ordering, environment promotion from DEV to UAT to PROD, and how to manually invalidate the S3 cache if a new resource appears between scheduled cache refresh cycles.

Final Technical Handover with Ms. Venkataraman and Mr. Vadla: I reviewed the Enterprise Dashboard BFF API contract with Ms. Venkataraman and Mr. Vadla, ensuring they understood the Inventory API endpoint schemas, the panel configuration JSON structure I had designed for the Infrastructure Monitoring Dashboard, and the @citco/core and @citco/theme component library migration plan I had documented. This alignment session ensured the portal team could continue

frontend development against the existing API without needing to reconstruct my design decisions independently.

By the end of Week 25, the full 900-hour co-op placement was complete. All deliverables were operational in production, handover documentation was live on Confluence, and I had formally scoped four workstreams for continuation in the September extension.

12. CONCLUSIONS

Throughout my co-op placement I worked toward the technical and non-technical goals established at the outset in consultation with Mr. Deotale and Dr. Pawan Lingras. The following sections evaluate each goal against the delivered outcomes and reflect on the broader impact of the engagement.

12.1 Learning Goals Evaluation

The learning goals established at the outset of my co-op placement included both technical and non-technical objectives. Table 11 provides an overview of each goal with its description and completion status.

Table 11: Learning Goals Overview

S.N.	Task	Description	Status
1	Master AWS Cloud Services	Develop proficiency in AWS services including Lambda, ECS, EC2, S3, CloudWatch, X-Ray, and Application Signals for infrastructure monitoring and observability.	✓
2	Apply SRE Principles	Learn and apply Site Reliability Engineering principles including the four golden signals, SLO definition, and incident response to improve system reliability.	✓

3	Infrastructure-as-Code	Gain hands-on experience with CloudFormation and SAM templates for automated, repeatable infrastructure provisioning across multiple environments.	✓
4	Observability Platform Development	Build comprehensive observability dashboards using AWS CloudWatch and Grafana AMG to provide real-time visibility into distributed systems.	✓
5	Proactive Reliability Engineering	Transition from reactive incident response to proactive monitoring with alarm-based detection that identifies issues before business impact.	✓
6	Python Development	Enhance Python programming skills through REST API development, comprehensive testing with pytest, and structured logging implementation.	✓
7	Software Engineering Practices	Apply modern software engineering practices including TDD, spec-driven development, code review, and CI/CD pipeline integration.	✓
8	AI-Augmented Engineering	Explore AI-assisted development workflows using Kiro IDE, Amazon Q, and Claude for architecture design and code generation.	✓
9	Agile Methodology	Practice Agile and Kanban methodologies including sprint planning, story decomposition, JIRA workflow management, and daily standups.	✓
10	Technical Communication	Develop technical writing skills through RCA reports, Confluence documentation, JIRA acceptance criteria, and formal project reporting.	✓

11	Production Incident Response	Gain experience in production support including incident triage, root cause analysis, and coordinated resolution across time zones.	✓
12	Cross-Functional Collaboration	Work effectively with distributed teams across Halifax and Hyderabad, participating in architecture sessions and knowledge transfer.	✓
13	Financial Domain Knowledge	Understand fund administration workflows, SLA requirements, and domain-specific terminology for financial services automation.	✓

All learning goals were achieved by the completion of my co-op placement. The deepest learning occurred at the intersection of engineering and operations: the direct experience of the Meridian Incident, followed immediately by building the instrumentation that would have detected it in under 30 seconds, compressed months of abstract SRE theory into a single, concrete engineering outcome.

12.2 Co-op Placement Hours Summary

Table 12: Co-op Placement Hours Summary

Work	Duration (hours)
Citco CTM Co-op Placement (Grafana dashboard, inventory API, RPA LLM agent, support rotations)	900
Report Writing (MCDA 5585 / 5586 Major Project Report)	120
Total	1020

The following chapters present the detailed evidence for each outcome: Chapter 13 covers deliverables and impact, Chapter 14 documents the engineering obstacles, and Chapter 15 outlines the planned next steps.

13. ACHIEVEMENTS & BUSINESS VALUE

Working on the IA-IT-SRE Infrastructure at Citco Technology Management, I achieved several significant milestones across observability platform development, API engineering, and production support operations. These accomplishments highlight my technical growth and the business value delivered during the co-op placement.

13.1 Key Achievements

- **CloudWatch Monitoring Platform:** Inventoried the complete AWS service fleet including Lambda functions, ECS clusters, and EC2 instances. Discovered and documented a critical tagging inconsistency affecting resource attribution. Developed a three-tier SQS fallback registry with comprehensive automated test coverage.
- **Grafana AMG Centralized Dashboard:** Designed and deployed a comprehensive observability workspace covering all six IA platforms. Implemented dynamic CloudWatch alarms via CloudFormation stacks with Lambda-backed provisioning. Successfully migrated the workspace through a breaking Grafana version upgrade.
- **Infrastructure Inventory REST API:** Built a production-ready Flask API with high test coverage. Implemented the Discoverer Registry pattern enabling extensible resource discovery. Integrated property-based testing with Hypothesis to verify tag disambiguation logic.
- **Container Insights Enablement:** Enabled ECS Container Insights across Meridian clusters, surfacing previously invisible task-level CPU and memory telemetry within hours of deployment.
- **Production Incident Response:** Completed multiple RPA support rotations, resolving notable production incidents. Delivered a critical CAIS Deadline hotfix within hours of initial alert, identifying a credential caching vulnerability during Secrets Manager rotation cycles.
- **Infrastructure-as-Code Proficiency:** Authored CloudFormation stacks with S3-staged templates and multi-environment reuse. Implemented a Lambda-backed dynamic provisioner that auto-discovers resources at deploy time.
- **Cross-Account IAM Architecture:** Designed a cross-account permission model enabling the Grafana workspace to query CloudWatch metrics across all IA platform AWS accounts.

- **SRE Principles Application:** Mapped the four golden signals (latency, traffic, errors, saturation) to concrete CloudWatch metric namespaces. Defined SLO targets for Lambda, ECS, MSK, and SQS services.
- **Test-Driven Development:** Applied TDD practices across all deliverables using pytest, achieving high coverage on the inventory API and SQS registry components.
- **AI-Augmented Engineering:** Leveraged Kiro IDE's spec-driven workflow for API development. Used Amazon Q for inline code assistance and Claude Sonnet for architecture reasoning.
- **Technical Documentation:** Produced RCA reports for production incidents, Confluence knowledge base articles, and comprehensive acceptance criteria for all deliverables.

13.2 Business Value

The shift from a purely reactive posture to a proactively monitored infrastructure is the central business value proposition of this co-op placement's work.

13.2.1 Incident Detection: From Hours to Seconds

The April 17, 2026 Meridian incident—in which approximately 9 million Kafka messages queued, producing a 7.5-hour processing delay affecting multiple client funds—was detected only because a fund administrator noticed missing outputs. No automated alarm existed.

The EstimatedMaxTimeLag alarm deployed as part of Centralized Amazon Managed Grafana (AMG) Dashboard directly addresses this gap:

- **Before:** Detection required human observation of missing downstream outputs, with latency of 2–3 hours when incidents began outside working hours.
- **After:** CloudWatch alarm fires within **30 seconds** of threshold breach, routing notification directly to the SRE team.

This improvement from multi-hour manual detection to sub-30-second automated alerting represents a qualitative step change in the team's ability to contain fund-impacting incidents before they reach business-level severity.

13.2.2 Operational Efficiency: Unified Fleet Visibility

Prior to this work, the IA-SRE team had no single view of all Lambda functions, ECS clusters, and EC2 instances under their operational mandate. Resource discovery was ad hoc and frequently lagged deployments by days or weeks.

The Grafana dashboard and inventory API together deliver:

- **Continuous visibility:** A single workspace replaces navigation across six separate AWS console tabs to assess platform health.
- **Machine-readable inventory:** The REST API enables downstream automation including tag compliance checks and dynamic alarm threshold recalibration.
- **Hidden issue discovery:** Container Insights immediately surfaced a previously invisible CPU saturation pattern on one of the Meridian Action Processor services.

13.2.3 Knowledge Transfer and Platform Resilience

Each support rotation produced documented incident summaries in the team's Confluence knowledge base. The CAIS hotfix analysis elevated a credential caching vulnerability to a platform-wide architectural concern, informing future RPA credential-handling guidelines and improving overall system resilience.

These achievements reflect my ability to deliver production-grade engineering solutions while contributing directly to the operational reliability of Citco's Intelligent Automation platform.

14. CHALLENGES & MITIGATIONS

Throughout this co-op placement, I encountered engineering challenges spanning AWS service limitations, Grafana platform constraints, corporate policy restrictions, and production incident responses. This chapter documents the most significant challenges that shaped delivery plans and produced lessons applicable to future SRE work.

14.1 AWS CloudWatch Limitations

CloudWatch imposes several hard limits encountered at modest operational scale (54 Lambda functions, 100+ dashboard panels):

- **Metrics Insights SQL 10 TPS limit:** With 90+ panels firing queries simultaneously, throttling caused intermittent panel errors. *Mitigation:* Collapsed rows by default, 5-minute refresh interval.
- **Log Insights 50 log group maximum:** The IA environment has 54+ Lambda log groups, exceeding the per-query limit. *Mitigation:* Filter to active log groups; separate saved query for full coverage.
- **SEARCH() visibility gap:** Functions without recent activity disappear from dashboards—the same root cause that made CAIS-Pricing invisible for 29 days. *Mitigation:* Replaced SEARCH() with Metric Insights SQL which enumerates all deployed functions.
- **No percentiles in GROUP BY mode:** P50/P99 statistics unavailable when grouping by dimension. *Mitigation:* Wildcard dimension queries for percentile panels; accepted tradeoff of missing idle functions.
- **51 KB CloudFormation dashboard limit:** Generated JSON exceeds inline CFN limits at scale. *Mitigation:* Python put_dashboard API deployment (no size limit).

14.2 Grafana Platform Constraints

Amazon Managed Grafana (AMG) with the CloudWatch plugin presented several limitations:

- **Mid-development version upgrade (10.4 → 12.4):** Breaking changes in query syntax required one full day of panel migration. *Lesson:* Design for portability across AMG versions.
- **Metrics Insights SQL no regex in WHERE clause:** The WHERE clause supports only LIKE patterns with simple wildcards, not regular expressions. Tag-based filtering (WHERE tag.AppName = 'MERIDIAN') is also unsupported. *Mitigation:* Implemented dashboard-level variables (\$platform, \$environment) with Infinity datasource backed by static inventory JSON files.
- **Dashboard freezing from excessive datapoints:** Before dashboard-level variables were introduced, panels rendered all resources simultaneously—54+ Lambda functions, 17+ ALBs, 30+ ECS services. The browser froze for 10–15 seconds on load as Grafana attempted to render thousands of data points across 113 panels. *Mitigation:* Platform variable filters reduced visible resources per dashboard load; collapsed rows defer queries until user expansion.

- **Single SQL query per panel:** Cannot combine multiple Metrics Insights queries on one panel—attempting to add a second query produces “Maximum number of queries exceeded” error. *Mitigation:* Split into separate panels (e.g., “Running Tasks” and “Desired Tasks” instead of one combined panel).
- **No percentiles in Metrics Insights SQL:** Aggregation functions like PERCENTILE(Duration, 50) are not supported. Percentile statistics (p50, p99) are only available via the standard Metric Search API, which lacks GROUP BY capability. *Tradeoff:* Accept either per-function grouping (Metrics Insights, no percentiles) or percentiles (Metric Search, only active functions visible).
- **No cross-panel filtering:** Clicking a resource in one panel does not filter other panels. *Mitigation:* Dashboard-level template variables with coordinated queries across all panels.
- **IAM credential rotation:** Monthly rotation breaks AMG access until trust policy is manually updated. *Mitigation:* Documented runbook with calendar reminders.

14.3 Infrastructure & Corporate Policy Challenges

Enterprise environment constraints required adaptation of standard AWS patterns:

- **Static metrics at deploy time:** Dashboard does not auto-discover new resources. *Mitigation:* Documented redeploy procedure with 3-tier fallback.
- **Lambda Insights layer stripped on redeploy:** Other teams’ deployments remove the monitoring layer. *Mitigation:* Periodic enforcement script (add_lambda_insights.py).
- **ECS Fargate no free cluster metrics:** Requires Container Insights at \$0.30/metric/month. *Mitigation:* Manual per-cluster enablement with cost approval.
- **CloudFormation ResourceExistenceCheck hook:** Corporate guardrail blocked alarm stack deployment. *Mitigation:* Dynamic Lambda provisioner bypasses CFN constraints.
- **IAM permission boundaries:** Subscription filter approach required permissions outside team scope. *Mitigation:* Redesigned to use Metric Insights SQL instead.
- **S3 encryption policy:** Silent failures without ServerSideEncryption=AES256. *Mitigation:* Pre-commit hook to flag missing encryption headers.

14.4 Production Incidents

Production incidents provided valuable learning experiences:

14.4.1 CAIS Deadline Pipeline Hotfix (June 2026)

The CAIS Deadline pipeline — responsible for uploading allocation sheets to the PDS database and generating pricing reports for alternative investment clients — failed silently: there were no error notifications and no data was delivered. I delivered a 3-part hotfix:

1. **UCM credential expiry:** SMB mount returned empty data on expired credentials without error. *Fix:* Explicit validation step verifying expected files exist.
2. **Empty list SQL error:** WHERE id IN () generated invalid SQL. *Fix:* Empty list guard before query construction.
3. **Module-level credential caching:** Stale credentials persisted across Secrets Manager rotation. *Fix:* Fresh credential retrieval before each use.

Key lesson: Silent failures are worse than loud failures. Validate expected state, not just absence of exceptions.

14.4.2 RPA LLM Agent Lambda Memory Exhaustion (August 2026)

Business users reported the RPA Status Bot intermittently failing to respond to queries. CloudWatch logs revealed out-of-memory (OOM) errors on the `rpa_llm_agent_event_service_cicd` Lambda function, which handles Bedrock Agent API calls for process status lookups.

Root cause: The Lambda was using CloudFormation's implicit default of 128 MB memory. Functions using pandas, boto3, and JSON serialization together exceeded this allocation during peak query loads.

Fix: Increased MemorySize from 128 MB to 256 MB in the CloudFormation template. Additionally standardized all nine Lambda functions in the RPA LLM stack to explicit 256 MB allocations, making memory settings visible and intentional in Infrastructure-as-Code rather than relying on implicit defaults.

Key lesson: Never rely on CloudFormation default values for production workloads. Explicit resource allocations prevent silent degradation and make capacity constraints visible in code review.

14.5 Summary

Key themes from these challenges:

1. **AWS service limits are real constraints** encountered at modest scale—not edge cases. The 10 TPS Metrics Insights throttle affected dashboard load with only 90 panels.
2. **Silent failures are dangerous:** Multiple incidents involved operations that “succeeded” but produced no useful output.
3. **Enterprise environments add complexity:** Corporate policies break standard AWS patterns; budget time for adaptations.
4. **Managed services still require operational planning:** AMG’s “serverless” nature does not eliminate IAM credential lifecycle overhead or CloudWatch API throttling mitigation.
5. **Query language limitations shape architecture:** Metrics Insights SQL’s lack of regex filtering and percentile aggregation required alternative approaches—dashboard variables backed by static inventory files and separate percentile panels using the legacy Metric Search API.

These themes inform the recommendations in Chapter 15.

15. FUTURE WORK

The work I delivered in this co-op placement establishes a monitoring foundation — not merely an endpoint. I have already defined several planned extensions in the JIRA backlog, scoped for implementation during the Mitacs BSI renewal (September 1 – December 31, 2026).

15.1 Distributed Tracing with X-Ray

The most technically compelling near-term extension is the integration of AWS X-Ray distributed tracing into the IA Lambda fleet using the `aws-lambda-powertools[tracer]` library. X-Ray would provide end-to-end request traces across Lambda invocations, SQS message dispatch, and downstream HTTP calls — the “service map” view that CloudWatch metrics alone cannot produce. At the 5% sampling rate recommended for production workloads, the additional cost is estimated at under \$2 per month for the current fleet size, making this an unusually cost-effective observability enhancement. A Grafana X-Ray data source plugin is available for AMG, meaning trace data would flow directly into the existing workspace without requiring a separate visualization tool.

15.2 Auto-Healing Agent

The alarm infrastructure I delivered closes the detection gap — the 2–3 hour window between failure onset and SRE awareness that the Meridian Incident exposed. But detection without automated remediation still leaves a response gap: a human must receive the SNS notification, triage the alarm, identify the correct recovery action, and execute it. For the most common and well-understood failure modes on the IA platform, this human step is unnecessary. The **auto-healing agent** eliminates it.

15.2.1 Concept

The auto-healing agent is an event-driven remediation layer that sits between the CloudWatch alarm infrastructure and the AWS services being monitored. Each alarm SNS message triggers a Lambda “healer” function. The healer evaluates the alarm payload, classifies the failure type, selects the appropriate recovery action from a configurable remediation catalog, and executes it — logging the entire sequence via the IA Event Store with structured `PROCESS_EVENT` codes so that every automated intervention is fully auditable.

15.2.2 Architecture

The proposed architecture connects directly to the existing dashboard alarm infrastructure without requiring changes to the Grafana dashboard or CloudFormation alarm stacks:

1. A CloudWatch Alarm breaches its threshold (e.g., ECS task desired count $\neq$ running count on a Meridian service).
2. The alarm fires an SNS message to the `ia-sre-auto-healing-topic` SNS topic (a new topic, parallel to the existing notification topic).
3. The SNS topic triggers the **Healer Lambda**, which receives the full alarm payload including the `AlarmName`, `NewStateValue`, `MetricName`, and any available dimensions (`ClusterName`, `ServiceName`, `FunctionName`).
4. The Healer Lambda looks up the alarm name in a **remediation catalog** — a DynamoDB table mapping alarm patterns to recovery action functions.
5. The matched recovery action is invoked: an AWS API call, a Step Functions state machine, or a queued message to a downstream remediation worker.

6. The Healer Lambda logs the intervention to the IA Event Store with a PROJ-BE-AUTOHEALING code, including the alarm name, recovery action taken, and outcome status.
7. If the recovery action fails or the alarm re-fires within a configurable cool-down window, the Healer escalates by publishing to the existing notification SNS topic to alert the SRE team directly.

15.2.3 Remediation Catalog (Initial Scope)

The first phase of the auto-healing agent targets the failure modes observed or anticipated from the four support rotation weeks:

Table 13: Auto-Healing Agent — Initial Remediation Catalog

Alarm / Trigger	Recovery Action	Escalation if Fails
ECS task desired $\neq$ running	ecs:UpdateService (force new deployment)	SNS alert to SRE
MSK consumer lag > 10,000 messages	Scale ECS consumer service (+1 task)	SNS + JIRA ticket
Lambda error rate > 1% over 5 min	Trigger Step Functions diagnostic workflow; capture last 50 error log lines to S3	SNS alert with S3 log link
SQS queue age > 1 hour	Invoke queue-processor Lambda with force=true parameter	SNS alert
ECS CPU > 80% sustained 5 min	Trigger ECS service auto-scaling policy	SNS if at max tasks
RPA bot missing heartbeat	Call UiPath Orchestrator API to restart robot job	SNS + support ticket

15.2.4 Design Principles

Three design principles constrain the auto-healing agent to avoid introducing new failure modes:

- **Idempotency.** Every recovery action must be safe to invoke multiple times. The DynamoDB remediation catalog tracks the last invocation time per alarm; the Healer Lambda checks this before acting and skips duplicate invocations within the cool-down window.
- **Bounded autonomy.** The Healer acts on a whitelist of known-safe recovery actions. Any alarm with no matching catalog entry falls through to standard SNS notification — the agent does not improvise. New recovery actions are added only after review and approval, following the same PR process as CloudFormation template changes.

- **Full auditability.** Every automated intervention is logged to the IA Event Store with a structured event code (PROJ-I-AUTOHEALING), the alarm trigger name, the action executed, the outcome status, and a link to the AWS resource modified. The Event Store trail ensures that automated interventions are visible in the same audit log as manual ones.

15.2.5 Relation to the PoC

The auto-healing agent is the direct successor to the alarm-and-notify posture delivered by this co-op placement. The detection layer (CloudWatch alarms + SNS) was built first because a remediation layer without a reliable detection layer is fragile — automated actions triggered by false alarms cause more harm than the failures they prevent. Now that the detection infrastructure has been in production for several months, the signal quality of the alarms is validated, and the false-positive rate is well-characterized. The auto-healing agent can therefore be implemented against a tested alarm baseline rather than a hypothetical one.

This progression — detect first, then automate remediation — is the correct sequence for building reliable auto-healing in a regulated financial services environment, where an incorrectly triggered service restart during a VPL pricing window or a CAIS Deadline processing run would itself constitute a production incident.

15.3 RPA Bot CloudWatch Integration

Two follow-on stories in my JIRA backlog propose bridging the Blue Prism (BPM) and UiPath Cloud operational metrics into CloudWatch via a polling Lambda function. The Lambda would query the Blue Prism API and UiPath Orchestrator API on a configurable schedule, emit custom CloudWatch metrics for queue depth, bot session count, and process error rates, and feed those metrics into the existing Grafana dashboard infrastructure. This would close the last remaining visibility gap: the current dashboard covers the AWS infrastructure layer comprehensively but has no visibility into the RPA execution layer above it.

15.4 Error Budget Burn-Rate Alerting

The current alarm model fires on threshold breaches without distinguishing between a momentary spike and a sustained reliability regression. Multi-window burn-rate alerting — as described in the Google SRE Workbook — uses 1-hour and 6-hour Metric Math expressions

to detect when the error budget is being consumed faster than the replenishment rate. Implementing this pattern in CloudWatch Metric Math would elevate the existing alarm infrastructure from threshold-based detection to SLO-aware alerting, enabling the team to distinguish incidents that require immediate response from those that are within the acceptable error budget envelope.

15.5 Tag Compliance Automation

The AppName tagging obligation I identified in Section 14.3 currently relies on a reactive CloudWatch alarm: the alarm fires after a non-compliant resource has already been deployed. A proactive alternative would query the inventory API on a schedule, compare the returned resource list against an expected tag manifest, and open a JIRA ticket automatically for any resource found to be non-compliant. This would shift tag compliance from a detection activity to a governance workflow.

A. APPENDIX: WORK LOGS

The complete weekly work log documenting **900 hours** of internship work across 25 weeks (March 16 – August 31, 2026) is provided in the supplementary file:

BhavikBhagat_A00494758_WorkLogs.xlsx

The work log includes:

- Weekly task descriptions and hours logged per activity
- Weekly hour totals and cumulative running total
- Meeting and support shift hours tracked separately
- Work stream categorization (WS1–WS6)

An additional **120 hours** were dedicated to report writing and academic documentation (MCDA 5585 and 5586), bringing the total project effort to **1,020 hours**.

REFERENCES

- [1] Amazon Web Services. *Amazon Bedrock Agents Documentation*. 2026. URL: <https://docs.aws.amazon.com/bedrock/latest/userguide/agents.html> (visited on 08/23/2026).

- [2] Amazon Web Services. *Amazon Bedrock Documentation*. 2026. URL: <https://aws.amazon.com/bedrock/> (visited on 08/23/2026).
- [3] Amazon Web Services. *Amazon Bedrock Knowledge Bases Documentation*. 2026. URL: <https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html> (visited on 08/23/2026).
- [4] Amazon Web Services. *Amazon CloudWatch Documentation*. 2026. URL: <https://aws.amazon.com/cloudwatch/> (visited on 08/23/2026).
- [5] Amazon Web Services. *Amazon ECS Documentation*. 2026. URL: <https://aws.amazon.com/ecs/> (visited on 08/23/2026).
- [6] Amazon Web Services. *Amazon Managed Grafana Documentation*. 2026. URL: <https://aws.amazon.com/grafana/> (visited on 08/23/2026).
- [7] Amazon Web Services. *Amazon OpenSearch Service Documentation*. 2026. URL: <https://aws.amazon.com/opensearch-service/> (visited on 08/23/2026).
- [8] Amazon Web Services. *Amazon SNS Documentation*. 2026. URL: <https://aws.amazon.com/sns/> (visited on 08/23/2026).
- [9] Amazon Web Services. *Amazon SQS Documentation*. 2026. URL: <https://aws.amazon.com/sqs/> (visited on 08/23/2026).
- [10] Amazon Web Services. *AWS CloudFormation Documentation*. 2026. URL: <https://aws.amazon.com/cloudformation/> (visited on 08/23/2026).
- [11] Amazon Web Services. *AWS CodeCommit Documentation*. 2026. URL: <https://aws.amazon.com/codecommit/> (visited on 08/23/2026).
- [12] Amazon Web Services. *AWS CodePipeline Documentation*. 2026. URL: <https://aws.amazon.com/codepipeline/> (visited on 08/23/2026).
- [13] Amazon Web Services. *AWS Distro for OpenTelemetry (ADOT)*. 2026. URL: <https://aws-otel.github.io/> (visited on 08/23/2026).
- [14] Amazon Web Services. *AWS Lambda Documentation*. 2026. URL: <https://aws.amazon.com/lambda/> (visited on 08/23/2026).
- [15] Amazon Web Services. *AWS Lambda Powertools for Python*. 2026. URL: <https://docs.powertools.aws.dev/lambda/python/> (visited on 08/23/2026).

- [16] Amazon Web Services. *AWS X-Ray Documentation*. 2026. URL: <https://aws.amazon.com/xray/> (visited on 08/23/2026).
- [17] Amazon Web Services. *CloudWatch Logs Insights Query Syntax*. 2026. URL: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/AnalyzingLogData.html> (visited on 08/23/2026).
- [18] Amazon Web Services. *CloudWatch Metrics Insights SQL Syntax*. 2026. URL: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/cloudwatch-metrics-insights-syntax.html> (visited on 08/23/2026).
- [19] Amazon Web Services. *Kiro: AI-Powered Development Environment*. 2026. URL: <https://kiro.dev/> (visited on 08/23/2026).
- [20] Amazon Web Services. *Office Hours: Grafana Setup on AWS | Amazon Managed Grafana*. 2026. URL: <https://www.youtube.com/watch?v=3lGMahpDYmA> (visited on 08/23/2026).
- [21] Amazon Web Services. *Set Up AWS X-Ray as a Data Source Plugin for Amazon Managed Grafana*. 2026. URL: <https://www.youtube.com/watch?v=4C94AtJGcxk> (visited on 08/23/2026).
- [22] Anthropic. *Claude: AI Assistant*. 2026. URL: <https://www.anthropic.com/claude> (visited on 08/23/2026).
- [23] Astral. *Ruff: Python Linter and Formatter*. 2026. URL: <https://docs.astral.sh/ruff/> (visited on 08/23/2026).
- [24] Astral. *uv: Python Package and Project Manager*. 2026. URL: <https://docs.astral.sh/uv/> (visited on 08/23/2026).
- [25] Atlassian. *Bitbucket: Git Code Management*. 2026. URL: <https://bitbucket.org/> (visited on 08/23/2026).
- [26] Atlassian. *Confluence: Team Collaboration Software*. 2026. URL: <https://www.atlassian.com/software/confluence> (visited on 08/23/2026).
- [27] Atlassian. *Jira: Project Management Software*. 2026. URL: <https://www.atlassian.com/software/jira> (visited on 08/23/2026).
- [28] Grafana Labs. *Grafana: Open Observability Platform*. 2026. URL: <https://grafana.com/> (visited on 08/23/2026).

- [29] Hynek Schlawack. *structlog: Structured Logging for Python*. 2026. URL: <https://www.structlog.org/> (visited on 08/23/2026).
- [30] LibreChat. *LibreChat: Open-Source AI Chat Platform*. 2026. URL: <https://www.librechat.ai/> (visited on 08/23/2026).
- [31] Pydantic. *Pydantic: Data Validation Library*. 2026. URL: <https://docs.pydantic.dev/> (visited on 08/23/2026).
- [32] pytest Development Team. *pytest: Python Testing Framework*. 2026. URL: <https://docs.pytest.org/> (visited on 08/23/2026).
- [33] Python Software Foundation. *Python Programming Language*. 2026. URL: <https://www.python.org/> (visited on 08/23/2026).
- [34] Sebastián Ramírez. *FastAPI: Modern Python Web Framework*. 2026. URL: <https://fastapi.tiangolo.com/> (visited on 08/23/2026).